

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ФАКУЛЬТЕТ ПРИКЛАДНОЙ МАТЕМАТИКИ И ИНФОРМАТИКИ
СПЕЦИАЛЬНЫЙ ФАКУЛЬТЕТ БИЗНЕСА И ИНФОРМАЦИОННЫХ
ТЕХНОЛОГИЙ



А.А. Безверхий
С.И. Кашкевич

ВВЕДЕНИЕ В ОПЕРАЦИОННЫЕ СИСТЕМЫ

Учебное пособие

Для студентов специальностей
«Экономическая кибернетика», «Актuarная математика»,
«Компьютерная безопасность»

Минск
УП «ИВЦ Минфина»
2004

УДК 681.3.06 (075.8)
ББК 32.973.26-018.2я73
Б-39

Рецензенты

заведующий кафедрой дискретной математики
и алгоритмики Белгосуниверситета,
доцент *В.М. Котов*,
доцент кафедры математического
обеспечения ЭВМ Белгосуниверситета *Л.В. Певзнер*

*Печатается по решению Ученого совета
факультета прикладной математики и информатики
Белорусского государственного университета
протокол № 5 от 27 апреля 2004 г.*

Б-39 Введение в операционные системы: Учеб. пособие / А.А. Безверхий,
С.И. Кашкевич. – Мн.: УП «ИВИЦ Минфина», 2004. 168 с.

ISBN 985-6648-60-2

В учебном пособии изложены основные положения теории операционных систем. Пособие содержит как обсуждение фундаментальных принципов построения, так и анализ основных особенностей современных ОС.

Книга адресована студентам, изучающим основы теории операционных систем, а также всем тем, кто интересуется вопросами построения и функционирования современных операционных систем.

УДК 681.3.062 (075.8)
ББК 32.973.26-018.2я73

ISBN 985-6648-60-2

© Безверхий А.А., Кашкевич С.И., 2004
© Республиканское унитарное предприятие
«Информационно-вычислительный центр
Министерства финансов Республики Беларусь», 2004

Введение

Назначение любой системы обработки данных, в том числе и электронных систем, состоит в преобразовании данных. Для достижения этих целей в электронных системах обработки данных используется сочетание аппаратных средств, программного обеспечения и собственно хранимых данных. Некоторые из названных ресурсов стоят достаточно дорого, поэтому важно, чтобы они использовались максимально эффективно. Создание операционных систем мотивировалось именно этой целью – повышением эффективности систем обработки данных.

К настоящему времени операционные системы прошли в своем развитии большой путь от простейших загрузчиков и библиотек ввода-вывода до сложных многофункциональных программ, управляющих всеми аспектами вычислительного процесса. Авторы попытались проследить эволюцию операционных систем.

В книге последовательно излагаются основные этапы развития операционных систем, их структурная организация, концепции и механизмы управления процессором, памятью и внешними устройствами, рассматриваются вопросы взаимодействия и синхронизации вычислительных процессов, обсуждаются особенности работы сетевых ОС, а также проблемы безопасности операционных систем.

Настоящая книга должна дать читателю основы понимания того, что такое операционная система и как она работает. Поэтому изложение носит характер вводного курса в теорию операционных систем. В первую очередь в рамках книги ставятся и описываются задачи, решаемые в процессе функционирования ОС, и гораздо меньшее внимание уделяется конкретным решениям поставленных задач.

Тем не менее, наряду с теоретическими сведениями, необходимыми для понимания принципов работы вычислительных систем, в книге рассмотрены примеры реализации этих принципов для различных современных операционных систем.

Книга адресована прежде всего студентам и аспирантам различных направлений специальностей «Экономическая кибернетика», «Актuarная математики», «Компьютерная безопасность» и др. как учебное пособие по курсу «Операционные системы». В основу книги положен многолетний опыт авторов по преподаванию указанного курса на кафедре математического обеспечения АСУ факультета прикладной математики и информатики Белорусского государственного университета. Книга может быть полезна и для специалистов в области системного программирования, которым необходимы знания принципов работы ОС.

Авторы выражают благодарность декану факультета прикладной математики и информатики Белгосуниверситета Павлу Алексеевичу Мандрику, декану спецфакультета бизнеса и информационных технологий Белгосуниверситета Валерию Николаевичу Шалиме за неоценимую помощь и поддержку в подготовке и издании учебного пособия, а также заведующему кафедрой математического обеспечения АСУ факультета прикладной математики и информатики Белгосуниверситета Виктору Владимировичу Краснопрошину и своим коллегам за полезные замечания и предложения, высказанные ими при обсуждении этой книги.

Все замечания и пожелания по поводу этой книги будут приняты авторами с благодарностью.

Глава 1. Основные понятия

1.1. Определение и структура операционных систем

Операционная система (ОС) – это комплекс управляющих и обрабатывающих программ, который предназначен для:

- наиболее эффективного использования ресурсов вычислительной системы и организации эффективных приложений;
- организации интерфейса между аппаратурой компьютера и пользователями с их задачами, образуя т.н. операционную среду.

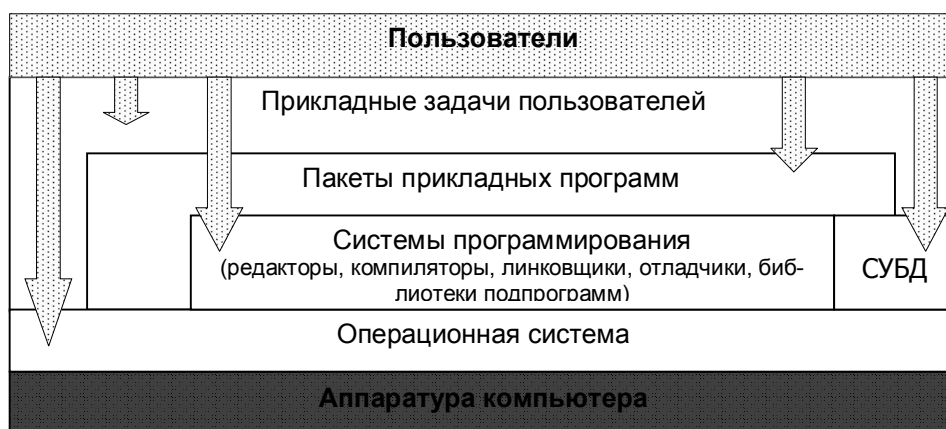


Рис. 1.1. Взаимодействие между различными компонентами вычислительной системы.

На рис.1.1 изображена упрощенная схема взаимодействия различных компонентов программного обеспечения, а также конечных пользователей. Каждая группа пользователей получает доступ только к компонентам, лежащим непосредственно под ней. Таким образом, ОС является универсальной оболочкой, с которой взаимодействуют все остальные компоненты.

В составе операционных систем можно выделить несколько основных структурных компонент:

1. **Управляющие программы (ядро ОС)**, которые выполняют следующие функции:

- идентификация всех программ и данных;
- планирование и диспетчеризация задач в соответствии заданными стратегиями и приоритетами;

- загрузка в оперативную память подлежащих выполнению программ, инициация программы (передача ей управления, в результате чего процессор исполняет программу);
- обеспечение режима мультипрограммирования, т.е. выполнение двух и более программ на одном процессоре, что создает иллюзию их одновременного исполнения;
- обеспечение функций по организации и исполнению всех операций ввода-вывода;
- распределение памяти, организация виртуальной памяти;
- организация обмена сообщениями и данными между выполняющимися программами;
- защита одних программ и данных от влияния других;
- разрешение конфликтов между одновременно выполняющимися программами;
- предоставление услуг на случай сбоя системы.

2. Системы управления файлами (файловые системы). Термин «файловые системы» в настоящее время употребляется в нескольких контекстах: принципы организации данных на внешних носителях на логическом уровне; реализация этих принципов на физическом уровне; наконец, программное обеспечение для реализации этих принципов. Как правило, все современные операционные системы имеют соответствующие системы управления файлами. Более того, ряд операционных систем позволяет работать с несколькими различными файловыми системами (например, Windows 2000 может поддерживать как FAT, так и NTFS). Верно и обратное утверждение: одна файловая система (речь здесь, конечно же, идет о принципах реализации, а не о программных модулях) может использоваться в рамках различных операционных систем. Примером такой файловой системы является FAT и ее различные модификации.

3. Модули организации интерфейса между операционной системой и другими программами, а также интерфейса с пользователями-людьми. Для взаимодействия различных компонент программного обеспечения с операционной системой в настоящее время используется механизм API (Application Program Interface) – комплекс описаний, системных переменных и подпрограмм для взаимодействия прикладных программ с операционной системой и друг с другом. Интерфейс API включает в себя управление процессами, памятью и вводом/выводом. Программист может при разработке собственных продуктов как пользоваться возможностями API напрямую, так и использовать специальные возможности, заложенные в системах программирования.

Что касается взаимодействия операционных систем с пользователями – людьми, то современные ОС предлагают для этого множество различных средств, в зависимости от квалификации пользователей. Прежде всего следу-

ет сказать о *языке команд*, присущем подавляющему большинству ОС. Этот язык предназначен для квалифицированных пользователей (разработчики программного обеспечения, администраторы и т.д.). Команды ОС могут подразделяться на *внутренние* и *внешние*. Внутренние команды обрабатываются системными модулями ОС, внешние команды служат для запуска любой программы (как входящей, так и не входящей в состав операционной системы) и позволяют передать ей необходимые параметры для работы. Кроме того, в состав командного языка могут входить специальные команды управления программным потоком (ветвления, циклы, переходы и т.д.), что дает возможность говорить о языке команд ОС как о простом языке программирования и писать программы на нем (пример – пакетные файлы в MS-DOS и Windows).

Для конечных пользователей (т.е. для тех, кто использует компьютер только как инструмент для решения своих прикладных задач) использование языка команд может оказаться неудобным. Для них могут использоваться дополнительные интерфейсные оболочки. Основное назначение таких оболочек – либо расширить возможности по управлению ОС, либо изменить встроенные в систему возможности. Интерфейсные оболочки могут входить в состав операционных систем (как, например, `explorer.exe` в Windows), а могут быть разработаны в дополнение к существующим модулям (KDE или GNOME для системы Linux) или для их замещения.

Следует сказать, что ряд современных операционных систем позволяет выполнить *эмуляцию* других ОС. Так, с помощью системы WMWARE можно в среде Linux запустить другую ОС, например, Windows.

4. Системные утилиты с точки зрения ядра операционных систем представляют собой обычные прикладные программы, но они используются для системных целей. Так, с их помощью можно подготавливать для работы носители данных, выполнять перекодирование данных, оптимизацию размещения данных на носителях, выполнять тестирование аппаратных компонент компьютера и выполнять целый ряд других работ, связанных с обслуживанием вычислительной системы.

1.2. Классификация операционных систем

Широко известно высказывание, согласно которому любая наука начинается с классификации. Само собой, вариантов классификации может быть очень много, все будет зависеть от выбранного признака, по которому один объект следует отличать от другого. Операционные системы могут различаться особенностями реализации внутренних алгоритмов управления основными ресурсами компьютера (процессорами, памятью, устройствами), особенностями использованных методов проектирования, типами аппаратных платформ, областями использования и многими другими свойствами. Тем не

менее для операционных систем достаточно давно сформировалось относительно небольшое число классификаций.

1. *По назначению* различают ОС *общего* и *специального* назначения (для переносимых компьютеров, различных встроенных систем, сетевые ОС, ОС реального времени, ОС для организации и ведения баз данных, распределенные ОС и т.д.).

2. *По режиму обработки задач* различают ОС, обеспечивающие *однопрограммный* и *мультипрограммный* режимы. Под мультипрограммированием понимается такой способ организации вычислений, когда на однопроцессорной вычислительной системе создается видимость одновременного выполнения нескольких программ. Любая задержка в решении программы (например, для организации ввода/вывода данных) используется для выполнения других программ. При этом параллельно выполняющиеся программы могут взаимодействовать между собой и конкурировать за различные ресурсы вычислительной системы. Распараллеливание возможно реализовать и в рамках одной программы, разбивая ее на несколько *нитей* или *тредов*. Но в этом случае забота о параллельном выполнении и взаимодействии нескольких тредов одного приложения ложится на плечи прикладных программистов. В этом случае говорят о *мультизадачном* режиме работы операционных систем.

Многозадачные ОС подразделяются на три типа в соответствии с использованными при их разработке критериями эффективности:

- системы пакетной обработки;
- системы разделения времени;
- системы реального времени.

Системы пакетной обработки предназначены для решения задач в основном вычислительного характера, не требующих быстрого получения результатов. Главной целью и критерием эффективности систем пакетной обработки является максимальная пропускная способность, то есть решение максимального числа задач в единицу времени. Для достижения этой цели в системах пакетной обработки используются следующая схема функционирования: в начале работы формируется пакет заданий, каждое задание которого содержит требования к системным ресурсам. Из этого пакета заданий формируется мультипрограммная смесь, то есть множество одновременно выполняемых задач. Для одновременного выполнения выбираются задачи, предъявляющие отличающиеся требования к ресурсам, так, чтобы обеспечивалась сбалансированная загрузка всех устройств вычислительной машины; так, например, в мультипрограммной смеси желательно одновременное присутствие вычислительных задач и задач с интенсивным вводом-выводом. Таким образом, выбор нового задания из пакета заданий зависит от внутренней ситуации, складывающейся в системе, то есть выбирается «выгодное» задание.

Следовательно, в таких ОС невозможно гарантировать выполнение того или иного задания в течение определенного периода времени.

В системах пакетной обработки переключение процессора с выполнения одной задачи на выполнение другой происходит только в случае, если активная задача сама отказывается от процессора, например, из-за необходимости выполнить операцию ввода-вывода. Поэтому одна задача может надолго занять процессор, что делает невозможным выполнение интерактивных задач. Таким образом, взаимодействие пользователя с вычислительной машиной, на которой установлена система пакетной обработки, сводится к тому, что он приносит задание, отдает его диспетчеру-оператору, а в конце дня после выполнения всего пакета заданий получает результат. Очевидно, что такой порядок снижает эффективность работы пользователя.

Системы разделения времени призваны исправить основной недостаток систем пакетной обработки - изоляцию пользователя-программиста от процесса выполнения его задач. Каждому пользователю системы разделения времени предоставляется терминал, с которого он может вести диалог со своей программой. Так как в системах разделения времени каждой задаче выделяется только квант процессорного времени, ни одна задача не занимает процессор надолго, и время ответа оказывается приемлемым. Если квант выбран достаточно небольшим, то у всех пользователей, одновременно работающих на одной и той же машине, складывается впечатление, что каждый из них единолично использует машину.

Ясно, что системы разделения времени обладают меньшей пропускной способностью, чем системы пакетной обработки, так как на выполнение принимается каждая запущенная пользователем задача, а не та, которая "выгодна" системе, и, кроме того, имеются накладные расходы вычислительной мощности на более частое переключение процессора с задачи на задачу. Критерием эффективности систем разделения времени является не максимальная пропускная способность, а удобство и эффективность работы пользователя.

Системы реального времени применяются для управления различными техническими объектами и процессами. В этих системах существует предельно допустимое время, в течение которого должна быть выполнена та или иная программа, управляющая объектом. Критерием эффективности для систем реального времени является их способность выдерживать заранее заданные интервалы времени между запуском программы и получением результата. Это время называется временем реакции системы, а соответствующее свойство системы - реактивностью. Для этих систем мультипрограммная смесь представляет собой фиксированный набор заранее разработанных программ, а выбор программы на выполнение осуществляется исходя из текущего состояния объекта или в соответствии с расписанием плановых работ.

Некоторые операционные системы могут совмещать в себе свойства систем разных типов, например, часть прикладных задач может выполняться в

режиме пакетной обработки, а часть - в режиме реального времени или в режиме разделения времени. В таких случаях режим пакетной обработки часто называют фоновым режимом.

Важнейшим разделяемым ресурсом является процессорное время. Способ распределения процессорного времени между несколькими одновременно существующими в системе процессами (или нитями) во многом определяет специфику ОС. Среди множества существующих вариантов реализации многозадачности можно выделить две группы алгоритмов:

- невытесняющая многозадачность (NetWare, Windows 3.x);
- вытесняющая многозадачность (Windows NT, OS/2, UNIX).

Основным различием между невытесняющим и вытесняющим вариантами многозадачности является степень централизации механизма планирования процессов. В одном случае механизм планирования процессов целиком сосредоточен в операционной системе, а в другом - распределен между системой и прикладными программами. При невытесняющей многозадачности активный процесс выполняется до тех пор, пока он сам, по собственной инициативе, не отдаст управление операционной системе для того, чтобы та выбрала из очереди другой готовый к выполнению процесс. При вытесняющей многозадачности решение о переключении процессора с одного процесса на другой принимается операционной системой, а не самим активным процессом.

3. *По способу взаимодействия с пользователем* различают *диалоговые* и *пакетные* ОС. В первом случае пользователь непосредственно взаимодействует с различными компонентами операционной системы и может влиять на ход выполнения вычислительного процесса. Во втором случае пользователь формирует т.н. пакет для выполнения своего задания и направляет его на выполнения. До окончания выполнения пакета взаимодействие пользователя и вычислительной системы прерывается.

4. При организации работы с вычислительной системой в диалоговом режиме следует провести классификацию *по количеству пользователей*: различают *однопользовательские (однотерминальные)* и *мультитерминальные* ОС. В мультитерминальных ОС с одной вычислительной системой могут работать несколько пользователей, каждый – со своего терминала. При этом у пользователей создается впечатление, что у каждого из них имеется собственная вычислительная система. Очевидно, что для реализации мультитерминального режима должен быть реализован мультипрограммный режим. Примером мультитерминальной системы является Linux. Главным отличием многопользовательских систем от однопользовательских является наличие средств защиты информации каждого пользователя от несанкционированного доступа других пользователей. Следует заметить, что не всякая многозадач-

ная система является многопользовательской, и не всякая однопользовательская ОС является однозадачной.

5. По архитектурному принципу стоит выделить микроядерные и монолитные ОС. В первом случае ядро ОС состоит из большого числа различных программных модулей и драйверов, и пользователь может сам управлять процессом создания ядра. Во втором случае ядро ОС пользователь не может изменить. Большинство ОС использует монолитное ядро, которое компонуется как одна программа, работающая в привилегированном режиме и использующая быстрые переходы с одной процедуры на другую, не требующие переключения из привилегированного режима в пользовательский и наоборот. Альтернативой является построение ОС на базе микроядра, работающего также в привилегированном режиме и выполняющего только минимум функций по управлению аппаратурой, в то время как функции ОС более высокого уровня выполняют специализированные компоненты ОС - серверы, работающие в пользовательском режиме. При таком построении ОС работает более медленно, так как часто выполняются переходы между привилегированным режимом и пользовательским, зато, с другой стороны, система получается более гибкой - ее функции можно наращивать, модифицировать или сужать, добавляя, модифицируя или исключая серверы пользовательского режима. Кроме того, серверы хорошо защищены друг от друга, как и любые пользовательские процессы.

Итак, микроядро реализует базовые функции операционной системы, на которые опираются другие системные службы и приложения. Следует отметить, что основной проблемой при конструировании микроядерной ОС является распознавание тех функций системы, которые могут быть вынесены из ядра. Такие важные компоненты ОС, как файловые системы, системы управления окнами и службы безопасности, становятся периферийными модулями, взаимодействующими с ядром и друг с другом. Компоненты, лежащие вне микроядра, используют его средства для обмена сообщениями, но взаимодействуют непосредственно. Микроядро лишь проверяет законность сообщений, пересылает их между компонентами и обеспечивает доступ к аппаратуре. Основным принципом организации микроядерных ОС является включение в состав микроядра только тех функций, которые абсолютно необходимо выполнять в режиме супервизора и в защищенной памяти. Обычно в микроядро включаются машинно-зависимые программы (включая поддержку мультипроцессорной работы), некоторые функции управления процессами, обработка прерываний и поддержка пересылки сообщений.

6. По принципу модифицируемости различают *открытые* и *закрытые* операционные системы. Следует сразу заметить, что этот принцип применяется не только к операционным системам, но и к любым программным продуктам. Открытый подход подразумевает под собой возможность пользо-

вателю модифицировать не только входные параметры программы, но и основные алгоритмы, на которых основана ее работа, а также комбинировать имеющиеся методы. Закрытый подход, напротив, предоставляет пользователю изменять только некоторый ограниченный набор входных параметров программы. Сам же программный продукт является «черным ящиком». Сокрытие основных алгоритмов не является отрицательным качеством того или иного продукта. Это может быть вызвано рядом причин, таких, как громоздкость (или достаточная простота) алгоритма, коммерческая ценность алгоритма как такового и т.д.

Альтернативой закрытым решениям построения ОС, преобладающим в настоящее время, является концепция открытых систем. Идея открытых систем исходит из того, что для различных задач необходимы разные системы - как специализированные, так и системы общего назначения, но по-разному настроенные и сбалансированные. Сложность состоит в том, что необходимо обеспечить:

- взаимодействие разнородных систем в сети;
- обмен данными между различными приложениями на различных платформах;
- переносимость прикладного ПО с одной платформы на другую (хотя бы путем перекомпиляции исходных текстов).
- по возможности однородный пользовательский интерфейс.

Эти задачи решаются при помощи открытых стандартов - стандартных сетевых протоколов, стандартных форматов данных, стандартизации программных интерфейсов, представленных в виде API, а также стандартизации пользовательского интерфейса.

Выбор типа операционной системы представляет собой в некоторых ситуациях непростую задачу. Некоторые приложения могут накладывать жесткие требования, которым удовлетворяют лишь небольшое количество систем. Другие приложения зачастую требуют высокой надежности и производительности (например, серверы баз данных).

1.3. Эволюция операционных систем

Чтобы понять основные требования, которые предъявляются к операционным системам, а также основные возможности современных операционных систем, необходимо проследить за их эволюцией, которая происходит на протяжении многих лет.

Первый период (1945 -1955)

Известно, что компьютер был изобретен английским математиком Чарльзом Бэббиджем в середине девятнадцатого века. Его «аналитическая машина»

стала прообразом современных компьютеров, так как включала их основные элементы: память, ячейки которой содержали бы числа, и арифметическое устройство, состоящее из рычагов и шестеренок. Бэббидж предусмотрел возможность вводить в машину инструкции при помощи перфокарт. Машина Бэббиджа так и не смогла заработать, т.к. технологии того времени не удовлетворяли требованиям по изготовлению деталей точной механики, которые были необходимы для вычислительной техники. Этот компьютер не имел (и не мог иметь) операционной системы.

Некоторый прогресс в создании цифровых вычислительных машин произошел после второй мировой войны. В середине 40-х были созданы первые ламповые вычислительные устройства. Это была скорее научно-исследовательская работа в области вычислительной техники, а не использование компьютеров в качестве инструмента решения каких-либо практических задач. Программирование осуществлялось исключительно на машинном языке. Об операционных системах не было и речи, все задачи организации вычислительного процесса решались вручную каждым программистом с пульта управления. Не было никакого другого системного программного обеспечения, кроме библиотек математических и служебных подпрограмм.

Второй период (1955 - 1965)

С середины 50-х годов начался новый период в развитии вычислительной техники, связанный с появлением новой технической базы - полупроводниковых элементов. Компьютеры второго поколения стали более надежными, теперь они смогли непрерывно работать настолько долго, чтобы на них можно было возложить выполнение действительно практически важных задач. Именно в этот период произошло разделение персонала на программистов, операторов, разработчиков вычислительных машин.

В эти годы появились первые алгоритмические языки, первые системные программы - компиляторы. Стоимость процессорного времени возросла, что потребовало уменьшения непроизводительных затрат времени между запусками программ. Появились первые системы пакетной обработки, которые просто автоматизировали запуск одной программ за другой и тем самым увеличивали коэффициент загрузки процессора. Системы пакетной обработки явились прообразом современных операционных систем, они стали первыми системными программами, предназначенными для управления вычислительным процессом. В ходе реализации систем пакетной обработки был разработан формализованный язык управления заданиями, с помощью которого программист сообщал системе и оператору, какую работу он хочет выполнить на вычислительной машине. Совокупность описаний нескольких заданий, оформленная в виде колоды перфокарт, получила название пакета заданий.

Третий период (1965 - 1980)

Следующий важный период развития вычислительных машин относится к 1965-1980 годам. В это время в технической базе произошел переход от отдельных полупроводниковых элементов (транзисторов) к интегральным микросхемам, что дало большие возможности новому, третьему поколению компьютеров.

Для этого периода характерно также создание семейств программно-совместимых машин. Первым семейством программно-совместимых машин, построенных на интегральных микросхемах, явилась серия машин IBM/360. Построенное в начале 60-х годов, это семейство значительно превосходило машины второго поколения по критерию «цена/производительность».

Программная совместимость требовала и совместимости операционных систем. Такие операционные системы должны были бы работать и на больших, и на малых вычислительных системах. Операционные системы, построенные с намерением удовлетворить всем этим противоречивым требованиям, оказались чрезвычайно сложными "монстрами". Они состояли из многих миллионов ассемблерных строк, написанных тысячами программистов, и содержали тысячи ошибок, вызывающих нескончаемый поток исправлений. В каждой новой версии операционной системы исправлялись одни ошибки и вносились новые.

Однако, несмотря на необозримые размеры и множество проблем, OS/360 и другие ей подобные операционные системы машин третьего поколения действительно удовлетворяли большинству требований потребителей. Важнейшим достижением ОС данного поколения явилась реализация мультипрограммирования. Как уже было сказано ранее, *мультипрограммирование* - это способ организации вычислительного процесса, при котором на одном процессоре попеременно выполняются несколько программ.

Другим нововведением явилось внедрение *спулинга* (spooling). Спулинг в то время определялся как способ организации вычислительного процесса, в соответствии с которым задания считывались с перфокарт на диск в том темпе, в котором они появлялись в помещении вычислительного центра. Затем, когда очередное задание завершалось, новое задание с диска загружалось в освободившийся раздел памяти.

Наряду с мультипрограммной реализацией систем пакетной обработки появился новый тип ОС - системы разделения времени. Вариант мультипрограммирования, применяемый в системах разделения времени, нацелен на создание для каждого отдельного пользователя иллюзии единоличного использования вычислительной машины.

Четвертый период (1980 - настоящее время)

Следующий период в эволюции операционных систем связан с появлением больших интегральных схем (БИС). В эти годы произошло резкое возрас-

тание степени интеграции и удешевление микросхем. Компьютер стал доступен отдельному человеку, и наступила эра персональных компьютеров. С точки зрения архитектуры персональные компьютеры ничем не отличались от класса миникомпьютеров типа PDP-11, но вот цена у них существенно отличалась. Если миникомпьютер дал возможность иметь собственную вычислительную машину отделу предприятия или университету, то персональный компьютер сделал это возможным для отдельного человека.

Компьютеры стали широко использоваться неспециалистами, что потребовало разработки «дружественного» программного обеспечения и положило конец кастовости программистов.

На рынке операционных систем доминировали две системы: MS-DOS и UNIX. Однопрограммная однопользовательская ОС MS-DOS широко использовалась для компьютеров, построенных на базе микропроцессоров Intel 8088, а затем 80286, 80386 и 80486. Мультипрограммная многопользовательская ОС UNIX доминировала в среде «не-интеловских» компьютеров, особенно построенных на базе высокопроизводительных RISC-процессоров.

В середине 80-х годов стали бурно развиваться сети персональных компьютеров, работающие под управлением сетевых, или распределенных операционных систем.

В сетевых ОС пользователи должны быть осведомлены о наличии других компьютеров и должны иметь доступ к ним, чтобы воспользоваться сетевыми ресурсами (преимущественно файлами). Сетевая ОС обязательно содержит программную поддержку для сетевых интерфейсных устройств (драйвер сетевого адаптера), а также средства для удаленного входа в другие компьютеры сети и средства доступа к удаленным файлам, однако эти дополнения существенно не меняют структуру самой операционной системы.

1.4. Основные функции операционных систем

Проанализировав этапы развития вычислительных систем, мы можем выделить основные функции, которые выполняли классические операционные системы в процессе своей эволюции:

- планирование заданий и использования процессора;
- обеспечение программ средствами коммуникации и синхронизации;
- управление памятью;
- управление файловой системой;
- управление вводом-выводом;
- обеспечение безопасности.

Каждая из приведенных функций обычно реализована в виде подсистемы, являющейся структурным компонентом ОС. В каждой конкретной операционной системе эти функции, конечно, реализовывались по-своему, в различ-

ном объеме. Они не были изначально придуманы как составные части деятельности операционных систем, а появились в процессе развития, по мере того, как вычислительные системы становились удобнее, эффективнее и безопаснее. Эволюция вычислительных систем, созданных человеком, пошла по такому пути, но никто еще не доказал, что это единственно возможный путь их развития. Операционные системы существуют потому, что на настоящий момент их существование - это разумный способ использования вычислительных систем.

Итак, с учетом современных представлений и требований, ОС должна выполнять следующие функции:

- обеспечивать загрузку пользовательских программ в оперативную память и их выполнение;
- обеспечивать управление памятью;
- обеспечивать работу с устройствами долговременной памяти, управлять свободным пространством на этих носителях и структурировать пользовательские данные в виде файловых систем;
- предоставлять стандартизированный доступ к различным периферийным устройствам;
- обеспечивать параллельное исполнение нескольких задач;
- обеспечивать организацию взаимодействия задач друг с другом;
- обеспечивать организацию межмашинного взаимодействия и распределения ресурсов;
- обеспечивать защиту системных ресурсов, данных и программ пользователя, исполняющихся процессов и самой себя от ошибочных и несанкционированных действий пользователей и их программ;
- предоставлять возможность аутентификации, авторизации и других средств обеспечения безопасности.

Рассмотрение общих принципов и алгоритмов реализации этих функций и будет составлять содержание большей части этой книги.

Глава 2. Процессы и ресурсы

Одной из важных функций ОС является организация рационального использования всех аппаратных и программных ресурсов системы. К основным ресурсам могут быть отнесены: процессоры, память, внешние устройства, данные и программы. Располагающая одними и теми же ресурсами, но управляемая различными ОС, вычислительная система может работать с разной степенью эффективности. Знание же внутренних механизмов операционной системы позволяет косвенно судить о ее эксплуатационных возможностях и характеристиках.

2.1. Основные определения

Понятие *вычислительного процесса* (или просто «процесса») является одним из основных при рассмотрении операционных систем. Будем придерживаться следующего неформального определения процесса: (последовательный) процесс – это выполнение отдельной программы с ее данными на последовательном процессоре. В качестве примеров можно назвать следующие процессы (задачи): прикладные задачи пользователей, работа различных систем программирования, утилит и других системных обрабатывающих программ. Для ряда ОС (например, QNX) работа отдельных модулей операционной системы также рассматривается как процесс. На концептуальном уровне именно планирование и управление процессами является основой работы операционных систем.

Для каждого запущенного в системе процесса строится специальная структура (т.н. *дескриптор процесса*). Конкретный вид этой структуры зависит от операционной системы, но следующие поля присутствуют практически всегда:

- идентификатор процесса;
- класс, или тип процесса;
- приоритет процесса, в соответствии с которым процессу предоставляются ресурсы;
- информация о текущем состоянии процесса;
- контекст процесса, т.е. та информация, которая необходима для возобновления работы процесса;
- информация о ресурсах, которыми владеет процесс;
- параметры запуска (момент времени, когда процесс должен активизироваться, периодичность запуска и т.д.).

Для своего выполнения процесс требует выделения ему *ресурсов* вычислительной системы. Понятие ресурса, как и процесса, является, пожалуй, ос-

новным при рассмотрении операционных систем. Ресурс – это повторно используемый, относительно стабильный и часто недостающий объект, который запрашивается, используется и освобождается процессами в период их активности. Иными словами, ресурсом называется всякий объект, который может распределяться внутри системы.

Ресурсы подразделяются на *неделимые* и *делимые*. Последние, в свою очередь, разделяются на используемые *одновременно* и *параллельно*.

Отметим, что процессы определяются рядом временных характеристик. В некоторый момент времени процесс может быть порожден (образован), а через некоторое время закончен. Интервал между этими моментами называют *интервалом существования процесса*.

В операционной системе принято различать процессы не только по времени, но и по месту их развития, т. е. по информации о том, на каком из процессоров выполняется программа процесса. Точкой отсчета принято считать центральный процессор (процессоры), на котором развиваются процессы, называемые *программными* или *внутренними*. Такое название указывает на возможность существования в системе процессов, называемых *внешними*. Это процессы, развитие которых происходит под контролем или управлением ОС на процессорах, отличных от центрального процессора. Ими могут быть, например, процессы ввода — вывода, развивающиеся в канале. Управление взаимосвязанными процессами в составе ОС основано на контроле и удовлетворении определенных ограничений, которые накладываются на порядок выполнения таких процессов. Данные ограничения определяют *виды отношений*, которые допустимы между процессами, и составляют в совокупности синхронизирующие правила:

- *отношение предшествования*. Для двух процессов это отношение означает, что первый процесс должен переходить в активное состояние всегда раньше второго;

- *отношение приоритетности*. Процесс с приоритетом P может быть переведен в активное состояние только при соблюдении двух условий: в состоянии готовности к рассматриваемому процессору нет процессов с большим приоритетом; процессор либо свободен, либо используется процессом с меньшим, чем P , приоритетом.

- *отношение взаимного исключения*. В этом случае два процесса используют обобщенный ресурс. При этом совокупность действий над этим ресурсом в составе одного процесса называют критической областью. Критическая область одного процесса не должна выполняться одновременно с критической областью другого процесса, связанной с одним и тем же ресурсом.

Трудность в реализации синхронизирующих правил в составе системы управления процессами обусловлена динамикой процессов, неопределенностью и непредсказуемостью порядка и частотой перехода процессов из со-

стояния в состояние по мере их развития. При этом в отношении каждой совокупности взаимосвязанных процессов приходится решать собственную задачу синхронизации, которая требует определенного порядка выполнения процессов. Помимо рассмотренных, в каждой из задач могут использоваться и другие, более сложные виды отношений, например, отношения «читатели—писатели», о которых будет более подробно сказано позднее.



Рис. 2.1. Классификация процессов

Исходя из сказанного выше, можно предложить многоуровневую классификацию процессов, показанную на рисунке 2.1.

Согласно принципам фон Неймана, в первых вычислительных системах все подсистемы и устройства компьютера управлялись исключительно центральным процессором. Так, во время выполнения операций ввода-вывода процессор не мог, например, производить вычисления. Естественно, подобные системы могли быть только однопрограммными.

Введение в состав вычислительной системы специальных контроллеров позволило распараллелить процессы ввода/вывода и обработки результатов. Однако процессор подолгу простаивал, дожидаясь завершения операции ввода/вывода. Поэтому организация мультипрограммных вычислений явилась естественным шагом в оптимизации работы компьютера.

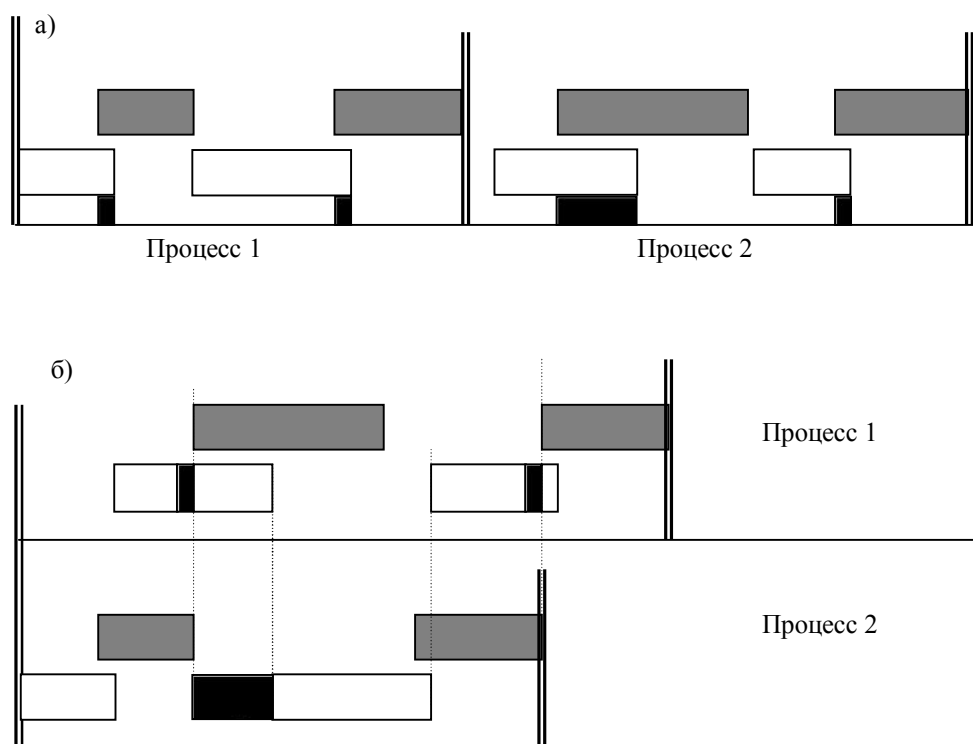


Рис. 2.2. Пример выполнения двух процессов:
а) в однопрограммном; б) в мультипрограммном режиме.

На рис. 2.2 показана ситуация, когда выполняются две задачи, которые требуют обращения к одному и тому же устройству ввода вывода. Рисунок 2.2 а) отражает ситуацию, когда процессы выполняются последовательно; прямоугольники без заливки соответствуют загрузке центрального процессо-

ра, а залитые прямоугольники - работе устройства ввода-вывода. Поскольку процессы ввода/вывода и обработки результатов распараллелены, между запросом на ввод-вывод и необходимостью обработки результатов этого запроса проходит какое-то время, в течение которого процесс может загружать процессор полезной работой. Эти промежутки времени отображаются более темным цветом.

На рисунке 2.2 б) показана временная диаграмма параллельной работы этих же задач. Следует заметить, что суммарное время выполнения этих задач меньше, чем в случае их последовательного выполнения (хотя время работы каждой задачи увеличивается). Время простоя каждого процесса также отражено более темным прямоугольником.

При мультипрограммном режиме работы требуется эффективно использовать имеющиеся ресурсы. Это достигается за счет организации *очереди процессов к ресурсам*. При необходимости использовать какой-либо ресурс процесс обращается к модулям ОС с *запросом*. При этом указывается вид ресурса, его объем и тип работы (монопольный или разделяемый режим).

Ресурс может быть выделен процессу, если:

- он свободен и нет запросов к этому же ресурсу от процессов с более высоким приоритетом;
- текущий запрос и ранее выданные запросы допускают совместное использование этого ресурса;
- ресурс используется задачей с более низким приоритетом и может быть временно отобран.

Если согласно этим критериям запрос на ресурс не может быть удовлетворен, то процесс ставится в очередь к этому ресурсу.

После окончания работы с ресурсом процесс опять-таки обращается к ОС с запросом об отказе от ресурса (ресурс может быть отобран системой принудительно). ОС освобождает ресурс и проверяет, имеется ли очередь к освобожденному ресурсу. Если упомянутая очередь не пуста, то управление либо передается одной из задач, ожидающих ресурс, либо возвращается задаче, освободившей ресурс. Конечно же, в зависимости от типа ресурсов используются специальные механизмы их учета и выделения, которые будут рассмотрены позже.

Рассмотрим сейчас понятие *состояния процесса*. Если рассматривать не только ОС общего назначения, но и системы реального времени, то можно сказать, что процесс может находиться в пассивном и активном состоянии. Процесс, находящийся в пассивном состоянии, только известен системе, однако он не участвует в конкуренции за ресурсы. Активный же процесс, в свою очередь, может находиться в одном из трех состояний:

- **ВЫПОЛНЕНИЕ** – все затребованные ресурсы, в том числе и время процессора, выделены. В этом состоянии может находиться только один процесс;

- **ГОТОВНОСТЬ** – процессу, находящемуся в этом состоянии, могут быть предоставлены все ресурсы, кроме времени процессора;

- **ОЖИДАНИЕ** (*блокирование*) – процесс находится в очереди к какому-либо ресурсу или ожидает окончания операции ввода-вывода.

В ходе жизненного цикла каждый процесс переходит из одного состояния в другое в соответствии с алгоритмом планирования процессов, реализуемым в данной операционной системе.

В состоянии **ВЫПОЛНЕНИЕ** в однопроцессорной системе может находиться только один процесс, а в каждом из состояний **ОЖИДАНИЕ** и **ГОТОВНОСТЬ** - несколько процессов, эти процессы образуют очереди соответственно ожидающих и готовых процессов.

Жизненный цикл процесса начинается с состояния **ГОТОВНОСТЬ**, когда процесс готов к выполнению и ждет своей очереди. При активизации процесс переходит в состояние **ВЫПОЛНЕНИЕ** и находится в нем до тех пор, пока либо он сам освободит процессор, перейдя в состояние **ОЖИДАНИЯ** какого-нибудь события, либо будет насильно "вытеснен" из процессора, например, вследствие исчерпания отведенного данному процессу кванта процессорного времени. В последнем случае процесс возвращается в состояние **ГОТОВНОСТЬ**. В это же состояние процесс переходит из состояния **ОЖИДАНИЕ**, после того, как ожидаемое событие произойдет.

В свою очередь, процессы могут перейти из состояния **ГОТОВНОСТЬ** в состояние **ОЖИДАНИЯ** и обратно, минуя стадию выполнения. Это может произойти, когда выполняющийся процесс запрашивает либо освобождает какой-либо ресурс, преобразуя тем самым очередь, соответствующую этому ресурсу.

Граф состояний процесса и возможных переходов между состояниями изображен на рис. 2.3.

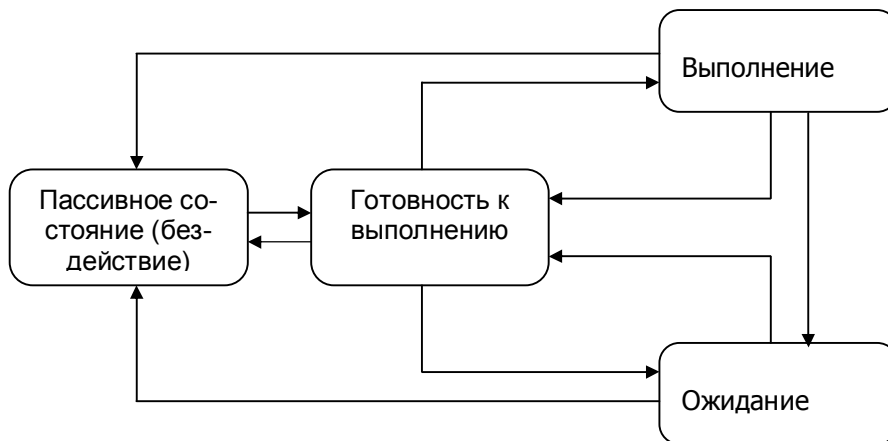


Рис. 2.3. Граф состояний процесса

На протяжении существования процесса его выполнение может быть многократно прервано и продолжено. Для того, чтобы возобновить выполнение процесса, необходимо восстановить состояние его операционной среды. Состояние операционной среды отображается состоянием регистров и программного счетчика, режимом работы процессора, указателями на открытые файлы, информацией о незавершенных операциях ввода-вывода, кодами ошибок выполняемых данным процессом системных вызовов и т.д. Эта информация называется *контекстом процесса*.

Кроме этого, операционной системе для реализации планирования процессов требуется дополнительная информация: идентификатор процесса, состояние процесса, данные о степени привилегированности процесса, место нахождения кодового сегмента и другая информация. В некоторых ОС (например, в ОС UNIX) информацию такого рода, используемую ОС для планирования процессов, называют *дескриптором процесса*.

Дескриптор процесса по сравнению с контекстом содержит более оперативную информацию, которая должна быть легко доступна подсистеме планирования процессов. Контекст процесса содержит менее актуальную информацию и используется ОС только после того, как принято решение о возобновлении прерванного процесса.

Очереди процессов представляют собой дескрипторы отдельных процессов, объединенные в списки. Таким образом, каждый дескриптор процесса содержит по крайней мере один указатель на другой дескриптор, соседствующий с ним в очереди. Такая организация очередей позволяет легко их перепорядочивать, включать и исключать процессы, переводить процессы из одного состояния в другое.

Программный код только тогда начнет выполняться, когда для него операционной системой будет создан процесс. При создании процесса выполняются следующие действия:

- создаются информационные структуры, описывающие данный процесс, т.е. его дескриптор и контекст;
- дескриптор нового процесса включается в очередь процессов, готовых к выполнению;
- кодовый сегмент процесса загружается в оперативную память или в область свопинга (понятие свопинга будет более подробно рассмотрено позднее, в главе 4).

Деятельность мультипрограммной операционной системы состоит из операций, выполняемых над различными процессами, и сопровождается переключением процессора с одного процесса на другой.

Для примера рассмотрим, как может выполняться операция разблокирования процесса, ожидающего операцию ввода-вывода, проиллюстрированная рисунком 2.4. При исполнении процессором некоторого процесса (на рисунке - процесс 1) возникает прерывание от устройства ввода-вывода,

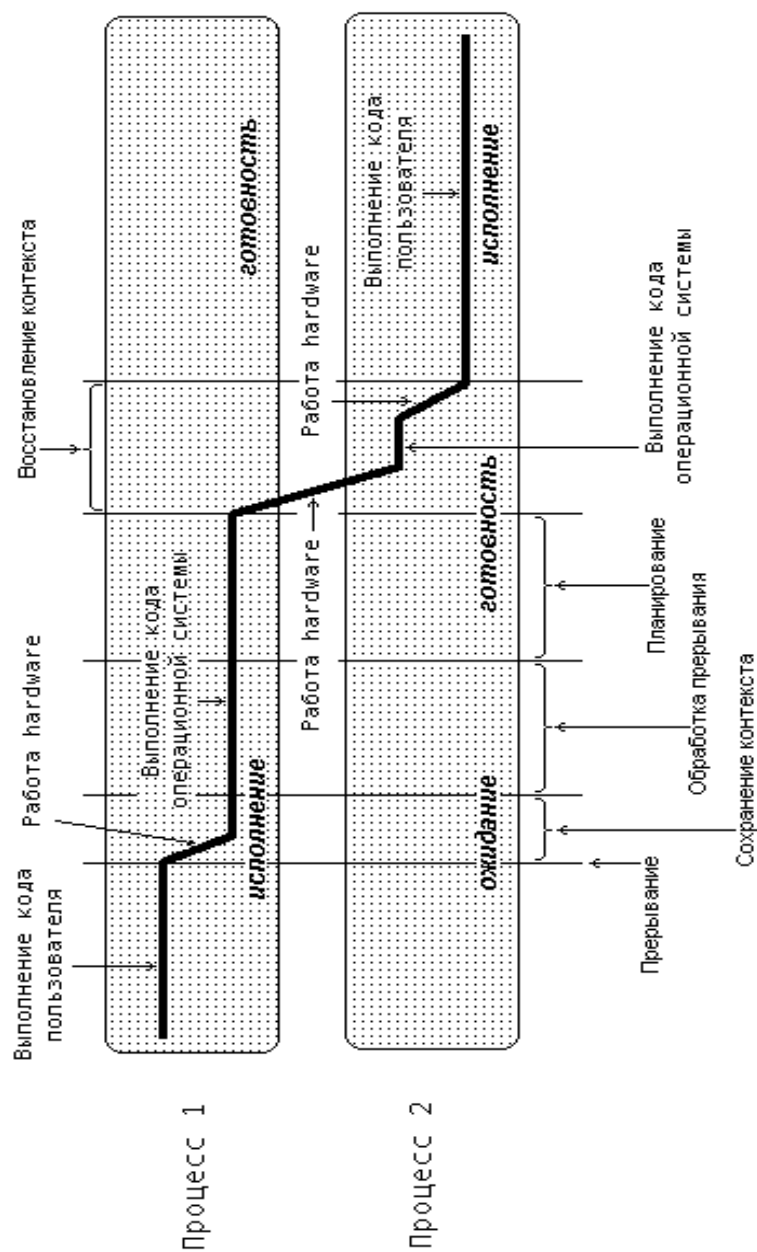


Рис 2.4. Выполнение операции переключения с одного процесса на другой

сигнализирующее об окончании операций на устройстве. Над выполняющимся процессом производится операция «приостановка». Далее, операционная система разблокирует процесс, инициировавший запрос на ввод-вывод (на

рисунке - процесс 2), и осуществляет запуск приостановленного или нового процесса, выбранного при выполнении планирования (на рисунке был выбран разблокированный процесс). В результате обработки информации об окончании операции ввода-вывода возможна смена процесса, находящегося в состоянии выполнения. Использование термина «код пользователя» не ограничивает общности рисунка только пользовательскими процессами.

Для корректного переключения процессора с одного процесса на другой необходимо сохранить контекст исполнявшегося процесса и восстановить контекст процесса, на который будет переключен процессор. Такая процедура сохранения/восстановления работоспособности процессов называется *переключением контекста*. Время, затраченное на переключение контекста, не используется вычислительной системой для совершения полезной работы и представляет собой накладные расходы, снижающие производительность системы. Оно меняется от машины к машине и обычно находится в диапазоне от 1 до 1000 микросекунд. Существенно сократить накладные расходы в современных операционных системах позволяет расширенная модель процессов, включающая в себя понятие *тредов* (threads of execution, нитей исполнения или просто нитей). Подробнее о нитях исполнения будет говориться в следующих главах.

Следует сказать, что переключение между различными процессами невозможно без использования механизма прерываний.

2.2. Прерывания

Концепция прерывания является одной из важнейших при проектировании и разработке операционных систем. Идея прерываний была предложена в середине 50-х годов; можно сказать, что она внесла наиболее весомый вклад в развитие вычислительной техники. Основная цель введения прерываний – реализация асинхронного режима работы вычислительного комплекса и параллеливание работы отдельных устройств.

Эта концепция означает, что выполнение процесса может быть прекращено вследствие наступления определенного события. После наступления прерывания выполняется его обработка. Механизм обработки прерываний реализуется как аппаратными, так и программными средствами и независимо от архитектуры вычислительной системы включает следующие элементы:

1. Установление факта прерывания (прием сигнала на прерывание) и идентификация прерывания.
2. Запоминание состояния прерванного процесса (например, адреса следующей команды, содержимого регистров процессора и другой информации).
3. Передача управления программе обработки прерывания. В простейшем случае (реальный режим работы процессора i80x86) перечень всех до-

пустимых прерываний и адреса соответствующих обработчиков хранятся в оперативной памяти (т.н. таблица IVT). В более развитых процессорах механизм поиска и загрузки соответствующего обработчика более сложен.

4. Сохранение информации о прерванной программе, которую не удалось сохранить на шаге 2.

5. Обработка прерывания.

6. Восстановление информации о прерванном процессе.

7. Возврат в прерванную программу.

Шаги 1-3 реализуются, как правило, аппаратно, а шаги 4-7 - программно.

Следует сказать, что такая схема напрямую используется только в однопрограммных системах. Для мультипрограммных систем характерны более сложные схемы обработки прерываний (так, после завершения обработки прерывания управление не обязательно передается прерванному процессу).

Прерывания, возникающие при работе вычислительной системы, можно разделить на два основных класса: *внешние* (асинхронные) и *внутренние* (синхронные). Внешние прерывания вызываются событиями, которые происходят вне прерываемого процесса и не обязательно связаны с ним. Так, к внешним прерываниям относятся:

- прерывания от таймера;
- прерывания от внешних устройств (прерывания ввода/вывода);
- прерывания по нарушению питания;
- прерывания от оператора вычислительной системы;
- прерывания от схем контроля компьютера (ошибки четности, сбой в работе устройств и т.д.).

Внутренние прерывания вызываются событиями, связанными с ходом выполняемого процесса, и являются синхронными с его операциями. К внутренним прерываниям можно отнести:

- прерывания, связанные с программными сбоями (недействительный код операции, обращение к несуществующему или запрещенному адресу памяти, деление на ноль, переполнение и т.д.);
- прерывания по обращению к ОС. В некоторых компьютерах команды разбиваются на обычные и привилегированные, которые могут выполняться только модулями ОС. К привилегированным командам, например, относятся команды обращения к внешним устройствам. Таким образом, для организации ввода/вывода необходимо обратиться к операционной системе;
- программные прерывания, которые происходят по соответствующей команде. Это позволяет обращаться к специальным модулям по тому же механизму, что и при возникновении обычного внутреннего прерывания.

Поскольку события, вызывающие прерывания, могут возникать независимо от хода выполнения задачи, может возникнуть ситуация, когда несколько

прерываний возникают одновременно, или когда во время обработки одного прерывания может наступить другое. Встает вопрос: что делать в этом случае? Для решения этого вопроса вводятся понятия *приоритета* прерываний и *дисциплины обслуживания* прерываний.

Каждому типу прерывания может приписываться свой приоритет, так, что из нескольких одновременно пришедших запросов на прерывание обслуживается тот, который имеет наивысший приоритет. Типичное разделение прерываний по уровням приоритета выглядит следующим образом (перечисление ведется от более высоких уровней к более низким):

- прерывания от схем контроля (очевидно, если аппаратура работает неправильно, нет смысла продолжать обработку информации);
- прерывания от таймера;
- прерывания от внешних устройств;
- программные прерывания.

Учет приоритетов прерываний может быть встроен в аппаратуру компьютера, а может быть реализован и программными средствами, т.е. средствами операционной системы. В этом случае можно регулировать обработку прерываний, заставляя процессор реагировать на них сразу по приходу, откладывать их обработку или вообще игнорировать. Прежде всего следует сказать, что обработку прерываний (кроме прерываний от схем контроля) можно отключить (*замаскировать* прерывания) для того, чтобы дать возможность выполнить какой-то критический участок.

Различают следующие дисциплины обработки прерываний:

- с *относительными приоритетами*. Если при обработке одного прерывания возникает другое, то обработка первого доводится до конца, даже если новое прерывание имеет более высокий приоритет. После окончания обработки первого прерывания из накопившихся к тому времени запросов на прерывания выбирается тот, который имеет наивысший приоритет. Реализация такой дисциплины очень проста: достаточно лишь наложить запрет на новые прерывания в программах-обработчиках;

- с *абсолютными приоритетами*. Если при обработке одного прерывания возникает другое с большим приоритетом, то обработка первого прерывания прекращается либо приостанавливается. Таким образом, становится возможным многоуровневая обработка прерываний;

- по *принципу стека (LIFO)*. Если при обработке одного прерывания возникает другое, то обработка первого прерывания прекращается либо приостанавливается, вне зависимости от того, каков приоритет нового прерывания.

Следует сказать, что многоуровневая обработка прерываний возможна только на шаге 5 схемы, описанной в начале параграфа, т.к. на этапе перехода от одного процесса к другому есть риск неправильно обслужить прерывание.

Обработка прерываний в мультипрограммных ОС отличается тем, что прерванный процесс переводится в состояние ожидания, а после обработки прерывания управление не передается прерванному ранее процессу, а выполняется выбор одного из числа находящихся в состоянии ожидания процессов, согласно принятой дисциплине обслуживания.

Прерывания должны быть скрыты как можно глубже в недрах операционной системы, чтобы как можно меньшая часть ОС имела с ними дело. Наилучший способ состоит в разрешении процессу, инициировавшему операцию, связанную с обращением к операционной системе, блокировать себя до завершения этой операции и наступления прерывания. При наступлении прерывания процедура обработки прерывания выполняет разблокирование процесса. В любом случае эффект от прерывания будет состоять в том, что ранее заблокированный процесс теперь продолжит свое выполнение.

2.3. Алгоритмы планирования процессов

Планирование процессов включает решение следующих задач:

- определение момента времени для смены процесса;
- выбор процесса на выполнение из очереди готовых процессов;
- переключение контекстов «старого» и «нового» процессов.

Первые две задачи решаются программными средствами, а третья в значительной степени аппаратно. Существует множество различных алгоритмов планирования процессов, которые различными способами решают вышеперечисленные задачи, преследуют различные цели и обеспечивают различное качество мультипрограммирования. Среди этого множества алгоритмов остановимся более подробно на двух группах наиболее часто встречающихся алгоритмов: алгоритмы, основанные на *квантовании*, и алгоритмы, основанные на *приоритетах*.

Принцип квантования основан на том, что процессорное время делится на небольшие промежутки (*кванты*), так что каждому процессу при переводе его в состояние ВЫПОЛНЕНИЕ может быть предоставлен только один квант времени. В соответствии с алгоритмами, основанными на квантовании, смена активного процесса происходит, если:

- процесс завершился и покинул систему;
- произошла программная ошибка;
- процесс перешел в состояние ОЖИДАНИЕ,
- исчерпан квант времени, отведенный данному процессу.

Процесс, который исчерпал свой квант, переводится в состояние ГОТОВНОСТЬ и ожидает, когда ему будет предоставлен новый квант времени, а на выполнение в соответствии с определенным правилом выбирается новый процесс из очереди готовых процессов. Таким образом, ни один процесс не занимает процессор надолго, поэтому квантование широко используется в системах разделения времени.

Кванты, выделяемые процессам, могут быть одинаковыми для всех процессов или различными. Кванты, выделяемые одному процессу, могут быть фиксированной величины или изменяться в разные периоды жизни процесса. Процессы, которые не полностью использовали выделенный им квант (например, из-за ухода на выполнение операций ввода-вывода), могут получать или не получать компенсацию в виде привилегий при последующем обслуживании.

Другая группа алгоритмов использует понятие приоритета процесса. Приоритет - это число, характеризующее степень привилегированности процесса при использовании ресурсов вычислительной машины, в частности, процессорного времени: чем выше приоритет, тем выше привилегии.

Приоритет может выражаться целыми или дробными, положительным или отрицательным значениями. Чем выше привилегии процесса, тем меньше времени он будет проводить в очередях. Приоритет может назначаться директивно администратором системы в зависимости от важности работы или внесенной платы, либо вычисляться самой ОС по определенным правилам; он может оставаться фиксированным на протяжении всей жизни процесса либо изменяться во времени в соответствии с некоторым законом. В последнем случае приоритеты называются динамическими.

Существует две разновидности приоритетных алгоритмов: алгоритмы, использующие относительные приоритеты, и алгоритмы, использующие абсолютные приоритеты.

В обоих случаях выбор процесса на выполнение из очереди готовых осуществляется одинаково: выбирается процесс, имеющий наивысший приоритет. Различными способами решается проблема определения момента смены активного процесса. В системах с относительными приоритетами активный процесс выполняется до тех пор, пока он сам не покинет процессор, перейдя в состояние ОЖИДАНИЕ (или же произойдет ошибка, или процесс завершится). В системах с абсолютными приоритетами выполнение активного процесса прерывается еще при одном условии: если в очереди готовых процессов появился процесс, приоритет которого выше приоритета активного процесса. В этом случае прерванный процесс переходит в состояние готовности. На рисунке 2.5 показаны графы состояний процесса для алгоритмов с относительными и абсолютными приоритетами.

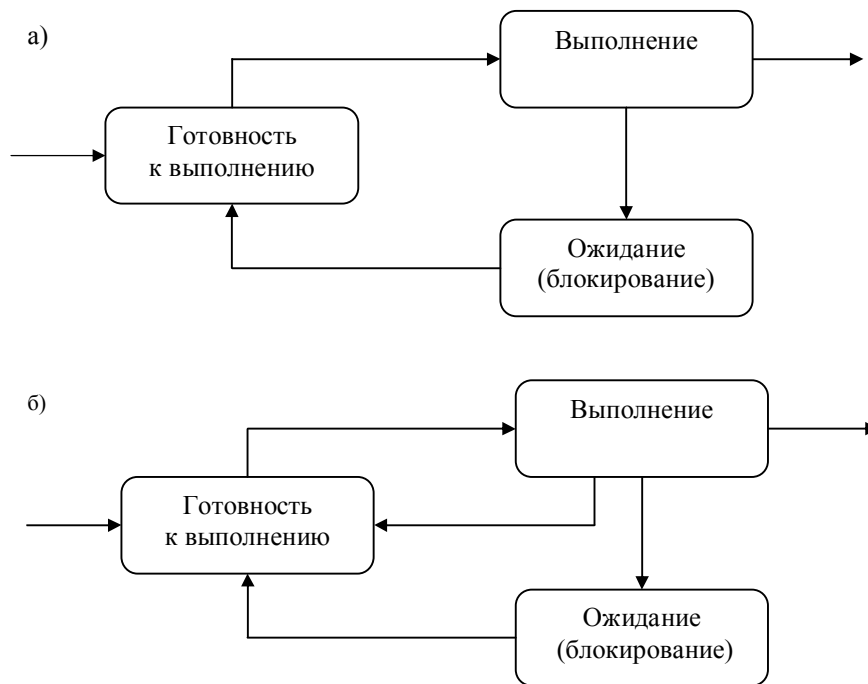


Рис. 2.5. Графы состояний процессов в системах
а) с относительными приоритетами; б) с абсолютными приоритетами

Во многих операционных системах алгоритмы планирования построены с использованием как квантования, так и приоритетов. Например, в основе планирования лежит квантование, но величина кванта и/или порядок выбора процесса из очереди готовых определяется приоритетами процессов.

2.4. Вытесняющие и невытесняющие алгоритмы планирования

Существует два основных типа процедур планирования процессов - вытесняющие (preemptive) и невытесняющие (non-preemptive).

Невытесняющая многозадачность (non-preemptive multitasking) - это способ планирования процессов, при котором активный процесс выполняется до тех пор, пока он сам, по собственной инициативе, не отдаст управление планировщику операционной системы для того, чтобы тот выбрал из очереди другой, готовый к выполнению процесс.

Вытесняющая многозадачность (preemptive multitasking), напротив, предполагает, что решение о переключении процессора с выполнения одного процесса на выполнение другого процесса принимается планировщиком операционной системы, а не самой активной задачей.

Понятия вытесняющей и невытесняющей многозадачности иногда отождествляются с понятиями приоритетных и беспriorитетных дисциплин, что совершенно неверно, а также с понятиями абсолютных и относительных приоритетов, что неверно отчасти. Вытесняющая и невытесняющая многозадачность - это более широкие понятия, чем типы приоритетности. Приоритеты задач могут как использоваться, так и не использоваться и при вытесняющих, и при невытесняющих способах планирования. Так, в случае использования приоритетов дисциплина относительных приоритетов может быть отнесена к классу систем с невытесняющей многозадачностью, а дисциплина абсолютных приоритетов - к классу систем с вытесняющей многозадачностью. С другой стороны, беспriorитетная дисциплина планирования, основанная на выделении равных квантов времени для всех задач, относится к вытесняющим алгоритмам.

Основным различием между вытесняющим и невытесняющим вариантами многозадачности является степень централизации механизма планирования задач. При вытесняющей многозадачности механизм планирования задач целиком сосредоточен в операционной системе, и программист пишет свое приложение, не заботясь о том, что оно будет выполняться параллельно с другими задачами. При этом операционная система выполняет следующие функции: определяет момент снятия с выполнения активной задачи, запоминает ее контекст, выбирает из очереди готовых задач следующую и запускает ее на выполнение, загружая ее контекст.

При невытесняющей многозадачности механизм планирования распределен между системой и прикладными программами. Прикладная программа, получив управление от операционной системы, сама определяет момент завершения своей очередной итерации и передает управление ОС с помощью какого-либо системного вызова, а ОС формирует очереди задач и выбирает в соответствии с некоторым алгоритмом (например, с учетом приоритетов) следующую задачу на выполнение. Такой механизм создает проблемы как для пользователей, так и для разработчиков.

Для пользователей это означает, что управление системой теряется на произвольный период времени, который определяется приложением (а не пользователем). Если приложение тратит слишком много времени на выполнение какой-либо работы, например, на форматирование диска, пользователь не может переключиться с этой задачи на другую задачу, например, на текстовый редактор, в то время как форматирование продолжалось бы в фоновом режиме. Эта ситуация нежелательна, так как пользователи обычно не хотят долго ждать, когда машина завершит свою задачу.

Поэтому разработчики приложений для невытесняющей операционной среды, возлагая на себя функции планировщика, должны создавать приложения так, чтобы они выполняли свои задачи небольшими частями. Например, программа форматирования может отформатировать одну дорожку дискеты и

вернуть управление системе. После выполнения других задач система возвратит управление программе форматирования, чтобы та отформатировала следующую дорожку. Подобный метод разделения времени между задачами работает, но он существенно затрудняет разработку программ и предъявляет повышенные требования к квалификации программиста. Программист должен обеспечить «дружественное» отношение своей программы к другим программам, выполняемым одновременно с ней, достаточно часто отдавая им управление. Крайним проявлением «недружественности» приложения является его зависание, которое приводит к общему краху системы. В системах с вытесняющей многозадачностью такие ситуации, как правило, исключены, так как центральный планирующий механизм снимет зависшую задачу с выполнения.

Однако распределение функций планировщика между системой и приложениями не всегда является недостатком, а при определенных условиях может быть и преимуществом, потому что дает возможность разработчику приложений самому проектировать алгоритм планирования, наиболее подходящий для данного фиксированного набора задач. Так как разработчик сам определяет в программе момент времени отдачи управления, то при этом исключаются нерациональные прерывания программ в «неудобные» для них моменты времени. Кроме того, легко разрешаются проблемы совместного использования данных: задача во время каждой итерации использует их монопольно и уверена, что на протяжении этого периода никто другой не изменит эти данные. Существенным преимуществом невытесняющих систем является более высокая скорость переключения с одной задачи на другую.

2.5. Взаимодействующие процессы

Для достижения поставленной цели различные процессы (возможно, принадлежащие разным пользователям) могут исполняться псевдопараллельно на одной вычислительной системе или параллельно на разных вычислительных системах, взаимодействуя между собой.

Для чего процессам нужно заниматься совместной деятельностью? Какие существуют причины для их кооперации?

Одной из причин является повышение скорости работы. Когда один процесс ожидает наступления некоторого события (например, окончания операции ввода-вывода), другие процессы в это время могут заниматься полезной работой, направленной на решение общей задачи. В многопроцессорных вычислительных системах программа разделяется на отдельные кусочки, каждый из которых будет исполняться на своем процессоре.

Второй причиной является совместное использование данных. Различные процессы могут, к примеру, работать с одной и той же динамической базой данных или с разделяемым файлом, совместно изменяя их содержимое.

Третьей причиной является модульная конструкция какой-либо системы. Типичным примером может служить микроядерный способ построения операционной системы, когда ее различные части представляют собой отдельные процессы, общающиеся путем передачи сообщений через микроядро.

Наконец, это может быть необходимо просто для удобства работы пользователя, желающего, например, редактировать и отлаживать программу одновременно. В этой ситуации процессы редактора и отладчика должны уметь взаимодействовать друг с другом.

Процессы не могут взаимодействовать, не общаясь. Общение процессов обычно приводит к изменению их поведения в зависимости от полученной информации. Если деятельность процессов остается неизменной при любой принятой ими информации, то это означает, что они на самом деле не нуждаются во взаимном общении. Процессы, которые влияют на поведение друг друга путем обмена информацией, принято называть *кооперативными* или *взаимодействующими* процессами, в отличие от независимых процессов, не оказывающих друг на друга никакого воздействия и ничего не знающих о взаимном сосуществовании в вычислительной системе.

Различные процессы в вычислительной системе изначально представляют собой обособленные сущности. Работа одного процесса не должна приводить к нарушению работы другого процесса. Для этого, в частности, разделены их адресные пространства и системные ресурсы, и для обеспечения корректного взаимодействия процессов требуются специальные средства и действия операционной системы. Нельзя просто поместить значение, вычисленное в одном процессе, в область памяти, соответствующую переменной в другом процессе, не предприняв каких-либо дополнительных организационных усилий.

Процессы могут взаимодействовать друг с другом только обмениваясь информацией. По объему передаваемой информации и степени возможного воздействия на поведение другого процесса все средства такого обмена можно разделить на три категории:

Сигнальные. Передается минимальное количество информации — один бит, “да” или “нет”. Такие средства обмена используются, как правило, для извещения процесса о наступлении какого-либо события. Степень воздействия на поведение процесса, получившего информацию, минимальна. Все зависит от того, знает ли процесс-получатель, что означает полученный сигнал, надо ли на него реагировать и каким образом сделать это.

Канальные. Общение процессов происходит через линии связи, предоставленные операционной системой, и напоминает общение людей по телефону, с помощью записок, писем или объявлений. Объем передаваемой информации в единицу времени ограничен пропускной способностью линий связи. С увеличением количества информации увеличивается и возможность влияния на поведение другого процесса.

Разделяемая память. Два или более процессов могут совместно использовать некоторую область адресного пространства. Созданием разделяемой памяти занимается операционная система. Возможность обмена информацией максимальна, как, впрочем, и влияние на поведение другого процесса, но требует повышенной осторожности. Использование разделяемой памяти для передачи/получения информации осуществляется с помощью средств обычных языков программирования, в то время как сигнальным и канальным средствам коммуникации для этого необходимы специальные системные вызовы. Разделяемая память представляет собой наиболее быстрый способ взаимодействия процессов в одной вычислительной системе.

2.6. Треды

Когда мы говорим о процессах, то подчеркиваем их обособленность друг от друга, которую поддерживает операционная система: у каждого процесса имеется свое виртуальное адресное пространство, каждому процессу выделяются свои ресурсы и т.д. В общем случае процессы никак не связаны между собой и могут даже принадлежать различным пользователям, разделяющим одну вычислительную систему. Роль арбитра в конкуренции между процессами по поводу ресурсов берет на себя операционная система.

Рассмотренные выше аспекты логической реализации относятся к средствам связи, ориентированным на организацию взаимодействия различных процессов. Однако усилия, направленные на ускорение решения задач в рамках классических операционных систем, привели к изменению самого понятия “процесс”.

В свое время внедрение идеи мультипрограммирования позволило повысить пропускную способность компьютерных систем, т.е. уменьшить среднее время ожидания результатов работы процессов. Но любой отдельно взятый процесс в мультипрограммной системе никогда не может быть выполнен быстрее, чем при выполнении в однопрограммном режиме на том же вычислительном комплексе. Тем не менее, если алгоритм решения задачи обладает определенным внутренним параллелизмом, мы могли бы ускорить его работу, организовав взаимодействие нескольких процессов. Рассмотрим следующий пример. Пусть у нас есть следующая программа на некотором псевдоязыке программирования:

```
Ввод массива a
Ввод массива b
Ввод массива c
a = a + b
c = a + c
Вывод массива c
```

При выполнении такой программы в рамках одного процесса этот процесс четырежды будет блокироваться, ожидая окончания операций ввода-вывода. Но наш алгоритм обладает внутренним параллелизмом. Вычисление суммы массивов $a + b$ можно было бы делать параллельно с ожиданием окончания операции ввода массива c :

Запрос на ввод массива a	$a=a+b$
Ожидание окончания операции ввода	
Запрос на ввод массива b	
Ожидание окончания операции ввода	
Запрос на ввод массива c	
Ожидание окончания операции ввода	
$c = a + c$	
Вывод массива c	
Ожидание окончания операции вывода	

Такое совмещение операций по времени можно было бы реализовать, используя два взаимодействующих процесса. Для простоты будем полагать, что средством коммуникации между ними служит разделяемая память. Тогда наши процессы могут выглядеть следующим образом:

Процесс 1	Процесс 2
Запрос на ввод массива a Ожидание окончания ввода Запрос на ввод массива b Ожидание окончания ввода	Ожидание ввода массивов a и b
Запрос на ввод массива b Ожидание окончания ввода Ожидание вычисления массива a	$a = a + b$
$c = a + c$ Вывести массив c Ожидание окончания операции вывода	

С одной стороны, предложен конкретный способ ускорения решения задачи. Однако в действительности дело обстоит не так просто. Второй процесс должен быть создан, оба процесса должны сказать операционной системе, что им необходима память, которую они могли бы разделить друг с другом, и, наконец, нельзя забывать о переключении контекста. Поэтому реальное поведение процессов будет выглядеть примерно так:

Процесс 1	Процесс 2
Создать процесс 2	
<i>переключение контекста</i>	

	Выделение общей памяти Ожидание ввода массивов а и b
<i>переключение контекста</i>	
Выделение общей памяти Запрос на ввод массива а Ожидание окончания ввода Запрос на ввод массива b Ожидание окончания ввода	
<i>переключение контекста</i>	
Запрос на ввод массива b Ожидание окончания ввода Ожидание вычисления массива а	$a = a + b$
<i>переключение контекста</i>	
$c = a + c$ Вывести массив с Ожидание окончания операции вывода	

Можно не только не выиграть во времени решения задачи, но даже и проиграть, так как потери времени на создание процесса, выделение общей памяти и переключение контекста могут превысить выигрыш, полученный за счет совмещения операций.

Логично было бы выделить программные модули, выполняющие такие длительные операции, в отдельные подпроцессы (задачи, потоки, нити, треды – от английского термина thread).

С другой стороны, для таких подпроцессов не имеет смысла организовывать полное управление со стороны операционной системы, так как они тесно связаны между собой. Единственный ресурс, который должна выделять для них ОС – время процессора, и в однопроцессорной среде треды разделяют между собой процессорное время так же, как это делают обычные процессы.

В качестве компромисса была предложена концепция *многопоточных* ОС. Согласно этой концепции, процесс может расщепляться на несколько потоков, и при переключении между различными потоками одного процесса не нужно перераспределять закрепленные за этим процессом ресурсы.

Каждый тред выполняется строго последовательно и имеет свой собственный стек и программный счетчик; все же другие ресурсы приложения доступны всем его тредам. Подобно традиционным процессам, тред может находиться в одном из описанных ранее состояний. Но, пока один тред процесса

находится в состоянии блокировки или ожидания, другой тред того же процесса может выполняться.

Все треды одного процесса разделяют виртуальное адресное пространство своего процесса. Это означает, что они разделяют одни и те же глобальные переменные, получают свободный доступ ко всем выделенным ресурсам своего процесса. Взаимодействие между тредами и возможные механизмы конкуренции должны реализовываться на уровне процесса, породившего эти треды.

Поскольку треды одного процесса разделяют существенно больше ресурсов, чем различные процессы, то операции создания нового треда и переключения контекста между тредами одного процесса занимают существенно меньше времени, чем аналогичные операции для процессов в целом. Предложенная схема совмещения работы в терминах треда одного процесса получает право на существование.

Тред 1	Тред 2
Создать тред 2	
<i>переключение контекста тредов</i>	
	Ожидание ввода массивов а и b
<i>переключение контекста тредов</i>	
Запрос на ввод массива а Ожидание окончания ввода Запрос на ввод массива b Ожидание окончания ввода	
<i>переключение контекста тредов</i>	
Запрос на ввод массива b Ожидание окончания ввода Ожидание вычисления массива а	$a = a + b$
<i>переключение контекста тредов</i>	
$c = a + c$ Вывести массив с Ожидание окончания операции вывода	

Различают операционные системы, поддерживающие треды на уровне ядра и на уровне библиотек. Всё, сказанное выше, справедливо для операционных систем, поддерживающих треды на уровне ядра. В них планирование использования процессора происходит в терминах тредов, а управление памятью и другими системными ресурсами остается в терминах процессов. В операционных системах, поддерживающих треды на уровне библиотек пользова-

телей, как планирование процессора, так и управление системными ресурсами осуществляется в терминах процессов. Распределение использования процессора по тредам в рамках выделенного процессу временного интервала осуществляется средствами библиотеки. В таких системах блокирование одного треда приводит к блокированию всего процесса, ибо ядро операционной системы ничего не знает о существовании тредов.

Однако различные треды в рамках одного процесса не настолько независимы, как отдельные процессы. Как уже было сказано ранее, все треды имеют одно и то же адресное пространство. Это означает, что они разделяют одни и те же глобальные переменные. Поскольку каждый тред может иметь доступ к любому доступному адресу, и в частности, может использовать стек другого треда. Между тредами нет полной защиты, потому что, во-первых, это невозможно, а во-вторых, не нужно. Все треды одного процесса всегда решают общую задачу одного пользователя, и описываемый механизм используется для более быстрого решения задачи путем ее распараллеливания. При этом программисту очень важно получить в свое распоряжение удобные средства организации взаимодействия частей одной задачи.

Разбиение процесса на треды повышает эффективность работы системы по сравнению с обычной многозадачной обработкой. Так, в текстовых редакторах логично разделить на различные треды модули, осуществляющие интерфейс с пользователем, модуль форматирования введенного текста, а также модуль грамматического контроля. В этом случае у пользователя этой программы создается впечатление, что обработка вводимого текста осуществляется «на лету».

2.7. Управление процессами и тредами в операционной системе UNIX

В операционной системе UNIX традиционно поддерживается классическая схема мультипрограммирования. Система поддерживает возможность параллельного выполнения нескольких пользовательских программ, каждой из которых соответствует процесс операционной системы. Каждый процесс выполняется в собственной виртуальной памяти, и тем самым процессы защищены один от другого, т.е. один процесс не в состоянии неконтролируемым образом прочитать что-либо из памяти другого процесса или записать в нее. Однако контролируемые взаимодействия процессов допускаются системой, в том числе за счет возможности разделения одного сегмента памяти между виртуальной памятью нескольких процессов.

Необходимо также защищать саму операционную систему от возможности ее повреждения пользовательским процессом. В ОС UNIX это достигается за счет того, что ядро системы работает в собственном "ядерном" вирту-

альном пространстве, к которому не может иметь доступа ни один пользовательский процесс.

Каждый процесс в процессе своего существования находится в одном из двух режимов: режиме ядра и режиме пользователя.

В последние годы в связи с широким распространением так называемых симметричных мультипроцессорных архитектур компьютеров (Symmetric Multiprocessor Architectures - SMP) в ОС UNIX был внедрен механизм *легковесных* процессов (light-weight processes), которые соответствуют рассмотренному в предыдущем параграфе механизму тредов. Иными словами, легковесный процесс - это процесс, выполняющийся в виртуальной памяти «тяжеловесного» (т.е. обладающего отдельной виртуальной памятью) процесса. В принципе, легковесные процессы использовались в операционных системах много лет назад. Однако до настоящего времени в практику программистов так и не были внедрены более безопасные методы параллельного программирования, а реальные возможности мультипроцессорных архитектур для обеспечения распараллеливания нужно было как-то использовать. Наиболее важно то, что для внедрения механизма нитей потребовалась существенная переделка ядра. Разные производители аппаратуры и программного обеспечения стремились как можно быстрее выставить на рынок продукт, пригодный для эффективного использования на SMP-платформах.

Контекст процесса системного уровня («тяжеловесного» процесса) в ОС UNIX состоит из статической и динамических частей. Для каждого процесса имеется одна статическая часть контекста и переменное число динамических частей. Статическая часть контекста процесса системного уровня включает:

- описатель процесса, т.е. элемент таблицы описателей существующих в системе процессов. В него входит, в частности, следующая информация:

- состояние процесса;
- физический адрес в основной или внешней памяти U-области процесса (описание U-области будет дано чуть позже);
- идентификаторы пользователя, от имени которого запущен процесс;
- идентификатор процесса;
- прочую информацию, связанную с управлением процессом.

- U-область (U-area) - индивидуальная для каждого процесса область пространства ядра. Она обладает тем свойством, что хотя U-область каждого процесса располагается в отдельном месте физической памяти, U-области всех процессов имеют один и тот же виртуальный адрес в адресном пространстве ядра. Именно это означает, что какая бы программа ядра не выполнялась, она всегда выполняется как ядерная часть некоторого пользовательского процесса, точнее, того процесса, U-область которого является "видимой" для ядра в данный момент времени. U-область процесса, в частности, содержит:

- указатель на описатель процесса;
- идентификаторы пользователя;
- счетчик времени, в течение которого процесс выполнялся (т.е. занимал процессор) в режиме пользователя и режиме ядра;
- таблицу дескрипторов открытых файлов;
- предельные размеры адресного пространства процесса.

Динамическая часть контекста процесса - это один или несколько стеков, которые используются процессом при его выполнении в режиме ядра. Число ядерных стеков процесса соответствует числу уровней прерывания, поддерживаемых конкретной аппаратурой.

Критерии для определения того, какому готовому к выполнению процессу и когда предоставить ресурс процессора, будут рассмотрены в главе 4, после изучения соответствующих понятий.

Глава 3. Взаимодействие и синхронизация вычислительных процессов

3.1. Взаимодействие процессов, состояние гонки и взаимоисключения

Отвлечемся на некоторое время от операционных систем, процессов и тредов и рассмотрим далее некоторые «активности», под которыми мы будем понимать последовательное выполнение некоторых действий, направленных на достижение определенной цели. Активности могут иметь место в программном и техническом обеспечении, в обычной деятельности людей и животных. Мы будем разбивать активности на некоторые неделимые или атомарные операции.

Неделимые операции могут иметь некоторые внутренние невидимые действия. Мы же называем их неделимыми потому, что считаем одним целым, выполняемыми за раз, без прерывания деятельности.

Пусть имеется две активности

P: a b c

Q: d e f,

где **a, b, c, d, e, f** - атомарные операции. При последовательном выполнении активностей мы получаем следующую последовательность атомарных действий:

PQ: a b c d e f

При исполнении этих активностей псевдопараллельно, в режиме разделения времени они могут расслоиться на неделимые операции с различным их чередованием, то есть может произойти взаимодействие процессов (interleaving). Атомарные операции активностей могут чередоваться всевозможными способами с сохранением своего порядка расположения внутри активностей. Рассмотрим возможные варианты чередования:

a b c d e f

a b d c e f

a b d e c f

a b d e f c

a d b c e f

.....

d e f a b c

Так как псевдопараллельное выполнение двух активностей приводит к чередованию их неделимых операций, то результат псевдопараллельного выполнения может отличаться от результата последовательного выполнения. Пусть есть две активности **P** и **Q**, состоящие из двух атомарных операций каждая:

$$\begin{array}{l|l} \text{P} & \text{Q:} \\ \hline x = 2 & x = 3 \\ y = x-1 & y = x+1 \end{array}$$

В результате их псевдопараллельного выполнения, если переменные x и y являются общими, для активностей возможны четыре разных набора значений для пары (x, y) : **(3, 4)**, **(2, 1)**, **(2, 3)** и **(3, 2)**. Набор активностей (например, программ) *детерминирован*, если всякий раз при псевдопараллельном исполнении для одного и того же набора входных данных он дает одинаковые выходные данные. В противном случае он *недетерминирован*. Выше приведен пример недетерминированного набора активностей. Понятно, что детерминированный набор активностей можно безбоязненно выполнять в режиме разделения времени. Для недетерминированного набора такое исполнение нежелательно.

Можно ли до получения результатов, заранее, определить является ли набор активностей детерминированным или нет? Для этого существуют достаточные условия Бернштейна. Рассмотрим их подробнее применительно к программам с разделяемыми переменными.

Введем наборы входных и выходных переменных программы. Для каждой атомарной операции наборы входных и выходных переменных — это наборы переменных, которые атомарная операция считывает и записывает. Набор входных переменных программы **R(P)** - объединение наборов входных переменных для всех ее неделимых действий. Аналогично, набор выходных переменных программы **W(P)** - объединение наборов выходных переменных для всех ее неделимых действий. Например, для фрагмента программы

P: $x = u + v$
 $y = x * w$

получаем **R(P) = {u, v, x, w}**, **W(P) = {x, y}**. Переменная x присутствует как в **R(P)**, так и в **W(P)**.

Теперь сформулируем условия Бернштейна.

Если для двух активностей **P** и **Q**:

- пересечение **W(P)** и **W(Q)** пусто,
- пересечение **W(P)** с **R(Q)** пусто,
- пересечение **R(P)** и **W(Q)** пусто,

то выполнение **P** и **Q** детерминировано.

Повторим, что условия Бернштейна являются лишь достаточными для определения детерминированности набора из двух активностей. Если они не

выполняются, сделать какой-либо вывод о детерминированности или ее отсутствии нельзя.

Условия Бернштейна информативны, но слишком жестки. Они требуют практически невзаимодействующих процессов. Хотелось бы, чтобы детерминированный набор образовывали активности, совместно использующие информацию и обменивающиеся ей. Для этого необходимо ограничить число возможных чередований атомарных операций, исключив некоторые чередования с помощью механизмов синхронизации выполнения программ, обеспечив тем самым упорядоченный доступ программ к некоторым данным.

Про недетерминированный набор программ (и активностей вообще) говорят, что он находится в *состоянии гонки* (race condition, состоянии состязания). В приведенном выше примере процессы состязаются за вычисление значений переменных x и y .

Задачу упорядоченного доступа к разделяемым данным (устранение состояния гонки), в том случае, если не важна его очередность, можно решить, если обеспечить каждому процессу эксклюзивное право доступа к этим данным. Каждый процесс, обращающийся к разделяемым ресурсам, исключает для всех других процессов возможность одновременного с ним общения с этими ресурсами, если это может привести к недетерминированному поведению набора процессов. Такой прием называется *взаимоисключением* (mutual exclusion). Если очередность доступа к разделяемым ресурсам важна для получения правильных результатов, то одними взаимоисключениями уже не обойтись.

3.2. Критические ресурсы и критические секции

Как уже отмечалось, мультипрограммирование предполагает, что у нас может быть несколько параллельных процессов, т.е. процессов, одновременно находящихся в одном из активных состояний. Эти процессы могут быть *независимыми* либо *взаимодействующими*.

Независимые процессы не обращаются к общим данным, либо делают это только в режиме чтения. Таким образом, независимые процессы не влияют на результаты работы друг друга; они могут только являться причиной задержек в работе, поскольку вынуждены разделять ресурсы системы.

Взаимодействующие процессы совместно используют некоторые общие данные, и выполнение одного процесса может повлиять на результаты выполнения другого.

В свою очередь, взаимодействующие процессы могут быть разделены на *конкурирующие* и *сотрудничающие*. Конкурирующие процессы в принципе тоже работают независимо друг от друга, и могут даже не знать о существовании других процессов. Однако в процессе своей работы конкурирующие процессы могут требовать т.н. *критические ресурсы* - такие ресурсы,

которые в каждый конкретный момент времени могут быть предоставлены только одному процессу.

Итак, процессам часто нужно взаимодействовать друг с другом, например, один процесс может передавать данные другому процессу, или несколько процессов могут обрабатывать данные из общего файла. Во всех этих случаях возникает проблема синхронизации процессов, которая может решаться их приостановкой и активизацией, организацией очередей, блокированием и освобождением ресурсов.

Пренебрежение вопросами синхронизации процессов, выполняющихся в режиме мультипрограммирования, может привести к их неправильной работе.

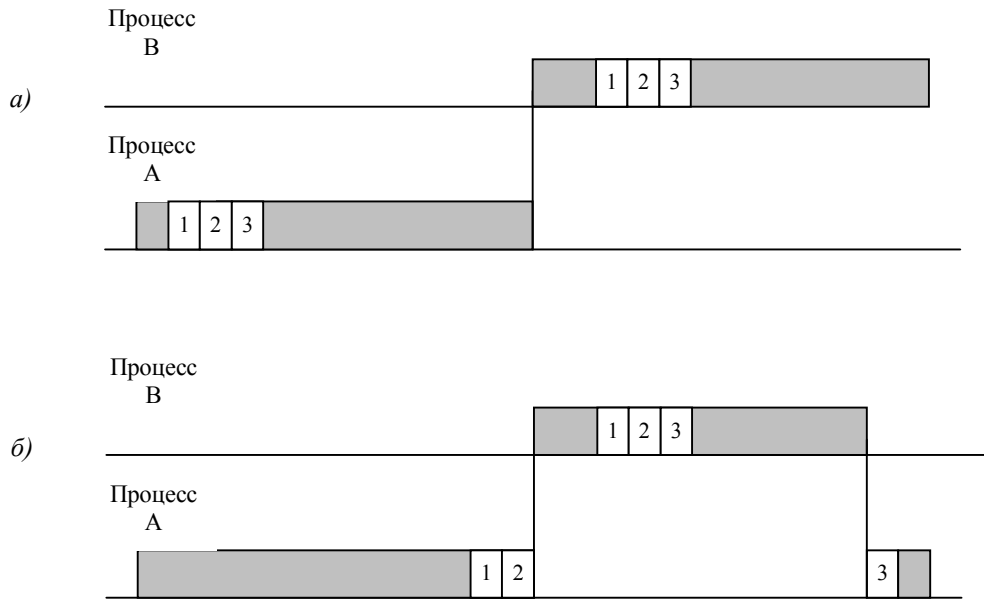


Рис. 3.1. Взаимодействие двух процессов с критическим ресурсом: а) нормальное; б) с неправильным изменением значения глобальной переменной

Рассмотрим классический пример взаимодействия конкурирующих процессов, показанный на рис. 3.1. Пусть каждое посещение веб-сайта обслуживается отдельным процессом. Кроме того, нам необходимо вести общий счетчик посещений. Это означает, что все обслуживающие посещения сайта процессы должны выполнить оператор

$R := R + 1;$

где R – глобальная переменная-счетчик. Предположим также, что компилятор преобразует указанный оператор в команды

```

mov ax, R          ; (1)
inc ax             ; (2)
mov R, ax          ; (3)

```

(важно, чтобы указанный оператор состоял более чем из одной машинной команды). Если два конкурирующих процесса запускаются так, что последовательность команд (1)-(3) не прерывается, то проблем не возникает (см. рис. 3.1а). Однако если процессы прерываются так, как показано на рис.3.1б, значение R увеличивается лишь на единицу. Такая ситуация, как уже было сказано в предыдущем параграфе, иногда называется состоянием гонок (race condition). Об этом уже говорилось в начале данной главы.

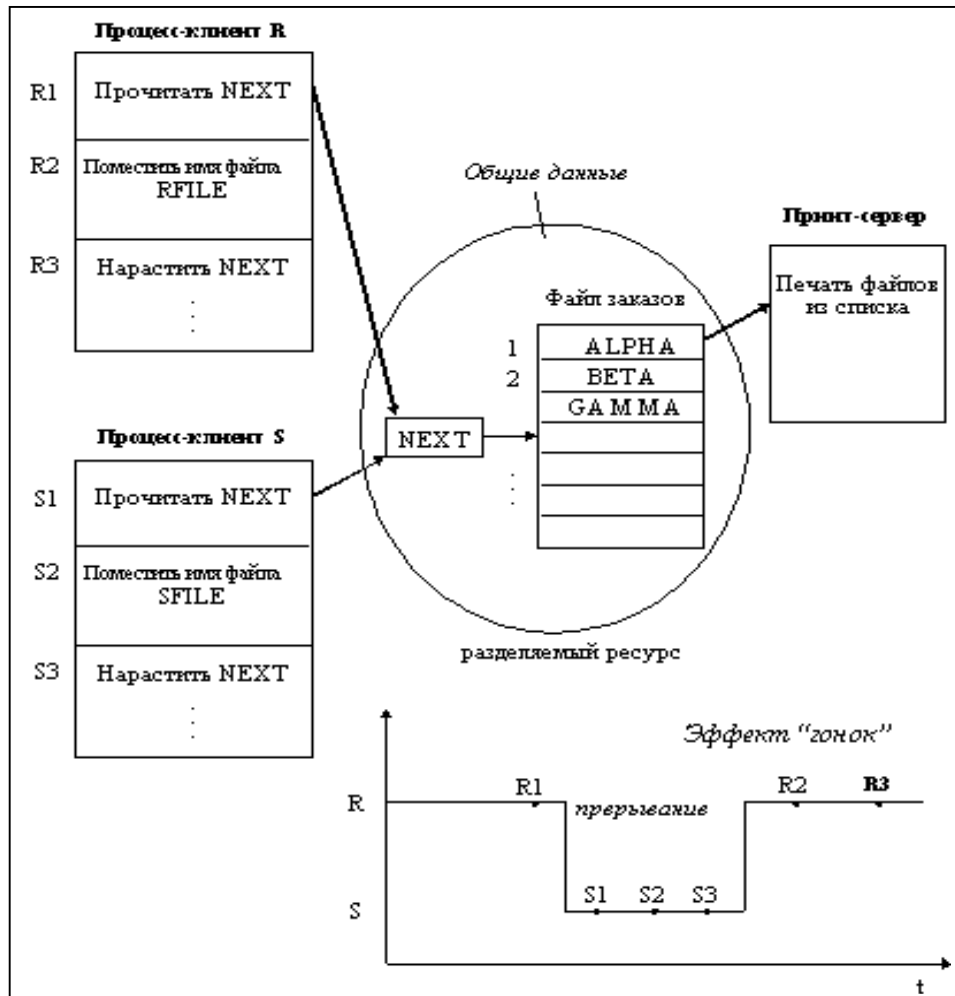


Рис. 3.2. Взаимодействие программ с принт-сервером

В качестве другого примера рассмотрим программу печати файлов (принт-сервер), работа которой проиллюстрирована рисунком 3.2. Эта программа печатает по очереди все файлы, имена которых последовательно (в порядке поступления) записываются в специальный общедоступный файл заказов другими программами. Переменная NEXT, также доступная всем процессам-клиентам, содержит номер первой свободной для записи имени файла позиции файла заказов. Процессы-клиенты читают эту переменную, записывают в соответствующую позицию файла заказов имя своего файла и наращивают значение NEXT на единицу. Предположим, что в некоторый момент процесс R решил распечатать свой файл, для этого он прочитал значение переменной NEXT, значение которой предположим равным 4. Процесс запомнил это значение, но поместить имя файла не успел, так как его выполнение было прервано. Очередной процесс S, желающий распечатать файл, прочитал то же самое значение переменной NEXT, поместил в четвертую позицию имя своего файла и нарастил значение переменной на единицу. Когда в очередной раз управление будет передано процессу R, то он, продолжая свое выполнение, в полном соответствии со значением текущей свободной позиции, полученным во время предыдущей итерации, запишет имя файла также в позицию 4, поверх имени файла процесса S.

Таким образом, процесс S никогда не увидит свой файл распечатанным. Сложность проблемы синхронизации состоит в нерегулярности возникающих ситуаций: в предыдущем примере можно представить и другое развитие событий: были потеряны файлы нескольких процессов или, напротив, не был потерян ни один файл. В данном случае все определяется взаимными скоростями процессов и моментами их прерывания. Поэтому отладка взаимодействующих процессов является сложной задачей.

Чтобы предотвратить некорректное исполнение конкурирующих процессов вследствие нерегламентированного доступа к разделяемым переменным, необходимо ввести механизм *взаимного исключения*, который не позволил бы двум процессам одновременно обращаться к критическому ресурсу.

Суть взаимного исключения можно выразить следующей фразой: до окончания обращения одного процесса к общим переменным следует исключить возможность обращения к этим переменным других процессов. Те участки процессов, в которых происходит обращение к критическим ресурсам, называются *критическими секциями* или *критическими интервалами*.

Следует сказать, что размер критической секции должен быть больше одной машинной команды. Это связано с тем, что все вычислительные системы имеют встроенные средства *блокировки памяти*, которые предотвращают исполнение двух и более команд, обращающихся к одной и той же ячейке памяти. Этого средства оказывается достаточно для непосредственной реализации взаимного исключения, ограниченного одной командой.

Таким образом, ход процесса состоит из последовательности критических и некритических интервалов. При этом только один из выполняющихся процессов может войти в свою критическую секцию. Схема работы с критическими секциями для первого из примеров показана на рисунке 3.3.

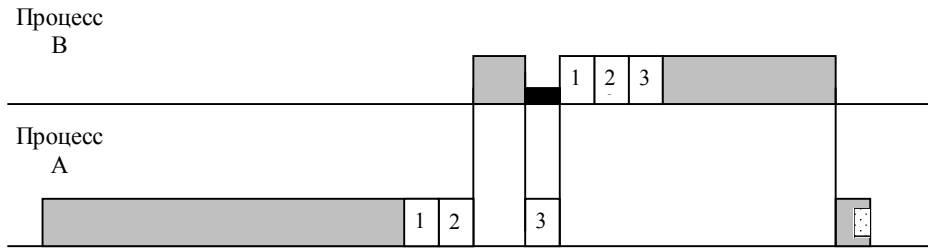


Рис. 3.3. Реализация критической секции (темным цветом показано ожидание входа в критический интервал для процесса В)

Простейший способ обеспечить взаимное исключение - позволить процессу, находящемуся в критической секции, запрещать все прерывания. Однако этот способ непригоден, так как опасно доверять управление системой пользовательскому процессу; он может надолго занять процессор, а при крахе процесса в критической области крах потерпит вся система, потому что прерывания никогда не будут разрешены.

Другим способом является использование блокирующих переменных. С каждым разделяемым ресурсом связывается двоичная переменная, которая принимает значение 1, если ресурс свободен (то есть ни один процесс не находится в данный момент в критической секции, связанной с данным процессом), и значение 0, если ресурс занят. На рисунке 3.4 приведен фрагмент алгоритма процесса, использующего для реализации взаимного исключения доступа к разделяемому ресурсу D блокирующую переменную $F(D)$. Перед входом в критическую секцию процесс проверяет, свободен ли ресурс D. Если он занят, то проверка циклически повторяется, если свободен, то значение переменной $F(D)$ устанавливается в 0, и процесс входит в критическую секцию. После того, как процесс выполнит все действия с разделяемым ресурсом D, значение переменной $F(D)$ снова устанавливается равным 1.

Если все процессы написаны с использованием вышеописанных соглашений, то взаимное исключение гарантируется. Следует заметить, что операция проверки и установки блокирующей переменной должна быть неделимой. Пусть в результате проверки переменной процесс определил, что ресурс свободен, но сразу после этого, не успев установить переменную в 0, был прерван. За время его приостановки другой процесс занял ресурс, вошел в свою критическую секцию, но также был прерван, не завершив работы с разделяемым ресурсом. Когда управление было возвращено первому процессу, он, считая ресурс свободным, установил признак занятости и начал выполнять

свою критическую секцию. Таким образом, был нарушен принцип взаимного исключения, что потенциально может привести к нежелательным последствиям. Во избежание таких ситуаций в системе команд машины желательно иметь единую команду "проверка-установка", или же реализовывать системными средствами соответствующие программные примитивы, которые бы запрещали прерывания на протяжении всей операции проверки и установки.

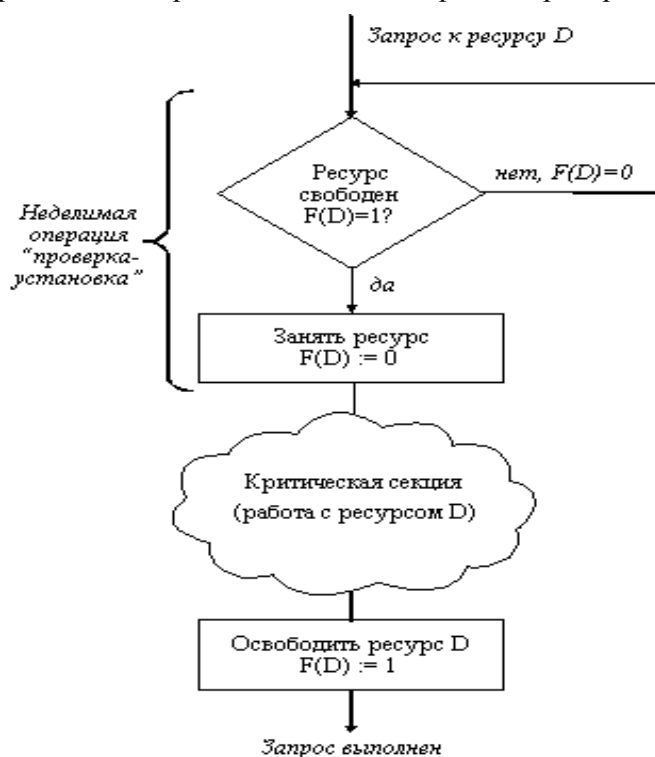


Рис. 3.4. Реализация критических секций с использованием блокирующих переменных

Сотрудничающие процессы обмениваются данными так, что результаты вычислений одного процесса в явном виде передаются другому. Взаимодействие сотрудничающих процессов удобнее всего рассматривать в схеме «поставщик – потребитель» (producer – consumer). Поставщик – это процесс, который отправляет порции информации (сообщения) другому процессу – потребителю.

Один из методов, применяемых при передаче сообщений от одного процесса к другому, состоит в том, что формируется *очередь сообщений*, каждый элемент которой может содержать одно сообщение. Процесс-поставщик добавляет свои сообщения в очередь, а процесс-потребитель – забирает их оттуда. Такое решение, хотя и кажется тривиальным, тем не менее, требует син-

хронизации процессов. Так, проверку на полноту очереди выполняет поставщик, а проверку пустоты – потребитель. Кроме того, необходимо правильно обработать одновременный прием-передачу сообщений. В этом случае переменные, отвечающие за работу очереди, являются критическими ресурсами, и для сотрудничающих процессов также необходимо выработать механизм взаимного исключения.

Если выполняющиеся процессы разделяют несколько критических ресурсов, то для каждого ресурса определяется своя критическая секция, причем критические секции разных ресурсов могут пересекаться в рамках одного процесса.

Обеспечение взаимоисключения является одной из ключевых проблем параллельного программирования. При этом можно перечислить следующие требования к критическим секциям:

- в любой момент времени только один процесс может находиться в своей критической секции;
- ни один процесс не должен находиться в своей критической секции бесконечно долго;
- ни один процесс не должен ждать бесконечно долго входа в свой критический интервал. В частности, никакой процесс, находящийся вне своей критической секции, не должен блокировать критическую секцию другого процесса. Кроме того, если два процесса желают одновременно войти в свои критические секции, то решение вопроса о том, кто первым войдет туда, не должно откладываться бесконечно долго;
- если процесс, находящийся в своем критическом интервале, завершается (естественным путем или аварийно), то механизм взаимного исключения должен быть отменен, с тем чтобы другие процессы смогли войти в свои критические секции.

Примечание. Иногда можно увидеть следующее определение критической секции: это участок процесса, в течение которого процессор не может быть передан другому процессу. Это определение, конечно же, решает проблему взаимоисключения, но чересчур кардинально. В случае длительных по времени критических секций производительность вычислительной системы при использовании этой концепции резко снижается.

Итак, чтобы исключить состояние гонок по отношению к некоторому ресурсу, необходимо организовать работу так, чтобы в каждый момент времени только один процесс мог находиться в своей критической секции, связанной с этим ресурсом. Иными словами, необходимо обеспечить реализацию взаимоисключения для критических секций программ. С практической точки зрения это означает, что по отношению к другим процессам, участвующим во взаимодействии, критическая секция выполняется как атомарная операция.

Организация взаимоисключения для критических участков, конечно, позволит избежать возникновения состояния гонок, но не является достаточной для правильной и эффективной параллельной работы кооперативных процессов. Сформулируем пять условий, которые должны выполняться для хорошего программного алгоритма организации взаимодействия процессов, имеющих критические участки, если процессы могут проходить их в произвольном порядке:

1. Задача должна быть решена чисто программным способом на обычной машине, не имеющей специальных команд взаимоисключения. При этом предполагается, что основные инструкции языка программирования являются атомарными операциями.

2. Не должно существовать никаких предположений об относительных скоростях выполняющихся процессов или числе процессоров, на которых они исполняются.

3. Если процесс *P* исполняется в своем критическом участке, то не существует никаких других процессов, связанных с этим же ресурсом, которые исполняются в своих соответствующих критических секциях. Это условие, о котором упоминалось в начале данной главы, получило название условия взаимоисключения.

4. Процессы, которые находятся вне своих критических участков и не собираются войти в них, не могут препятствовать другим процессам войти в их собственные критические участки. Если нет процессов в критических секциях, и имеются процессы, желающие войти в них, то только те процессы, которые не находятся в состоянии ожидания, должны принимать решение о том, какой процесс войдет в свою критическую секцию. Такое решение не должно приниматься бесконечно долго. Это условие получило название условия прогресса.

5. Не должно возникать бесконечного ожидания для входа процесса в свой критический участок. От того момента, когда процесс запросил разрешение на вход в критическую секцию, и до того момента, когда он это разрешение получил, другие процессы могут пройти через свои критические участки лишь ограниченное число раз. Это условие получило название условия ограниченного ожидания.

Примечание [A1]: Объяснить понятие

3.3. Реализация взаимного исключения для конкурирующих процессов

Реализация механизмов взаимного исключения может быть выполнена как аппаратными, так и программными средствами. К программным средствам, в

частности, относится уже известный механизм блокировки памяти. В дальнейшем мы будем считать, что оператор

переменная := константа;

выполняется за одну машинную команду и в соответствии с этим принципом не требует дополнительной синхронизации.

Что касается алгоритмов синхронизации, будем рассматривать их в порядке их появления и по возрастанию сложности.

3.3.1. Алгоритмы синхронизации, использующие механизм ожидания

В настоящее время эти алгоритмы представляют лишь теоретический интерес, поскольку их общим недостатком является то, что процесс, ожидающий входа в критическую секцию, находится в бесконечном цикле и напрасно занимает время процессора, оперативную память и другие ресурсы.

Реализация критических секций с использованием блокирующих переменных имеет существенный недостаток: в течение времени, когда один процесс находится в критической секции, другой процесс, которому требуется тот же ресурс, будет выполнять рутинные действия по опросу блокирующей переменной, бесполезно тратя процессорное время. Для устранения таких ситуаций может быть использован так называемый аппарат событий. С помощью этого средства могут решаться не только проблемы взаимного исключения, но и более общие задачи синхронизации процессов. В разных операционных системах аппарат событий реализуется различными способами.

Далее при объяснении многих алгоритмов будем считать, что в вычислительной системе находятся два конкурирующих процесса, и будем использовать следующую схему записи:

Запуск двух параллельных процессов;	
Процесс А;	процесс В;
Завершение процессов;	

Процессы, записанные в соседних столбцах таблицы, выполняются параллельно, причем нет никаких соглашений о взаимных скоростях выполнения каждого процесса. Каждый процесс представляет собой бесконечный цикл, включающий критическую и некритическую секцию. Завершение процесса происходит в некритической секции.

Поочередный вход в критическую секцию. Согласно этому алгоритму, вход в критическую секцию обоих процессов контролируется одной переменной С.

Var C: byte; Begin C := 1;	
while true do begin while C=2 do; {критическая секция} C := 2; {некритическая секция} end;	while true do begin while C=1 do; {критическая секция} C := 1; {некритическая секция} end;

Очевидно, этот алгоритм приведет к блокированию одного из процессов, если другой достаточно долго будет находиться в некритической секции.

Использование своих переключателей для каждого процесса. Пусть с каждым из процессов: А и В - связана своя глобальная логическая переменная-переключатель с именем СА и СВ соответственно. Переменная СА равна true, если процесс А находится в своей критической секции, и false в противном случае. Прежде чем войти в свой критический интервал, процесс проверяет значение переключателя для другого процесса и переходит в бесконечный цикл до сброса этого переключателя. Рассмотрим схему этого алгоритма.

Var CA, CB: Boolean; Begin CA := false; CB := false;	
while true do begin while CB do; /* CA := true; /** {критическая секция} CA := false; {некритическая секция} end;	while true do begin while CA do; CB := true; {критическая секция} CB := false; {некритическая секция} end;

Данный алгоритм не гарантирует полного выполнения условия нахождения только одного процесса внутри своего критического интервала. Дело в том, что между выполнением строк (*) и (**) процесс А может прерваться, и за это время процесс В успеет проскочить свой некритический интервал, проверку СА, и войдет в критическую секцию. После этого, получив доступ к процессору, туда же попадет и процесс А.

Для решения этой проблемы усилим взаимное исключение, включив переключатель процесса до проверки другого переключателя:

Var CA, CB: Boolean; Begin CA := false; CB := false;	
while true do begin CA := true; while CB do; {критическая секция} CA := false; {некритическая секция} end;	while true do begin CB := true; while CA do; {критическая секция} CB := false; {некритическая секция} end;

Однако в этом случае оба процесса могут одновременно установить свои переключатели в true и, как следствие, войдут в бесконечный цикл ожидания.

Алгоритм Деккера. Этот алгоритм использует идеи первого и третьего из представленных ранее алгоритмов. Так, переключатели CA и CB, как и ранее, сигнализируют о том, что процессы A и B соответственно вошли в критические интервалы. Переменная Q показывает, чья очередь войти в критическую секцию, если оба процесса желают это сделать.

Var CA, CB: Boolean; Q: byte; Begin CA := false; CB := false; Q := 1;	
while true do begin CA := true; 1: if CB then if Q=1 then goto 1 else begin CA := false; while Q=2 do; end else begin {критическая секция} Q := 2; CA := false; end; {некритическая секция} end;	while true do begin CB := true; 2: if CA then if Q=2 then goto 2 else begin CB := false; while Q=1 do; end else begin {критическая секция} Q := 1; CB := false; end; {некритическая секция} end;

Описанный алгоритм, хотя и решает исчерпывающим образом проблему взаимного исключения, в настоящее время практически не используется из-за своей сложности (особенно при попытке распространить его на 3 и более процессов).

3.3.2. Использование семафоров

Понятие «семафор» было введено в начале 70-х годов. *Семафор* – это специальная переменная S , которая принимает значение «открыт» и «закрыт» и доступна параллельным процессам для выполнения только для выполнения специальных операций $P(S)$ и $V(S)$. Как уже было сказано ранее, эти операции должны быть неделимыми.

Первая из этих операций проверяет, свободен ли ресурс, точнее, открыт ли семафор, связанный с этим ресурсом. Если да, то процесс закрывает семафор и проходит в свою критическую секцию. В противном случае процесс, запросивший операцию $P(S)$, переводится в состояние пассивного ожидания.

Операция $V(S)$ выполняется после выхода из критической секции. Она открывает семафор и при необходимости переводит один из заблокированных процессов в состояние готовности к выполнению.

Кроме этого, необходима специальная операция *инициализации* семафора, которая гарантирует, что в начале работы семафор примет конкретное значение (будет открыт либо закрыт).

Реализация семафора в простейшем случае (т.н. *двоичный* семафор) выглядит так: семафор представляет собой целую переменную и принимает значения 1 (семафор открыт), 0 (семафор закрыт), $-N$ (семафор закрыт, и в очереди к нему находится N процессов).

Таким образом, схема взаимодействия конкурирующих процессов будет выглядеть следующим образом:

Var S : TSemaphore; Begin InitSemaphore(S , 1);	
while true do begin $P(S)$; {критическая секция} $V(S)$; {некритическая секция} end;	while true do begin $P(S)$; {критическая секция} $V(S)$; {некритическая секция} end;

Реализация описанных выше операций выглядит так:

$P(S)$	$S := S - 1$; if $S < 0$ then {заблокировать процесс, поставив в очередь к S }
--------	---

V(S)	if S<0 then {перевести один из процессов, находящихся в очереди к S, в состояние готовности} S := S+1;
InitSemaphore(a)	S := a;

В этом случае процесс, стоявший в очереди к семафору, не должен повторно выполнять операцию P(S), т.к. в этом случае возможно возникновение тупиковой ситуации. Действительно, после того как процесс 1 начал выполнять критическую секцию, а процесс 2 «уснул», значение семафора стало равным -1. В момент разблокировки процесса 2 значение семафора будет равным нулю, и повторное выполнение операции P(S) приведет к тому, что процессы 1 и 2 снова «уснут», хотя критическая секция свободна!

Возможен и другой вариант реализации семафора:

P(S)	if S>=1 then S := S-1 Else {заблокировать процесс, поставить его в очередь к S}
V(S)	if S<0 then {перевести один из процессов, находящихся в очереди к S, в состояние готовности} S := S+1;

Но теперь повторное выполнение операции P(S) после разблокировки обязательно!

Кроме рассмотренных операций, для семафоров может быть введена условная операция, подобная P, уменьшающая значение семафора и возвращающая логическое значение «истина» в том случае, когда значение семафора остается положительным. Если в результате операции значение семафора должно стать отрицательным или нулевым, никаких действий над ним не производится и операция возвращает логическое значение «ложь». Такая операция получила название CP (conditional P).

Рассмотрим еще один феномен, связанный с использованием семафоров в однопроцессорной системе. Предположим, что два процесса, A и B, конкурируют за семафор. Процесс A обнаруживает, что семафор свободен и что процесс B приостановлен; значение семафора равно -1. Когда с помощью операции V процесс A освобождает семафор, он выводит тем самым процесс B из состояния приостановки и вновь делает значение семафора нулевым. Теперь предположим, что процесс A, по-прежнему выполняясь в режиме ядра, пытается снова заблокировать семафор. Производя операцию P, процесс приостановится, поскольку семафор имеет нулевое значение, несмотря на то, что ре-

курс пока свободен. Системе придется «раскошелиться» на дополнительное переключение контекста. С другой стороны, если бы блокировка была реализована на основе схемы с ожиданиями, описанной в п. 3.3.1, процесс А получил бы право на повторное использование ресурса, поскольку за это время ни один из процессов не смог бы заблокировать его. Для этого случая схема с ожиданиями более подходит, чем схема с использованием семафоров.

Использование семафоров, несмотря на свою кажущуюся простоту, не лишено недостатков. Во-первых, семафорные операции P и V чересчур примитивны, они не связаны с каким-либо конкретным ресурсом, и для организации эффективной защиты требуется одновременно создать несколько семафоров, а это снижает читабельность программы. Во-вторых, перевод процессов из одного состояния в другое – это все-таки прерогатива операционной системы!

Для устранения этих недостатков Хоаром было введено понятие *мониторов* – совокупности системных переменных и повторно входных процедур доступа к ним. В терминах ООП монитор уместно считать системным объектом, имеющим свои защищенные поля и методы. Каждый монитор, как правило, связан с одним защищаемым ресурсом. Процессы могут, например, выполнять методы монитора Request (выдать запрос на получение ресурса) и Release (освободить ресурс). Вход в монитор осуществляется под жестким контролем: только один процесс должен находиться внутри монитора. Поэтому попытка выполнить метод Request может привести к блокированию процесса. Выполнение метода Release, наоборот, влечет деблокирование одного из находящихся в очереди к ресурсу процессов.

3.3.3. Пример использования семафоров

Рассмотрим неожиданный (но лишь на первый взгляд) пример использования операций, подобных семафорным. Пусть в среде Windows должен работать только один экземпляр приложения MS-DOS, и нам необходимо предотвратить повторный запуск этого приложения. Для этого создадим файл-семафор, имя которого будет свидетельствовать о его состоянии: 0.xxx – семафор открыт, 1.xxx – семафор закрыт. Содержимое этого файла значения не имеет. Для запуска приложения используется следующий bat-файл (предполагается, что защищаемое приложение находится в каталоге C:\mydosapp и имеет имя dosapp):

```
@echo off
c:
cd \mydosapp
if exist 1.xxx goto m1
if not exist 0.xxx copy nul nul >0.xxx
ren 0.xxx 1.xxx
dosapp
```



```
ren 1.xxx 0.xxx
exit
:m1
echo Повторный запуск программы или программа заблокирована!
pause
```

Предложенный метод имеет существенный недостаток: при зависании и последующей перезагрузке Windows во время работы DOS-приложения файл-семафор остается закрытым! Приходится создавать специальный bat-файл для разблокировки:

```
@echo off
c:
cd \mydosapp
if exist 1.xxx goto m1
if not exist 0.xxx copy nul nul >0.xxx
exit
:m1
echo Программа заблокирована или уже работает!
choice Разблокировать ее?
if errorlevel 2 exit
del 1.xxx
if not exist 0.xxx copy nul nul >0.xxx
```

3.3.4. Примеры алгоритмов с использованием семафоров в UNIX

Далее рассмотрим несколько алгоритмов ядра, реализованных с использованием семафоров. Алгоритм *выделения буфера* иллюстрирует сложную схему блокирования, на примере алгоритма *ожидания* показана синхронизация выполнения процессов, схема *блокирования драйверов* реализует подход к решению данной проблемы, а метод решения проблемы *холостой работы процессора* показывает, что нужно сделать, чтобы избежать конкуренции между процессами.

Выделение буфера. Алгоритм выделения буфера работает с тремя структурами данных: заголовком буфера, очередью буферов и списком свободных буферов. Ядро связывает семафор со всеми экземплярами каждой структуры. Другими словами, если у ядра имеются в распоряжении 200 буферов, заголовок каждого из них включает в себя семафор, используемый для захвата буфера; когда процесс выполняет над семафором операцию P, другие процессы, тоже пожелавшие захватить буфер, приостанавливаются до тех пор, пока первый процесс не исполнит операцию V. У каждой очереди буферов (хеш-очереди) также имеется семафор, блокирующий доступ к этой очереди. В однопроцессорной системе блокировка очереди не нужна, ибо процесс никогда не переходит в состояние приостановки, оставляя очередь в несогласованном

(неупорядоченном) виде. В многопроцессорной системе, тем не менее, возможны ситуации, когда с одной и той же очередью работают два процесса; в каждый момент времени семафор открывает доступ к очереди только для одного процесса. По тем же причинам и список свободных буферов нуждается в семафоре для защиты информации, содержащейся в нем, от искажения.

Просматривая буфер в поисках указанного блока, ядро с помощью операции Р захватывает семафор, принадлежащий очереди. Если над семафором уже кем-то произведена операция данного типа, текущий процесс приостанавливается до тех пор, пока процесс, захвативший семафор, не освободит его, выполнив операцию V. Когда текущий процесс получает право исключительного контроля над очередью, он приступает к поиску подходящего буфера. Предположим, что буфер находится в очереди. Ядро (процесс А) пытается захватить буфер, но если оно использует операцию Р и если буфер уже захвачен, ядру придется приостановить свою работу, оставив очередь заблокированной и не допуская таким образом обращений к ней со стороны других процессов, даже если последние ведут поиск незахваченных буферов. Пусть вместо этого процесс А захватывает буфер, используя операцию СР; если операция завершается успешно, буфер становится открытым для процесса. Процесс А захватывает семафор, принадлежащий списку свободных буферов, выполняя операцию СР, поскольку семафор захватывается на непродолжительное время и, следовательно, приостанавливать свою работу, выполняя операцию Р, процесс просто не имеет возможности. Ядро убирает буфер из списка свободных буферов, снимает блокировку со списка и с очереди и возвращает захваченный буфер. Предположим, что операция СР над буфером завершилась неудачно из-за того, что семафор, принадлежащий буферу, оказался захваченным. Процесс А освобождает семафор, связанный с очередью, и приостанавливается, пытаясь выполнить операцию Р над семафором буфера. Операция Р над семафором будет выполняться, несмотря на то, что операция СР уже потерпела неудачу. По завершении выполнения операции процесс А получает власть над буфером. В оставшейся части алгоритма предполагается, что буфер и очередь захвачены. Процесс А теперь пытается захватить очередь. Поскольку очередность захвата здесь (сначала семафор буфера, потом семафор очереди) обратна вышеуказанной очередности, над семафором выполняется операция СР. Если попытка захвата заканчивается неудачей, имеет место обычная обработка, требующаяся по ходу задачи. Но если захват удастся, ядро не может быть уверено в том, что захвачен корректный буфер, поскольку содержимое буфера могло быть ранее изменено другим процессом, обнаружившим буфер в списке свободных буферов и захватившим на время его семафор. Процесс А, ожидая освобождения семафора, не имеет ни малейшего представления о том, является ли интересующий его буфер тем буфером, который ему нужен, и поэтому он должен убедиться в правильности содержимого буфера; если проверка дает отрицательный результат, алгоритм

запускается сначала. Если содержимое буфера корректно, процесс А завершает выполнение алгоритма.

Ожидание. Во время выполнения системной функции ожидания процесс приостанавливает свою работу до момента завершения выполнения своего потомка. В многопроцессорной системе перед процессом встает задача не упустить при выполнении алгоритма ожидания своего потомка, который уже прекратил существование. Например, если в то время, пока на одном процессоре процесс-родитель запускает функцию ожидания, на другом процессоре его потомок завершил свою работу, родителю нет необходимости приостанавливать свое выполнение в ожидании завершения второго потомка. В каждой записи таблицы процессов имеется семафор, именуемый `zombie_semaphore` и имеющий в начале нулевое значение. Этот семафор используется при организации взаимодействия `wait/exit`. Когда потомок завершает работу, он выполняет над семафором своего родителя операцию V, выводя родителя из состояния приостановки, если тот перешел в это состояние во время исполнения функции `wait` (вместо захвата хеш-очереди в этом месте можно было бы установить соответствующий флаг, проверяемый далее перед выполнением операции V, но проиллюстрируем схему захвата семафоров в обратной последовательности). Если потомок завершился раньше, чем родитель запустил функцию `wait`, этот факт будет обнаружен родителем, который тут же выйдет из состояния ожидания. Если оба процесса исполняют функции `exit` и `wait` параллельно, но потомок исполняет функцию `exit` уже после того, как родитель проверил его статус, операция V, выполненная потомком, воспрепятствует переходу родителя в состояние приостановки. В худшем случае процесс-родитель просто повторяет цикл лишний раз.

Блокирование драйверов. В многопроцессорной реализации вычислительной системы UNIX семафоры в структуру загрузочного кода драйверов не включаются, а операции типа P и V выполняются в точках входа в каждый драйвер. Интерфейс, реализуемый драйверами устройств, характеризуется очень небольшим числом точек входа (на практике их около 20). Защита драйверов осуществляется на уровне точек входа в них:

```
P(семафор драйвера);  
открыть драйвер;  
V(семафор драйвера);
```

Если для всех точек входа в драйвер использовать один и тот же семафор, но при этом для разных драйверов - разные семафоры, критический участок программы драйвера будет исполняться процессом монопольно. Семафоры могут назначаться как отдельному устройству, так и классам устройств. Так, например, отдельный семафор может быть связан и с отдельным физическим терминалом и со всеми терминалами сразу. В первом случае быстроедействие

системы выше, ибо процессы, обращающиеся к терминалу, не захватывают семафор, имеющий отношение к другим терминалам, как во втором случае. Драйверы некоторых устройств, однако, поддерживают внутреннюю связь с другими драйверами; в таких случаях использование одного семафора для класса устройств облегчает понимание задачи. В качестве альтернативы в некоторых вычислительных системах предоставлена возможность такого конфигурирования отдельных устройств, при котором программы драйвера запускаются на точно указанных процессорах.

Проблемы возникают тогда, когда драйвер прерывает работу системы и его семафор захвачен: программа обработки прерываний не может быть вызвана, так как иначе возникла бы угроза разрушения данных. С другой стороны, ядро не может оставить прерывание необработанным. Система способна выстраивать прерывания в очередь и ждать момента освобождения семафора, когда вызов программы обработки прерываний не будет иметь опасные последствия.

Фиктивные процессы. Когда ядро выполняет переключение контекста в однопроцессорной системе, оно функционирует в контексте процесса, уступающего управление. Если в системе нет процессов, готовых к запуску, ядро переходит в состояние простоя в контексте процесса, который выполнялся последним. Получив прерывание от таймера или других периферийных устройств, оно обрабатывает его в контексте того же процесса.

В многопроцессорной системе ядро не может простаивать в контексте процесса, выполнявшегося последним. Рассмотрим, что произойдет после того, как процесс, приостановивший свою работу на процессоре А, выйдет из состояния приостановки. Процесс в целом готов к запуску, но он запускается не сразу же по выходе из состояния приостановки, даже несмотря на то, что его контекст уже находится в распоряжении процессора А. Если этот процесс выбирается для запуска процессором В, последний переключается на его контекст и возобновляет его выполнение. Когда в результате прерывания процессор А выйдет из простоя, он будет продолжать свою работу в контексте процесса А до тех пор, пока не произведет переключение контекста. Таким образом, в течение короткого промежутка времени с одним и тем же адресным пространством (в частности, со стеком ядра) будут вести работу (и, что весьма вероятно, производить запись) сразу два процессора.

Решение данной проблемы состоит в создании некоторого фиктивного процесса; когда процессор находится в состоянии простоя, ядро переключается на контекст фиктивного процесса, делая этот контекст текущим для бездействующего процессора. Контекст фиктивного процесса состоит только из стека ядра; этот процесс не является выполнимым и не выбирается для запуска. Поскольку каждый процессор простаивает в контексте своего собственного фиктивного процесса, навредить друг другу процессоры уже не могут.

3.4. Взаимодействие сотрудничающих процессов

3.4.1. Реализация взаимного исключения

Сотрудничающие процессы, как уже было сказано ранее, не только работают с одними и теми же ресурсами, но и обмениваются информацией друг с другом. Поэтому помимо рассмотренных ранее алгоритмов, обеспечивающих их синхронизацию при борьбе за ресурсы, необходимо разработать новые. В этом случае будем пользоваться описанными ранее операциями $P(S)$ и $V(S)$.

Рассмотрим несколько классических схем взаимодействия сотрудничающих процессов.

«Поставщики» - «потребители». Как уже было сказано ранее, «поставщик» – это процесс, который время от времени отправляет порции информации (сообщения) другому процессу - «потребителю». Конкретный механизм передачи сообщений будет рассмотрен нами позднее в этом параграфе, сейчас лишь ограничимся тем, что основой для его реализации является очередь. Если она не пуста и не заполнена до конца, то в принципе синхронизация и не требуется (вспомните принципы работы с очередью!). Но если поставщик записывает первое сообщение, а потребитель в это время пытается его прочитать, то возникают проблемы!

Для реализации механизма синхронизации в этой схеме предлагается следующий алгоритм:

Var S, SFree, SFull: Tsemaphore; Begin InitSemaphore(S, 1); InitSemaphore(SFree, N); // N – объем очереди InitSemaphore(SFull, 0);	
ПОСТАВЩИК: while true do begin {приготовить сообщение} P(SFree); P(S); {послать сообщение} V(SFull); V(S); end;	ПОТРЕБИТЕЛЬ: while true do begin P(SFull); P(S); {прочитать сообщение} V(SFree); V(S); {обработать сообщение} end;

Семафор SFree отслеживает количество свободных элементов в очереди, а семафор SFull – количество занятых. Тем самым предотвращаются попытки прочитать из пустой очереди или записать новое сообщение в полностью заполненную очередь. Назначение семафора S состоит в предотвращении одновременной работы с очередью поставщика и потребителя.

Ожидание. Рассмотрим другую задачу взаимодействия двух процессов: пусть процесс 1 («хозяин») должен запустить процесс 2 («служу»), после чего работа хозяина делится на две части: независимую от работы слуги и часть, которая может быть выполнена только после окончания работы подчиненного процесса. В этом случае работает следующий алгоритм:

Var S: Tsemaphore; Begin InitSemaphore(S, 0);	
ХОЗЯИН: begin {запустить процесс-служу} {независимая часть работы} P(S); {зависимая часть работы} end;	СЛУГА: begin {работа слуги} V(S); end;

«Читатели» – «писатели». Эта задача также является классической при рассмотрении взаимодействия сотрудничающих процессов. Суть ее заключается в следующем. Работающие с одним общим ресурсом процессы делятся на две группы. «Читатели» могут только читать информацию, хранящуюся в этом ресурсе, и, следовательно, допускается их параллельная работа. Процессы-«писатели» могут изменять содержимое этого ресурса, исключая как друг друга, так и процессы-читатели.

Примером этой задачи может служить система автоматизированной продажи билетов. Справочные мониторы только отображают текущую информацию о наличии свободных мест и по сути являются процессами-читателями. Процесс-писатель запускается с рабочего места кассира, когда он оформляет тот или иной билет. Одновременно может работать большое количество как читателей, так и писателей.

Различают две модификации представленной задачи, в зависимости от приоритета читателей либо писателей. Если приоритет в доступе к критическому ресурсу имеется у читателей, то если хотя бы один из читателей пользуется критическим ресурсом, то этот ресурс недоступен для «писателей». Во втором случае при появлении запроса от «писателя» необходимо закрыть доступ к критическому ресурсу всем «читателям», которые позже выдадут запрос на этот ресурс.

В зависимости от модификации, рассмотрим два различных алгоритма для этой задачи. Пусть вначале приоритет имеют читатели:

<pre> var Sread, SWrite: Tsemaphore; NR: integer; // число читателей в системе begin InitSemaphore(SRead, 1); InitSemaphore(SWrite, 1); NR := 0; </pre>	
<pre> ЧИТАТЕЛЬ: while true do begin P(Sread); NR := NR+1; if NR=1 then P(SWrite); V(Sread); {чтение данных} P(Sread); NR := NR-1; if NR=0 then V(SWrite); V(Sread); {некритическая секция} end; </pre>	<pre> ПИСАТЕЛЬ: while true do begin P(SWrite); {запись данных} V(SWrite); {некритическая секция} end; </pre>

В случае, когда приоритет имеют писатели, необходимо предотвратить доступ новых читателей до тех пор, пока появившийся в очереди к ресурсу писатель не пройдет свой критический интервал. Это достигается введением дополнительного семафора, как показано в следующем алгоритме:

<pre> var S, SRead, SWrite: TSemaphore; NR: integer; // число читателей в системе begin InitSemaphore(S, 1); InitSemaphore(SRead, 1); InitSemaphore(SWrite, 1); NR := 0; </pre>	
<pre> ЧИТАТЕЛЬ: while true do begin P(S); P(SRead); NR := NR+1; if NR=1 then P(SWrite); V(S); V(SRead); {чтение данных} </pre>	<pre> ПИСАТЕЛЬ: while true do begin P(S); P(SWrite); {запись данных} V(S); V(SWrite); {некритическая секция} end; </pre>

<pre> P(SRead); NR := NR-1; if NR=0 then V(SWrite); V(Sread); {некритическая секция} end; </pre>	
---	--

Наконец, рассмотрим случай, когда «писатель» не может ждать освобождения ресурса «читателями», занявшими его. В этом случае можно объявить ресурс некритическим. Поскольку «писатель» только один, то описанной в предыдущем примере неверной модификации данных не произойдет. С другой стороны, есть опасность того, что «читатель» получит неправильную информацию. Для устранения этой проблемы предлагается использовать аппарат версий. «Писатель» при каждом изменении данных увеличивает номер версии на единицу, а «читатели» проверяют номер версии в начале и конце чтения:

<pre> var NewVersion, OldVersion: Integer; begin NewVersion:= 0; OldVersion:= 0; </pre>	
<pre> ЧИТАТЕЛЬ: var Version: integer; while true do begin repeat Version := OldVersion; {чтение данных} until Version = NewVersion; {некритическая секция} end; </pre>	<pre> ПИСАТЕЛЬ: while true do begin inc(NewVersion); {запись данных} OldVersion := NewVersion; {некритическая секция} end; </pre>

3.4.2. Организация обмена данными

Взаимодействие между процессами предполагает не только их синхронизацию, но и обмен данными. Ранее уже упоминался механизм обмена сообщениями; сейчас рассмотрим его более подробно.

Конвейеры (pipe, каналы, транспортеры) представляют собой средство для обмена данными в режиме «поставщик» - «потребитель». принцип работы конвейера основан на следующем механизме: поставщик при передаче информации выполняет те же действия, что и при обычном выводе в поток; потребитель читает данные из потока. Таким образом, обоим процессам совершенно необязательно даже знать о существовании конвейера. Это упрощает

программирование и избавляет программистов от использования каких-то новых механизмов. Кроме того, конвейеры требуют согласования форматов передаваемых данных только между поставщиком и потребителем. Операционная система не вмешивается в тонкости передачи данных, она лишь создает конвейер и предоставляет процессам мониторы для работы с ним.

На физическом уровне каждому конвейеру соответствует буфер в оперативной памяти, организованный по принципу циклической очереди. Поставщик при выполнении операции записи заносит в буфер очередную порцию данных, потребитель извлекает информацию из буфера при чтении. Ситуация «очередь пуста» влечет блокирование потребителя, переполнение очереди приводит к блокированию поставщика.

Реализация конвейера должна быть хорошо знакома вам по ее использованию в bat-файлах: оператор «| » приводит к соединению в конвейер файла `stdout` первой команды и файла `stdin` – второй:

```
if exist a:\*.* echo y | del a:\*.*
```

Для задания конвейера могут использоваться следующие функции Windows API:

- `CreatePipe` – создать анонимный конвейер. При этом указывается размер буфера, в который помещаются и из которого читаются данные. При успешном завершении функция возвращает дескрипторы для чтения и записи данных;
- `ReadFile` – читать из конвейера как из файла (в качестве дескриптора файла указывается дескриптор, возвращенный функцией `CreatePipe`);
- `WriteFile` – писать в конвейер.

Сообщения представляют собой четкую структуру, которая в общем случае включает следующие поля:

- идентификатор отправителя;
- идентификатор получателя;
- текст сообщения (или адрес, по которому находится этот текст);
- приоритет сообщения;
- дополнительные параметры (например, «ждать ответа»)

По сравнению с контейнерами сообщения представляют собой более сложный механизм передачи данных. Во-первых, при приеме-передаче сообщений не происходит задержка процесса, которая возникает при попытке записать в полный конвейер или прочитать из пустого. Во-вторых, если при чтении данных из конвейера они удаляются, то сообщения можно прочитать несколько раз. В третьих, порядок чтения данных из очереди сообщений может не совпадать с порядком их помещения.

3.5. Тупики

При параллельном программировании могут возникнуть ситуации, когда два или более процессов все время находятся в заблокированном состоянии. Самым простым является случай, когда один из процессов ожидает ресурс, занятый другим процессом. Из-за такого ожидания ни один из процессов не может продолжить работу и освободить в конечном итоге ресурс, необходимый другому процессу. Такая ситуация называется *тупиком* (deadlock, clinch). Говорят, что процесс находится в состоянии тупика, если он ожидает события, которое никогда не произойдет. Причиной возникновения тупиков могут быть как ошибки программирования, так и конкуренция несвязанных процессов.

Проблема тупиков включает в себя следующие задачи:

- предотвращение тупиков,
- распознавание тупиков,
- восстановление системы после тупиков.

Следует отметить, что тупики могут быть предотвращены на стадии написания программ. Программы должны быть написаны таким образом, чтобы тупик не мог возникнуть ни при каком соотношении взаимных скоростей процессов. Второй подход к предотвращению тупиков называется динамическим и заключается в использовании определенных правил при назначении ресурсов процессам, например, ресурсы могут выделяться в определенной последовательности, общей для всех процессов.

В некоторых случаях, когда тупиковая ситуация образована многими процессами, использующими много ресурсов, распознавание тупика является нетривиальной задачей. Существуют формальные, программно реализованные методы распознавания тупиков, основанные на ведении таблиц распределения ресурсов и таблиц запросов к занятым ресурсам. Анализ этих таблиц позволяет обнаружить взаимные блокировки.

Если же тупиковая ситуация возникла, то не обязательно снимать с выполнения все заблокированные процессы. Можно снять только часть из них, при этом освобождаются ресурсы, ожидаемые остальными процессами, можно вернуть некоторые процессы в область свопинга, можно совершить откат некоторых процессов до так называемой контрольной точки, в которой запоминается вся информация, необходимая для восстановления выполнения программы с данного места. Контрольные точки расставляются в программе в местах, после которых возможно возникновение тупика.

3.5.1. Примеры тупиковых ситуаций

Кольцевая передача сообщений. Пусть N процессов время от времени обмениваются сообщениями следующим образом: для выполнения некоего уча-

стка каждый процесс должен послать сообщение последующему и дожидаться сообщения от предыдущего. Иными словами, работа процесса с номером i ($1 \leq i \leq N$) выглядит следующим образом:

<i>Неправильно</i> <pre> while true do begin {HKY} if i=1 then WaitMessage(N) Else WaitMessage(i-1); if i=N then SendMessage(1) Else SendMessage(i+1); {KY} end;</pre>	<i>Правильно</i> <pre> while true do begin {HKY} if i=N then SendMessage(1) else SendMessage(i+1); if i=1 then WaitMessage(N) else WaitMessage(i-1); {KY} end;</pre>
---	---

Заметим, что тупик, который может возникнуть в этом случае, является следствием ошибки в программировании, в отличие от следующего примера.

Взаимный захват ресурсов. Пусть работа двух процессов, работающих с семафорами S1 и S2, описывается следующим образом:

{Участок 1}	{Участок 1}
P(S1);	P(S2);
{Участок 2}	{Участок 2}
P(S2);	P(S1);
{Участок 3}	{Участок 3}
V(S2);	V(S1);
{Участок 4}	{Участок 4}
V(S1);	V(S2);
{Участок 5}	{Участок 5}

В этом случае процессы могут пропасть в тупик, а могут и не попасть: все зависит от продолжительности их участков. Опишем схему работы диаграммой состояний, показанное на рисунке 3.5. Вертикали таблицы соответствуют участкам первого процесса, горизонтали – участкам второго. Клетка (i, j) таблицы соответствует ситуации, когда первый процесс проходит участок i, а второй – участок j.

Итак, для того, чтобы возник тупик, требуется выполнение следующих четырех условий:

- 1) взаимного исключения, при котором процессы осуществляют монопольный доступ к ресурсам;

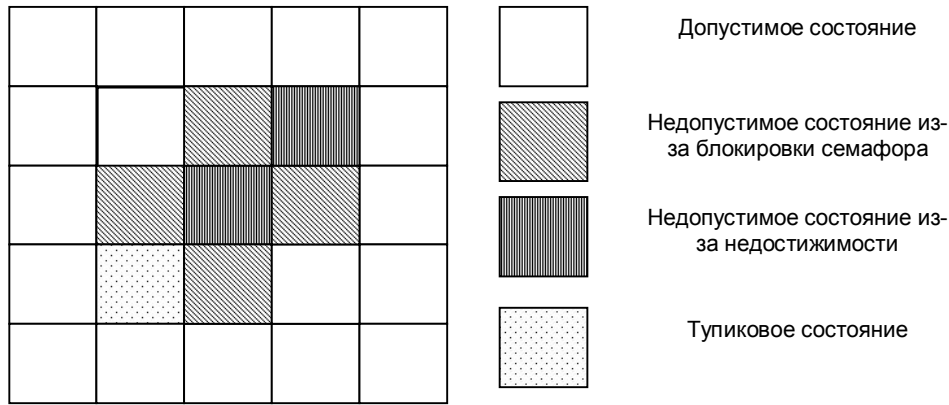


Рис. 3.5. Пространство состояний системы из двух параллельных конкурирующих процессов

2) ожидания, при котором процесс, запросивший ресурс, ждет удовлетворения своего запроса, удерживая при этом все выделенные ему ранее ресурсы;

3) отсутствия перераспределения: если ресурсы выделены процессу, то отобрать их у него уже нельзя;

4) кругового ожидания, т.е. наличия замкнутой цепи процессов, каждый из которых ждет ресурс, удерживаемый его предшественником в этой цепочке.

3.5.2. Формальные модели изучения тупиковых ситуаций

К настоящему времени разработано несколько десятков моделей взаимодействия параллельных вычислительных процессов для анализа тупиковых ситуаций. Сейчас рассмотрим одну из них (модель пространства состояний системы), а позже – другую (модель Холта).

Пусть состояние системы будет сводиться к состоянию имеющихся в ней ресурсов (свободны они или выделены каким-то процессам). Если процесс не блокирован в каком-то состоянии, то он может изменить его на новое состояние. Однако поскольку мы не знаем заранее, какой путь изберет произвольный процесс в своей программе, то новое состояние может быть любым из некоторого конечного множества возможных. Дадим формальные определения:

Пусть задано конечное множество состояний системы $\{S_1, \dots, S_n\}$. Процесс P_j есть частичная функция, отображающая состояние в подмножество ее же состояний. Если $S_k \in P_j(S_i)$, будем говорить, что P_j может изменить состояние S_i на S_k . Обозначим через $D(S_i)$ множество состояний, в которые можно перейти из состояния S_i .

Процесс P_j заблокирован в состоянии S_i , если $P_j(S_i) = \emptyset$. Процесс P_j находится в тупике в состоянии S_i , если $P_j(S_k) = \emptyset$ для всех $S_k \in D(S_i)$.

Разобьем теперь множество состояний на группы:

- состояние S_i называется *тупиковым*, если существует процесс, находящийся в тупике в этом состоянии;
- состояние S_i называется *опасным*, если $D(S_i)$ содержит только тупиковые состояния;
- состояние S_i называется *безопасным*, если $D(S_i)$ не содержит тупиковых состояний;
- состояние S_i называется *обычным*, если оно не принадлежит к одной из описанных ранее групп.

Рассмотрим пример. Пусть система состоит из 4 состояний и 2 процессов. Известно, что

$P_1(S_1) = \{S_1, S_2\};$	$P_2(S_1) = \{S_3\};$
$P_1(S_2) = \emptyset;$	$P_2(S_2) = \{S_1, S_4\};$
$P_1(S_3) = \{S_4\};$	$P_2(S_3) = \emptyset;$
$P_1(S_4) = \{S_3\};$	$P_2(S_4) = \emptyset;$

В этом случае состояния S_3 и S_4 являются тупиковыми для процесса P_2 .

3.5.3. Методы борьбы с тупиками

Борьба с тупиковыми ситуациями основывается на одной из следующих трех стратегий:

- предотвращения тупика;
- обход тупика;
- распознавание тупика с последующим восстановлением.

Предотвращение тупиков основывается на предположении, что его стоимость чрезвычайно высока. Поэтому лучше потратить дополнительные ресурсы системы, чтобы исключить вероятность возникновения тупика при любых обстоятельствах.

Как уже было сказано ранее, дисциплина, предотвращающая тупик, должна препятствовать наступлению хотя бы одного из перечисленных в п. 3.5.1 условий.

Например, условие ожидания можно подавить, предварительно выделяя ресурсы. При этом процесс может начать работу, только получив все необходимые ему ресурсы заранее.

Условие отсутствия перераспределения можно исключить, если дать возможность системе отнимать у процесса ресурсы.

Условие кругового ожидания исключается за счет принципа иерархического выделения ресурсов: все ресурсы образуют некую иерархию. Процесс, затребовавший некоторый ресурс, в дальнейшем может требовать только ресурсы более высокого уровня; он может также освободить конкретный ресурс

только после освобождения всех ресурсов на всех более высоких уровнях. В приведенном на рисунке 3.5 примере работа процесса 2 должна выглядеть так (если семафор S2 защищает ресурс более высокого уровня):

```
{Участок 1}  
P(S1);  
P(S2);  
{Участок 2}  
{Участок 3}  
{Участок 4}  
V(S2);  
V(S1);  
{Участок 5}
```

В целом стратегия предотвращения тупиков – это очень дорогое решение проблемы, и оно используется нечасто.

Обход тупиков можно интерпретировать как запрет входа в опасное состояние. Классическое решение этой задачи известно как *алгоритм банкира Дейкстры*. Рассмотрим его.

Пусть в системе существует R единиц ресурса и N процессов, для каждого из которых известно максимальное количество потребностей в ресурсе M_i . Пусть также каждому процессу выделено K_i единиц ресурса, а процесс j требует дополнительного выделения ему T единиц ресурса. Алгоритм может быть описан следующей процедурой:

```
function CanSatisfy: Boolean;  
var  
  Free: integer;  
  Rest: array [1..N] of integer;  
  Finished: array [1..N] of Boolean;  
  i: integer;  
  flag: Boolean;  
begin  
  Free := R-T;  
  for i := 1 to N do begin  
    Free := Free - K[i];  
    Rest[i] := M[i] - K[i];  
    if i=j then  
      Rest[i] := Rest[i]-T;  
      Finished[i] := false;  
  end;  
  flag := true;  
  while flag do begin  
    flag := false;  
    for i := 1 to N do begin
```

```

if not Finished[i] and (Rest[i] <= Free) then begin
    Finished[i] := true;
    Free := Free + K[i];
    if i=j then
        Free := Free+T;
        flag := true;
    end;
end;
end;
CanSatisfy := (Free=R);
end;

```

Рассмотрим пример работы этого алгоритма. Пусть $R=10$, $N=3$, остальные данные сведены в таблицу:

Процесс	M_i	K_i	T
1	5	2	2
2	5	3	
3	5	2	

В этом случае после выделения требуемого числа ресурсов процессы могут завершиться в последовательности 1 – 2 – 3, и ресурс может быть выделен. Однако, если $M_1 = 6$, то запрос не может быть удовлетворен.

3.5.4. Распознавание тупиков

Этот подход основывается на предположении, противоположном тому, которое высказывалось ранее: хоть возникновение тупика влечет за собой снятие и перезапуск по меньшей мере одного процесса, однако вероятность возникновения тупикового состояния невелика, а предотвращение тупика влечет резкое снижение производительности вычислительной системы, и потери от этого сравнимы с потерями от возникновения тупиков. Поэтому нам надо лишь распознавать возникшие тупиковые ситуации, но не принимать никаких мер к их предотвращению. В частности, мы можем применить следующую дисциплину выделения ресурсов: если запрашиваемые каким-то процессом ресурсы свободны, они немедленно распределяются этому процессу (дисциплину *немедленного выделения ресурсов*).

Для анализа тупиковых ситуаций в случае повторно используемых ресурсов удобно использовать т.н. *модель Холта*, которая представляет собой двудольный граф¹ с вершинами- процессами (которые обозначаются квадратами) и вершинами-ресурсами (обозначающимися кругами). Вершина-ресурс может содержать вес – количество единиц имеющегося в системе ресурса. Дуга от

¹ Двудольный граф – граф с вершинами двух классов, причем дуги могут идти только из вершин одного класса в вершины другого класса.

процесса к ресурсу означает, что соответствующий процесс запросил этот ресурс, дуга от ресурса к процессу – что ресурс выделен процессу. Веса дуг означают соответственно количество запрошенного и выделенного ресурса. На рисунке 3.6 приведен пример диаграммы Холта в предположении, что немедленное выделение ресурсов отключено. В данном случае процессы P_1 и P_3 требуют выделения одного и того же ресурса, причем имеющихся 5 единиц явно недостаточно для удовлетворения этих запросов.

Чтобы распознать тупиковое состояние, необходимо для каждого присутствующего в системе процесса определить, сможет ли он когда-нибудь снова

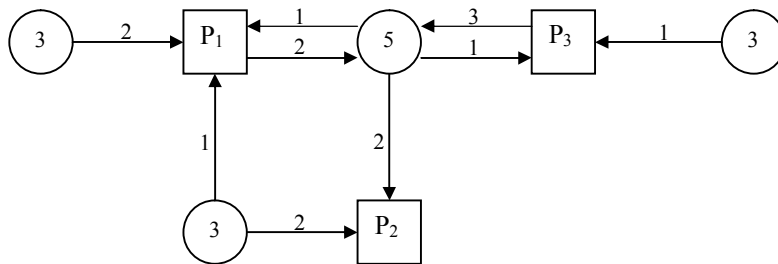


Рис. 3.6. Пример диаграммы Холта

развиваться, т.е. изменять свое состояние. При этом, в отличие от алгоритма банка, мы будем рассматривать не наихудший, а наилучший варианты.

Очевидно, что если процесс не является заблокированным, т.е. все затребованные ресурсы ему выделены, то он может завершиться, и при этом освободить занятые ресурсы. После этого смогут «проснуться» другие, ранее заблокированные процессы. Продолжая по цепочке этот процесс, мы можем либо разблокировать все процессы (и в этом случае тупика не будет), либо убедиться, что несколько процессов (не менее двух, ибо самоблокировка является ошибкой программирования) остаются заблокированными. Тогда исходное состояние будем считать тупиковым.

Рассмотрим формальные определения и обозначения.

Обозначим через $W(R_j)$, $W(P_i, R_j)$, $W(R_j, P_i)$ соответственно веса вершин-ресурсов и дуг разного типа.

Вершина-процесс P_i считается *заблокированной* ресурсом R_j , если

$$W(P_i, R_j) \geq W(R_j) - \sum_p W(R_j, P)$$

Граф повторно используемых ресурсов *редуцируется* (сокращается) процессом P_0 , который не является ни заблокированной, ни изолированной вершиной, с помощью удаления всех входящих и исходящих ребер. Эта процедура считается эквивалентной приобретению процессом P_0 всех нужных ему

ресурсов, а затем освобождения всех занятых ресурсов. После этого P_0 становится изолированной вершиной.

В примере, показанном на рисунке 3.6, может быть выполнена редукция процессов в следующей последовательности: P_2, P_1, P_3 .

Граф повторно используемых ресурсов называется *несокращаемым*, если он не может быть редуцирован ни одним процессом.

Граф повторно используемых ресурсов называется *полностью сокращаемым*, если существует последовательность сокращений, которая удаляет все дуги графа.

Лемма. Порядок сокращений в графе несущественен; все сокращения ведут к одному и тому же несокращаемому графу (*без доказательства*)

Теорема 1. Состояние S является тупиковым тогда и только тогда, когда соответствующая ему диаграмма Холта не является полностью сокращаемой.

Доказательство. Предположим, что P_0 находится в состоянии тупика в состоянии S . Тогда для всех $S_k \in D(S)$ P_0 будет заблокирован в этих состояниях, в том числе и в том, к которому мы придем серией последовательных сокращений. Таким образом, после всех возможных сокращений вершина P_0 останется заблокированной, т.е. граф сократится не полностью.

Предположим теперь, что граф состояния S не является полностью сокращаемым. Тогда существует хотя бы один процесс P_0 , который останется заблокированным после всех операций сокращения. Но поскольку последовательные сокращения гарантируют, что все повторно используемые ресурсы, которые когда-либо могли стать доступными, освобождены, то P_0 заблокирован навсегда, и, следовательно, находится в тупике. ■

Из приведенной теоремы непосредственно следует алгоритм распознавания тупиковых ситуаций: нужно просто попытаться сократить граф по возможности наиболее эффективным способом. Один из таких способов основан на следующих утверждениях:

Теорема 2. Наличие цикла в графе повторно используемых ресурсов является необходимым условием тупика.

Доказательство. Предположим, что граф, соответствующий тупиковой ситуации, не содержит циклов. Тогда его вершины можно перенумеровать таким образом, что если существует путь из вершины i в вершину j , то $i < j$. Теперь, проводя редукцию начиная с вершин с меньшими номерами, мы в конце концов уберем все дуги, что противоречит нашему предположению. ■

Теорема 3. Если S не является тупиковым состоянием, и существует тупиковое состояние $S_t \in P_j(S_i)$, то соответствующая операция процесса P_j есть запрос на ресурс, и процесс P_j оказывается в тупике в состоянии S_t (*без доказательства*)

Исходя из последней теоремы, проверку на тупик стоит осуществлять только при наличии неудовлетворенного запроса на ресурсы. При этом сокращение графа надо доводить только до того процесса, который становится заблокированным.

Теорема 4. Если все вершины-ресурсы диаграммы Холта имеют вес 1, то наличие цикла является необходимым и достаточным условием тупиковой ситуации.

Доказательство. Нам достаточно показать только достаточность наличия цикла (необходимость показана в теореме 2). Будем рассматривать только вершины диаграммы Холта, входящие в цикл. Из того, что вес каждой вершины ресурса равен 1, следует, что она имеет только одну выходящую дугу, и соответствующий ресурс принадлежит процессу, принадлежащему этому же циклу. С другой стороны, этот ресурс затребован другим процессом, входящим в рассматриваемый цикл. Таким образом, мы имеем цепочку процессов, каждый из которых блокируется другим процессом из этой цепочки, что и доказывает тупик. ■

На рисунке 3.7 показана диаграмма Холта для рассмотренной ранее (см. рисунок 3.5) тупиковой ситуации, в которую могут попасть два процесса, конкурирующие за два ресурса.

Рассмотрим теперь один из алгоритмов определения тупиковой ситуации,

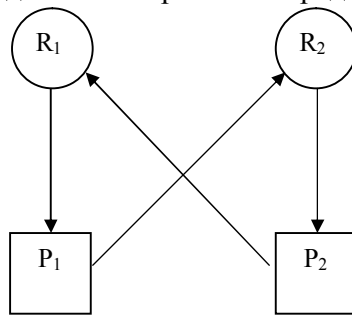


Рис. 3.7. Диаграмма Холта, иллюстрирующая тупиковую ситуацию

использовавшийся в одной из ОС компании IBM. Пусть в системе имеется N процессов и M ресурсов; и те, и другие пронумерованы, начиная с единицы. Алгоритм основан на ведении двух массивов: $P[N]$ и $R[M]$. $P[i] = 0$, если процесс не заблокирован, и $P[i] = j$, если он заблокирован ресурсом j . Аналогично, $R[j] = 0$, если ресурс j свободен, и $R[j] = i$, если он занят процессом i . При обслуживании запроса на выделение процессу p_0 ресурса r_0 работает следующий алгоритм:

```

var
  I, J, K: integer;
begin
  I := p0;
  J := r0;
  if R[J] = 0 then begin
    R[J] := i;
    exit; // процессу выделяется свободный ресурс
  end;
  P[I] := J;
  while true do begin
    I := R[J];
    if I = p0 then begin
      {обработать тупиковую ситуацию}
      exit;
    end;
    if P[I]=0 then // цепочка процессов заканчивается
      exit; // незаблокированным - тупика нет
    J := P[I];
  end;
end;

```

Рассмотрим пример работы этого алгоритма при N=3 и M=5. Пусть вначале все ресурсы свободны, и запросы на их занятие идут в следующем порядке:

Процесс	1	2	3	2	1	1	2	3
Ресурс	1	2	3	4	5	3	3	5

Первые пять запросов удовлетворяются без труда, следующие два – приводят к блокировке вызвавших их процессов. Состояние массивов P и R приведено в следующей таблице

P	3	3	0		
R	1	2	3	2	1

И наконец, последний запрос приведет к тупику.

Глава 4. Управление центральным процессором и памятью

4.1. Общие положения

Поскольку время центрального процессора и оперативная память должны выделяться каждому работающему в вычислительной системе процессу, имеет смысл выделить рассмотрение особенностей работы с этими ресурсами в рамках отдельной главы.

В рамках данной темы мы будем вместо понятий «процесс» и «тред» пользоваться термином «задача». При рассмотрении механизмов управления центральным процессором следует учитывать, что все треды конкурируют между собой за этот ресурс, в то время как оперативная память, как правило, выделяется процессу при его создании.

Как известно, операционная система выполняет следующие основные функции, связанные с управлением задачами:

- создание и удаление задач;
- планирование процессов и диспетчеризация задач;
- синхронизация задач, обеспечение их средствами коммуникации (эта функция не будет рассматриваться в рамках этой главы).

Создание и удаление задач осуществляется по соответствующим запросам от пользователей или самих задач. При этом пользователь, как правило, может отправить запрос на создание нового процесса (запустить новое приложение). Задачи же могут создавать как новые треды в рамках собственного процесса, так и инициировать работу новых процессов. При удалении задач также следует разделять запросы пользователей на удаление любого процесса и запросы от задач (только на удаление подчиненных тредов).

Планирование процессов можно подразделить на долгосрочное и краткосрочное. Долгосрочное планирование выполняется перед запуском процессов. Очевидно, что на распределение ресурсов влияют конкретные потребности тех задач, которые должны выполняться параллельно. Так, может сложиться ситуация, когда за одно устройство с последовательным доступом, которое по определению не может быть разделено между несколькими задачами, начинается конкуренция, которая приводит к тому, что ожидание других процессов (в свою очередь, владеющих другими ресурсами) может продлиться недопустимо долго.

Если до запуска процесса известно, какие ресурсы он затребует, то можно спланировать последовательность запуска процессов так, чтобы конфликты

между процессами возникали как можно реже. при этом из-за отсутствия дополнительных ожиданий процессы смогут выполняться быстрее, да и имеющиеся в системе ресурсы будут использоваться более эффективно.

В качестве примера можно назвать следующие стратегии долгосрочного планирования процессов:

- по возможности заканчивать вычисления в том же порядке, в котором они были начаты;
- отдавать предпочтение более коротким процессам;
- предоставлять всем пользователям (т.е. их процессам) одинаковое время ожидания;
- формировать т.н. *рабочую смесь* из разнородных процессов или из процессов, требующих различные ресурсы.

Задача планирования процессов возникла достаточно давно, во времена первых пакетных ОС, когда пользователь был физически отделен от процесса вычислений. В настоящее время, в связи с практически повсеместным использованием интерактивного режима работы пользователей с вычислительной системой, актуальность долгосрочного планирования снизилась. Действительно, порядок запуска процессов определяется пользователем, и именно он несет ответственность за планирование. Конечно же, какие-то элементы планирования присутствуют на интуитивном уровне: например, при подготовке текста этой книги запущена программа Light Alloy, так как ресурсов системы при работе с текстовым процессором и проигрывателем MP3-файлов должно хватить. В большинстве современных операционных систем долгосрочный планировщик отсутствует.

С другой стороны, задачи краткосрочного планирования, или *диспетчеризации задач* вышли на первый план. Краткосрочный планировщик решает, какая из задач, находящихся в состоянии готовности к выполнению, должна быть передана на исполнение.

Память является важнейшим ресурсом, требующим тщательного управления со стороны мультипрограммной операционной системы. Распределению подлежит вся оперативная память, не занятая операционной системой. Обычно ОС располагается в самых младших адресах, однако может занимать и самые старшие адреса. Функциями ОС по управлению памятью являются:

- отслеживание свободной и занятой памяти;
- выделение памяти процессам и освобождение памяти при завершении процессов;
- вытеснение процессов из оперативной памяти на диск, когда размеры основной памяти не достаточны для размещения в ней всех процессов, и возвращение их в оперативную память, когда в ней освобождается место;
- настройка адресов программы на конкретную область физической памяти.

4.2. Управление временем центрального процессора

4.2.1. Дисциплины диспетчеризации

Имеется большое количество различных правил (дисциплин диспетчеризации), в соответствии с которыми формируется очередь готовых к исполнению задач. В первую очередь следует разбить их на *приоритетные* и *бесприоритетные*. Беспriorитетные дисциплины производят выбор задачи из списка готовых к выполнению без учета внешних факторов и времени их обслуживания. В случае приоритетных схем каждой задаче присваивается статический или динамически изменяющийся приоритет, и выбор задачи для исполнения производится исходя из ее приоритета.

Кроме того, на выбор дисциплины планирования оказывает влияние и то, какой механизм реализации мультизадачности (вытесняющая или нет) реализуется при разработке операционной системы. (Понятия вытесняющей и невытесняющей многозадачности рассмотрены ранее в п. 2.4.) Вытесняющие и не вытесняющие механизмы имеют свои преимущества и недостатки. Не вытесняющие механизмы более просты и требуют меньше времени для своего исполнения, но при этом предполагается, что задачи должны учитывать возможности работы в мультипрограммном режиме (например, время от времени посылать супервизору специальный запрос типа «переведи меня в очередь готовых задач»).

Рассмотрим несколько наиболее часто используемых дисциплин диспетчеризации, прежде всего – беспriorитетных.

Дисциплина FCFS (first come – first served) предполагает, что задачи обслуживаются в порядке их появления. Диспетчер задач ведет две очереди. В первую попадают задачи, которые уже получали время центрального процессора, но были заблокированы. Вторая очередь состоит из вновь запускаемых задач. Выбор задач для выполнения происходит вначале из первой очереди, и лишь при ее пустоте – из второй. Такой подход позволяет реализовать стратегию «по возможности завершать задачи в порядке их появления».

К достоинствам этой дисциплины относится простота реализации и малые затраты ресурсов при формировании очереди задач. Однако при увеличении загрузки оборудования растет среднее время обслуживания, в том числе и для коротких задач.

Дисциплина SJN (shortest job next) требует, чтобы для каждой задачи были заранее известны потребности в машинном времени. При реализации этой дисциплины ведется только одна очередь готовых к выполнению задач, и включение в эту очередь происходит в соответствии с этой оценкой. Такая дисциплина позволяет снизить среднее время ожидания обслуживания. К недостаткам этой схемы следует отнести то, что оценку времени выполнения не

всегда можно выполнить, и всегда есть риск злоупотреблений со стороны пользователей (занижение оценки). Кроме того, длительные задания, близкие к завершению, должны ожидать своей очереди достаточно долго.

Дисциплина SRT (shortest remaining time) является модификацией предыдущей дисциплины и позволяет устранить последний из упомянутых выше недостатков.

Все три рассмотренные дисциплины являются не вытесняющими и предназначены прежде всего для работы в пакетном режиме. Для интерактивной системы желательно прежде всего обеспечить приемлемое время реакции системы и принцип равенства в обслуживании (для многотерминальных задач). Если же система является однопользовательской, но многозадачной, то в этом случае стоит разделить задачи на активные и фоновые. Однако мы можем потребовать, чтобы фоновые процессы получали необходимую им долю процессорного времени.

Для решения этих проблем используется дисциплина RR (round robin) – циклического обслуживания. Она относится к вытесняющим дисциплинам и предполагает квантование времени. После окончания кванта времени задача снимается с процессора и ставится в очередь задач, готовых к выполнению. Величина кванта выбирается как компромисс между временем реакции системы и накладными расходами на смену контекста задач. Эта величина может быть постоянной и переменной. Так, например, вновь стартующим задачам стоит дать большие кванты, поскольку им требуется дополнительное время для начала работы.

Дисциплина RR может быть как беспriorитетной, так и учитывать приоритеты задач, что достигается, как правило, за счет ведения нескольких очередей.

4.2.2. Использование динамических приоритетов

Рассмотрим механизмы установления динамических приоритетов для задач в различных операционных системах.

Windows NT поддерживает 32 уровня приоритетов. При этом приоритеты с номерами 16..31 присваиваются *потокам реального времени* – т.е. таким потокам, которые требуют немедленного внимания системы (по терминологии Microsoft).

Большинство потоков в системе относятся к классу переменного приоритета, и им присваивается приоритет от 0 до 15. Если поток прерван по истечению кванта времени, то диспетчер задач понижает его приоритет на единицу. Таким образом, приоритет задачи, требующей много вычислений, постепенно понижается до некоей базовой величины. С другой стороны, диспетчер задач повышает приоритет задач после их выхода из состояния ожидания.

Все сказанное относится к базовому приоритету процесса; его треды могут иметь свои приоритеты, отличающиеся от базового на ± 2 .

ОС UNIX ведет динамические приоритеты по другому принципу. Для вычисления приоритета используются две величины: p_nice (относительный приоритет, назначаемый при старте) и p_cpu (текущий приоритет, который изменяется диспетчером задач). Реальный приоритет задачи определяется по формуле

$$p_real = a * p_nice - b * p_cpu$$

Значение p_cpu пересчитывается каждый тик таймера (увеличивается для работающих и уменьшается для готовых к выполнению процессов). Что касается заблокированных процессов, то им присваивается значение *приоритета сна* (весьма высокое), так что после выхода из режима ожидания текущий приоритет становится равным приоритету сна, и разблокированный процесс становится в начало очереди.

4.2.3. Критерии качества обслуживания

Приведем краткое описание критериев для сравнения различных алгоритмов диспетчеризации. Это:

- гарантия обслуживания задачи (процессы не должны быть «забыты» или по крайней мере должны быть завершены к указанному сроку);
- использование центрального процессора (процент загрузки процессора);
- пропускная способность, т.е. количество процессов, выполненных в единицу времени;
- время оборота (полное время от запуска до завершения работы процесса);
- время ожидания процесса, т.е. суммарное время нахождения в очереди готовых к выполнению процессов;
- время отклика, т.е. время от попадания процесса во входную очередь до момента первого обращения к терминалу.

4.3. Основные концепции управления памятью

При рассмотрении механизмов управления памятью необходимо рассмотреть две проблемы:

- распределения памяти между процессами;
- защиты памяти, т.е. предотвращения доступа одного процесса к памяти, занимаемой другим процессом (эта проблема актуальна только для мультипрограммных систем).

Предварительно, однако, следует изучить некоторые общие аспекты взаимодействия процессов и памяти.

4.3.1. Пространства адресов

При рассмотрении механизмов управления оперативной памятью следует прежде всего рассмотреть механизм пространства адресов и отображения различных пространств друг в друга. Это отображение представлено на рисунке 4.1.

Пространство имен представляет собой совокупность идентификаторов, которые использует программа для обращения к тем или иным областям памяти (например, имен переменных, точек входа для программных модулей и т.д.). Как правило, на пространстве имен отсутствует порядок, т.е. порядок задания имен не имеет ничего общего с реальным расположением соответствующих областей памяти. Исключением является описание массивов (элементы массива должны идти непосредственно друг за другом в порядке возрастания их индексов) и структур (поля структуры также должны располагаться в смежных областях памяти).

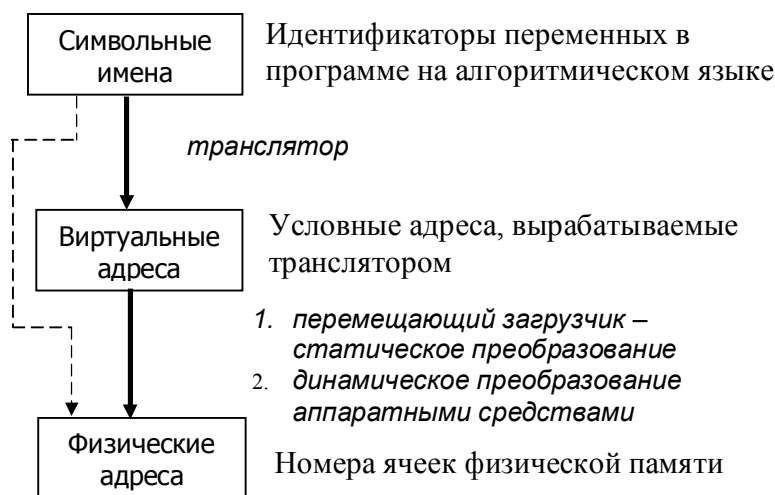


Рис. 4.1. Типы адресов

Логическое пространство адресов представляет собой множество числовых значений, которые могут быть использованы для доступа к памяти в машинных командах. Мощность этого множества зависит от архитектуры вычислительной системы. Так, процессор i8086 имеет 20-битную шину адреса, следовательно, логическое пространство для него содержит 2^{20} адресов; максимальный адрес, доступный для этого процессора (а также для реального режима работы старших моделей процессоров) – 1 Мбайт. Переход к 32-разрядной шине дает возможность оперировать с адресами до 4 Гбайт. Логический адрес часто представляется в формате «базовый адрес + смещение»,

где в качестве базового адреса может выступать, например, значение сегментного регистра.

Наконец, физическое пространство адресов ограничено объемом установленной в компьютере памяти.

Преобразование пространства имен в логическое пространство адресов выполняется на этапе подготовки исполняемого модуля и его загрузки в память перед выполнением. Рассмотрим этот этап подробнее.

При *компиляции* программы отдельным символическим именам ставятся в соответствие логические адреса. Эти адреса имеют вид смещения относительно некоего базового адреса. Значение базового адреса при этом остается неопределенным. Оставшиеся символические имена преобразуются в т.н. внешние ссылки, которые должны быть разрешены на последующих этапах обработки программы, в частности, на этапе *линкования (редактирования связей)*. Этот этап может либо быть интегрирован в единое целое с компиляцией, либо быть выделен в отдельный этап. На этапе линкования выполняется разрешение ссылок, оставшихся после компиляции, за счет внешних библиотек и пр. В исполняемом модуле неразрешенных внешних ссылок в идеале быть не должно, за исключением динамически подключаемых имен. Полученный исполняемый модуль является, как правило, перемещаемым, т.е. все логические адреса в нем представлены в виде смещений относительно одного или нескольких базовых адресов. Наконец на этапе *загрузки* программ для выполнения выполняется настройка базовых адресов. Полученные адреса, тем не менее, остаются логическими.

При преобразовании логических адресов в физические необходимо рассмотреть два случая:

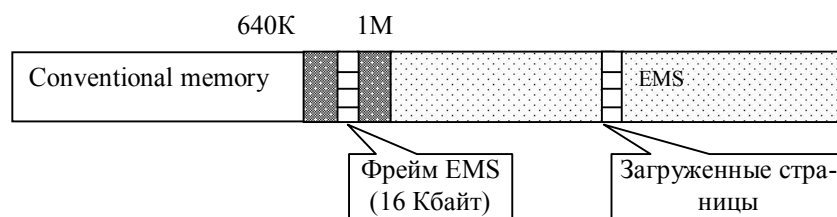


Рис. 4.2. Иллюстрация механизма EMS

1) Пространство физических адресов больше пространства логических. В этом случае для эффективного использования всей физической памяти программы должны использовать специальные механизмы расширения памяти. В качестве примера одного из таких механизмов рассмотрим реализацию EMS (Extended Memory System), характерную для поздних моделей MS-DOS. Суть ее состоит в следующем: память свыше 1 Мбайта, объявленная как EMS, разбивается на страницы размером в 4 Кбайта. С другой стороны, в области

старших адресов памяти (в границах адресов 0A0000h – 0FFFFh) выделяется фрейм размером в 16 Кбайт, в который перемещается информация из области EMS. Иллюстрация этого механизма приведена на рисунке 4.2.

2) Пространство логических адресов больше пространства физических. В этом случае возможности для преобразования оказываются гораздо более богатыми. Они основаны на концепции виртуальной памяти: на диске создается специальный файл подкачки (файл виртуальной памяти), разбитый на страницы. Подлежащая распределению оперативная память также разбивается на страницы того же размера. Логический адрес интерпретируется как «номер виртуальной страницы + смещение в странице». Некоторые из логических страниц загружаются в страницы оперативной памяти. Операционная система ведет специальную таблицу соответствия логических и физических страниц. При любом обращении к логическим адресам выполняется проверка того, находится ли эта страница в оперативной памяти. Если страница отсутствует, происходит ее загрузка (возможно, с вытеснением на диск одной из страниц, загруженных ранее). Механизм работы с виртуальной памятью проиллюстрирован на рисунке 4.3.

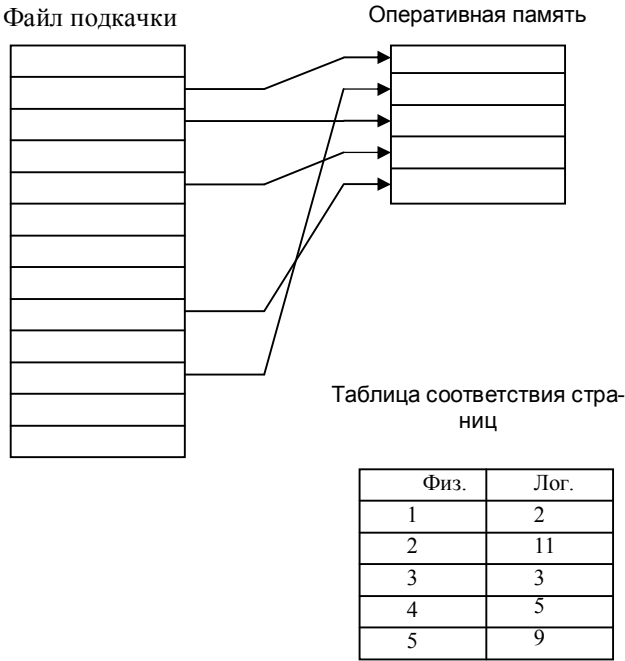


Рис. 4.3. Организация виртуальной памяти

Переход от виртуальных адресов к физическим может осуществляться двумя способами. В первом случае замену виртуальных адресов на физические делает специальная системная программа - перемещающий загрузчик.

Перемещающий загрузчик на основании имеющихся у него исходных данных о начальном адресе физической памяти, в которую предстоит загружать программу, и информации, предоставленной транслятором об адресно-зависимых константах программы, выполняет загрузку программы, совмещая ее с заменой виртуальных адресов физическими.

Второй способ заключается в том, что программа загружается в память в неизменном виде в виртуальных адресах, при этом операционная система фиксирует смещение действительного расположения программного кода относительно виртуального адресного пространства. Во время выполнения программы при каждом обращении к оперативной памяти выполняется преобразование виртуального адреса в физический. Второй способ является более гибким, он допускает перемещение программы во время ее выполнения, в то время как перемещающий загрузчик жестко привязывает программу к первоначально выделенному ей участку памяти. Вместе с тем использование перемещающего загрузчика уменьшает накладные расходы, так как преобразование каждого виртуального адреса происходит только один раз во время загрузки, а во втором случае - каждый раз при обращении по данному адресу.

В некоторых случаях (обычно в специализированных системах), когда заранее точно известно, в какой области оперативной памяти будет выполняться программа, транслятор выдает исполняемый код сразу в физических адресах.

4.3.2. Типы исполняемых модулей

Различают следующие три типа исполняемых модулей в соответствии с механизмом использования памяти:

- простые (линейные);
- оверлейные (с перекрытием);
- динамические.

В *простых* модулях каждая процедура получает свой блок логических адресов, не пересекающийся с блоками адресов других процедур. Из этого следует, что все процедуры занимают свои участки памяти. Такой механизм прост в использовании, поскольку после запуска процесса нет необходимости применять к нему какие-либо усилия со стороны загрузчика. Однако простая структура исполняемого модуля приводит к неоправданно большому объему выделяемой памяти в случае, если схема взаимодействия между процедурами имеет древовидную структуру. При этом может сложиться ситуация, когда пространства физических (или логических) адресов может не хватить для загрузки всей программы.

Альтернативой простой схеме загрузки модулей в память является *оверлейная* структура. При использовании этой концепции выделяются два или более модулей, которые не обращаются друг к другу. Для этих модулей указывается единая точка перекрытия (точка оверлея), и все перекрывающиеся

модули загружаются в память, начиная с этой точки. Это позволяет уменьшить суммарный объем загруженных модулей, но вынуждает тратить время на подгрузку перекрывающихся частей. Различия между простой и оверлейной структурой программы показаны на рисунке 4.4.

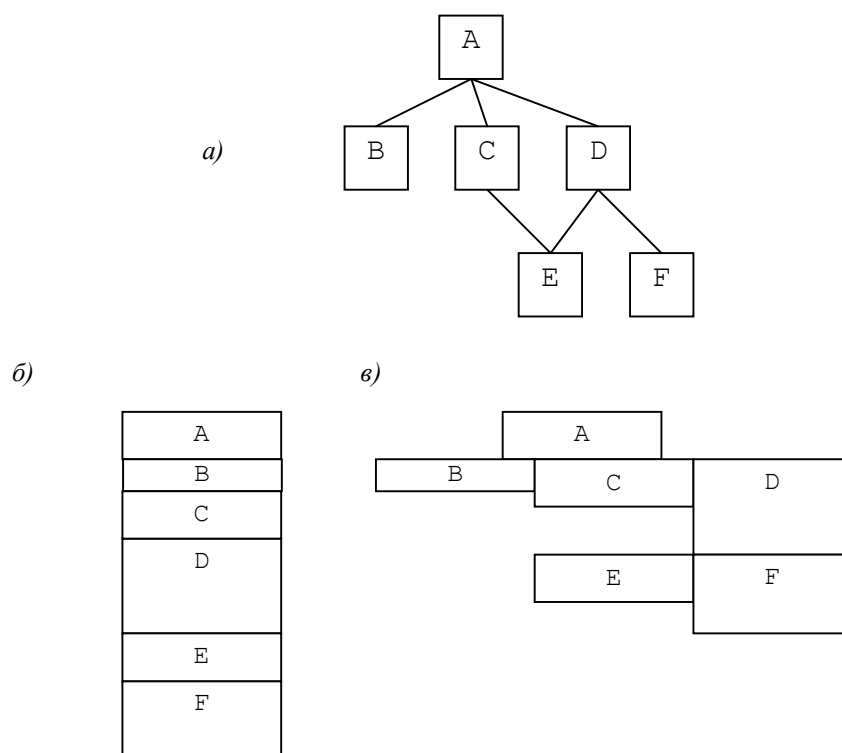


Рис. 4.4. Различия между простыми и оверлейными модулями: а) схема вызова подпрограмм; б) структура простого модуля; в) структура оверлейного модуля

Оверлейные структуры могут быть построены как вручную (т. е. программист сам указывает количество точек перекрытия и соответствующих им модулей), так и автоматически. Кроме того, оверлеи реализуются как средствами операционной системы, так и с помощью соответствующих систем программирования.

Как простые, так и оверлейные схемы предполагают, что информация о подпрограммах, которые могут быть затребованы, известна до начала работы приложения. В отличие от них, *динамическая* схема позволяет определить это во время работы процесса.

Рассмотрим механизм создания динамических модулей на примере динамической загрузки DLL (т.н. механизм explicit link).

Для динамического подключения к DLL необходимо:

- описать указатель на функцию, которую мы желаем вызвать;
- загрузить DLL в память с помощью функции API LoadLibrary;
- получить адрес требуемой функции с помощью функции API GetProcAddress;
- вызывать функцию, используя полученный указатель;
- по окончании работы с DLL выгрузить ее из памяти с помощью функции API FreeLibrary.

Рассмотрим пример программы, обращающейся к DLL по принципу explicit link. Пусть нам доступна DLL с именем dll0, содержащая функцию NumWords, считающую количество слов в ASCIIZ-строке. В качестве разделителя слов может быть использован любой символ, переданный в качестве параметра. Заголовок этой функции на языке C++ выглядит следующим образом:

```
unsigned __stdcall NumWords(char *S, char Divider=';');
```

Тогда обращение к этой функции может быть оформлено следующим образом:

```
#include <windows.h>
#include <iostream.h>
typedef unsigned (__stdcall* LPNUMWORDS) (char *,char=';');

int main(){
    HINSTANCE hDLL;           // Handle to DLL
    LPNUMWORDS lpNumWords;    // Function pointer
    hDLL = LoadLibrary("c:\\cpp\\dll0\\debug\\dll0");
    if (hDLL != 0){
        lpNumWords =
            (LPNUMWORDS)GetProcAddress(hDLL, "NumWords");
        if (lpNumWords == NULL) {           // handle the error
            FreeLibrary(hDLL);
            cout << "Error on GetProcAddress\n";
            return -1;
        }
        else { // call the function
            char S[100];
            cout << "Enter a string:";
            cin >> S;
            cout << "This string contains " <<
                lpNumWords(S)<< " word(s)" <<endl;
            return 0;
        }
    }
    else {
        cout << "Error on LoadLibrary\n";
    }
}
```

```

    return -2;
}
}

```

4.4. Механизмы распределения памяти в мультипрограммных системах

Все методы управления памятью могут быть разделены на два класса: методы, которые используют перемещение процессов между оперативной памятью и диском, и методы, которые не делают этого. Эта классификация проиллюстрирована рисунком 4.5.

Мы начнем рассматривать механизмы распределения памяти, показанные на рисунке 4.5, начиная с самых простых, несмотря на то, что к настоящему времени они представляют лишь чисто академический интерес.

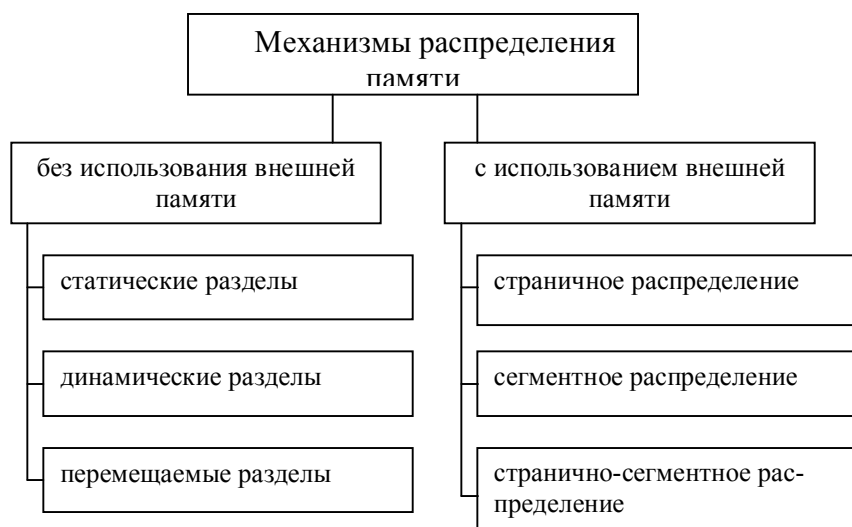


Рис. 4.5. Классификация методов распределения памяти

4.4.1. Распределение памяти разделами

Самая простая схема распределения памяти между несколькими задачами предполагает, что память, не занятая ядром ОС, разбивается на несколько непрерывных частей, называемых *разделами*. Каждый раздел характеризуется именем, типом, границами (в качестве которых чаще всего задается адрес начала раздела и его длина). При разбиении памяти на разделы мы предполагаем, что пространства логической и физической памяти совпадают. Разбиение памяти на разделы может быть как статическим, так и динамическим.

При статическом распределении памяти, т.е. при распределении в процессе генерации ОС или в момент ее запуска мы имеем т.н. *разделы с фиксированными границами*. Схема распределения памяти с фиксированными границами приведена на рисунке 4.6.

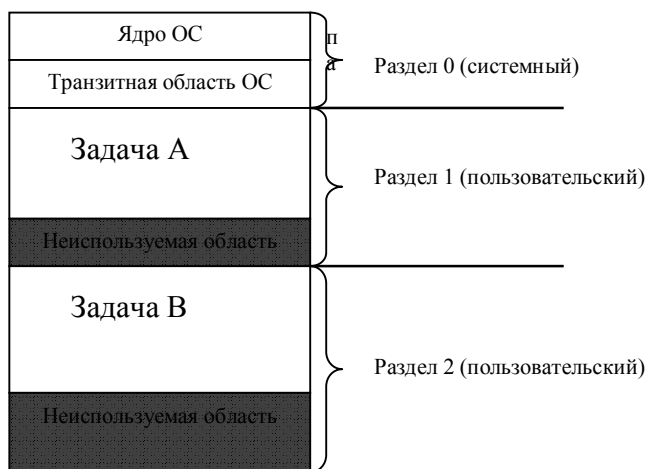


Рис. 4.6. Распределение памяти разделами с фиксированными границами

В каждом разделе в принципе может располагаться одна программа (задача). Однако при наличии нескольких интерактивных программ, которые практически постоянно ожидают окончания выполнения операций ввода-вывода, возможно размещение нескольких таких задач в одном разделе за счет *свопинга* (swapping). При свопинге прерванная задача выгружается из оперативной памяти на жесткий диск, а на ее место с диска загружается готовая к выполнению задача. Заметим, что при свопинге на диск и обратно перемещается вся программа, а не ее отдельная часть!

Основным недостатком такого способа распределения памяти является ее *фрагментация* - наличие нескольких участков неиспользуемой памяти в каждом из разделов. Такая фрагментация называется внутренней, поскольку она появляется внутри разделов, а сами разделы размещаются в памяти плотно.

Динамическое разбиение памяти на разделы (*разделы с подвижными границами*) предполагает, что разделы создаются в момент запуска задачи и уничтожаются после ее завершения, так что появляется возможность размещать задачи плотно, одну за другой. При использовании этой схемы в состав ОС входит специальный модуль – диспетчер памяти, который ведет список адресов свободной оперативной памяти. При появлении новой задачи диспетчер памяти просматривает этот список и выделяет для нее раздел, объем которого равен необходимому для работы задачи объему памяти. При завершении задачи и освобождении раздела диспетчер пытается слить его с одним из

свободных участков, если это возможно. Работа этого режима продемонстрирована на рисунке 4.7. Задачами операционной системы при реализации данного метода управления памятью является:

- ведение таблиц свободных и занятых областей, в которых указываются начальные адреса и размеры участков памяти,
- при поступлении новой задачи - анализ запроса, просмотр таблицы свободных областей и выбор раздела, размер которого достаточен для размещения поступившей задачи,
- загрузка задачи в выделенный ей раздел и корректировка таблиц свободных и занятых областей,
- корректировка таблиц свободных и занятых областей после завершения задачи.

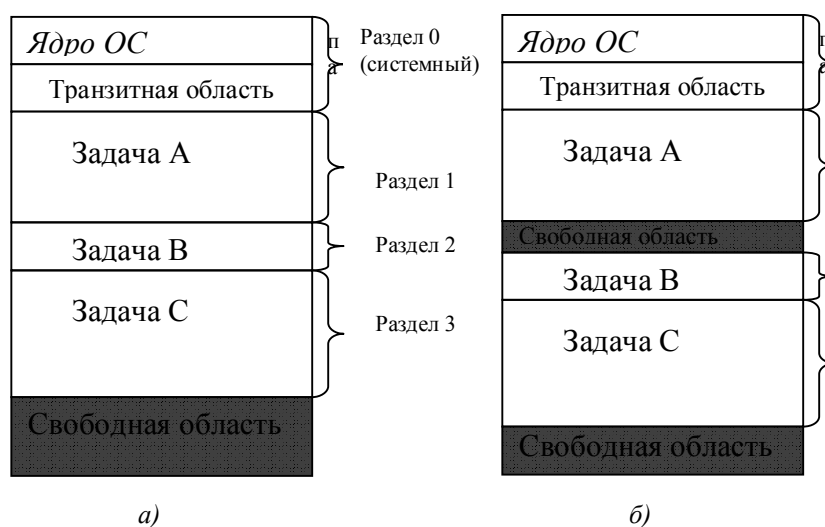


Рис. 4.7. Распределение памяти разделами с подвижными границами: а) идеальный случай; б) реальная ситуация

Выделение свободного участка для новой задачи может происходить по следующим алгоритмам:

- первый подходящий участок (список свободных участков отсортирован по возрастанию адресов);
- самый подходящий участок (список отсортирован по возрастанию длины участков);
- самый неподходящий участок (список отсортирован по убыванию длины участков).

Очевидно, что и эта схема приводит к фрагментации, поскольку задачи появляются и завершаются в произвольные моменты времени, да к тому же и имеют различный объем. В отличие от предыдущего механизма, эта фрагмен-

тация является внешней, т.к. операционная система знает о свободных участках и пытается выделить их запускаемым процессам - но они слишком малы. Заметим, что третий алгоритм выделения памяти является, как это ни странно, наиболее удачным!

4.4.2. Разрывное распределение памяти

Методы распределения памяти, при которых задаче может не предоставляться непрерывная область памяти, называют *разрывными (несмежными)*. Идея выделять программе память не сплошной единой областью, а фрагментами требует соответствующей аппаратной поддержки. Адрес, используемый в программном коде, должен состоять как минимум из двух частей: информацию о фрагменте программы (например, адрес его начала) и смещение относительно этого адреса. Таким образом, при использовании этого механизма виртуальный адрес не совпадает с реальным. Механизм преобразования виртуальных адресов в реальные для ускорения работы лучше реализовывать аппаратным путем.

Первым среди разрывных методов распределения памяти был сегментный. В соответствии с его концепцией программа разбивается на логические элементы – сегменты. В принципе любую совокупность программных модулей можно объявить как сегмент и загружать в память как самостоятельную единицу. Логически обращение к элементам программы будет представляться как указание имени сегмента и смещения относительно начала этого сегмента. Система программирования должна преобразовать имя сегмента в его порядковый номер.

При загрузке задачи в оперативную память ОС строит специальную структуру – *таблицу дескрипторов сегментов (ТДС)*. Каждый элемент этой таблицы содержит информацию о номере сегмента, физическом адресе его начала, длине, а также признак наличия в оперативной памяти (если реализован механизм свопинга), сведения о правах доступа и некоторую другую информацию. Адрес начала ТДС хранится в специальном регистре, а номер сегмента представляет собой смещение в этой таблице.

Механизм преобразования из логического адреса в физический показан на рисунке 4.8.

Следует сказать, что при таком механизме распределения памяти проблема фрагментации памяти решается лишь частично, поскольку размеры сегментов могут сильно различаться друг от друга и быть достаточно большими. Затруднена также проблема свопинга, поскольку размеры вытесняющего и вытесняемого сегмента различаются. Кроме того, мы имеем большие потери памяти и процессорного времени на размещение и обработку дескрипторных страниц, ведь на каждую задачу необходимо вести свою ТДС.

Другим способом разрывного распределения памяти является *страничная организация*. При этом способе работы с памятью все фрагменты задачи

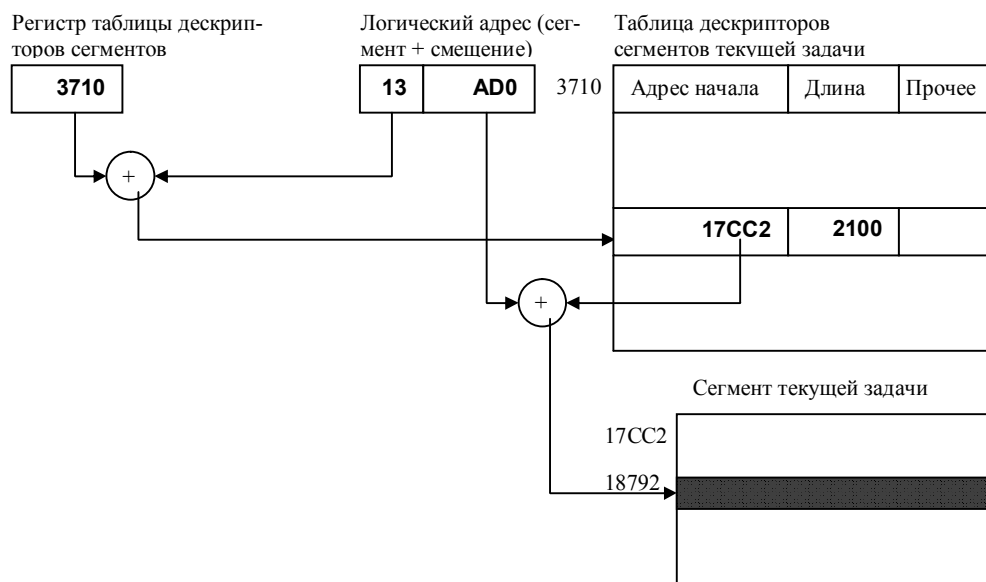


Рис. 4.8. Сегментный способ организации памяти

имеют одинаковую длину и размер, кратный степени двойки. При этом вместо последней операции сложения достаточно выполнить более простую операцию конкатенации. Для страничной организации памяти удобно проводить операцию замены адресов виртуальных страниц на реальные, описанную ранее. В остальном же механизм работы со страницами подобен работе с сегментами.

Основным достоинством страничной организации памяти является минимально возможная фрагментация (в среднем потери памяти равны половине страницы на задачу). Однако эта организация имеет и свои недостатки. Во-первых, из-за небольшого размера страницы (4 Кбайта для процессоров x86) объем таблицы страниц достаточно высок. Во-вторых, программы разбиваются на страницы случайно, без учета существующих логических взаимосвязей. Это приводит к тому, что межстраничные переходы выполняются гораздо чаще, чем межсегментные.

Сегментно-страничная организация памяти предназначена для избежания второго недостатка, правда, за счет увеличения накладных расходов. Ее суть заключается в том, что задача разделяется на сегменты, а последние – на страницы. Виртуальный адрес, таким образом состоит из трех частей: номера сегмента, номера страницы в сегменте и смещения в странице.

При загрузке в память одной страницы сегмента подгружаются все остальные страницы этого же сегмента. Таким образом, уменьшается число загрузок страниц.

4.4.3. Проблемы организации виртуальной памяти

Таким образом, виртуальная память - это совокупность программно-аппаратных средств, позволяющих пользователям писать программы, размер которых превосходит имеющуюся оперативную память; для этого виртуальная память решает следующие задачи:

- размещает данные в запоминающих устройствах разного типа, например, часть программы в оперативной памяти, а часть на диске;
- перемещает по мере необходимости данные между запоминающими устройствами разного типа, например, подгружает нужную часть программы с диска в оперативную память;
- преобразует виртуальные адреса в физические.

Все эти действия выполняются автоматически, без участия программиста, то есть механизм виртуальной памяти является прозрачным по отношению к пользователю.

При организации работы с виртуальной памятью возникают следующие две проблемы:

- когда загружать в оперативную память новую страницу – по запросу или предварительно;
- как определить страницу, которая должна быть выгружена из оперативной памяти.

При использовании сегментно-страничной организации памяти частично выполняется опережающая подкачка страниц. Во всех других случаях такой механизм признан нецелесообразным, поскольку вероятность определения того, к какой из отсутствующих в оперативной памяти страниц произойдет обращение в ближайшем будущем, крайне мала.

Для решения проблемы замещения используются следующие дисциплины:

- FIFO – выгружается та страница, которая находилась в памяти дольше всего;
- LRU (Least Recently Used) – выгружается страница, к которой дольше всего не было обращений;
- LFU (Least Frequently Used) – выгружается страница, к которой обращались реже всего;
- выгружается случайная страница.

В абсолютном большинстве современных операционных систем используется дисциплина LRU, однако для Windows NT применен механизм FIFO с предварительной буферизацией.

Если объем физической памяти небольшой и даже часто требуемые страницы не удастся одновременно загрузить в оперативную память, то возникает

ситуация *пробуксовки* – когда загрузка очередной страницы приводит к вытеснению той страницы, с которой мы также активно работаем. При этом основное время процессора тратится не на полезную работу, а на перемещение страниц из оперативной памяти во внешнюю и обратно. Чтобы не допускать пробуксовки, следует либо увеличить объем требуемой оперативной памяти, либо уменьшить количество одновременно выполняемых задач.

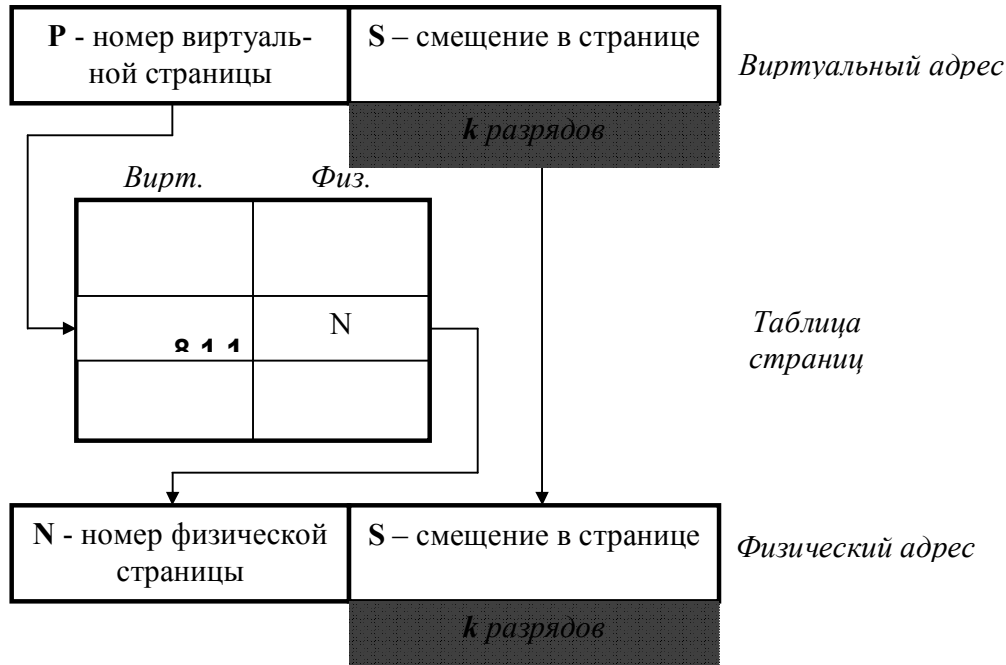


Рис. 4.9. Механизм преобразования виртуального адреса в физический при страничной организации памяти

Далее рассмотрим механизм преобразования виртуального адреса в физический при страничной организации памяти, показанный на рисунке 4.9..

Виртуальный адрес при страничном распределении может быть представлен в виде пары (p, s), где p - номер виртуальной страницы процесса (нумерация страниц начинается с 0), а s - смещение в пределах виртуальной страницы. Учитывая, что размер страницы равен 2^k в степени k, смещение s может быть получено простым отделением k младших разрядов в двоичной записи виртуального адреса. Оставшиеся старшие разряды представляют собой двоичную запись номера страницы p.

При каждом обращении к оперативной памяти аппаратными средствами выполняются следующие действия:

1. на основании начального адреса таблицы страниц (содержимое регистра адреса таблицы страниц), номера виртуальной страницы (старшие разряды виртуального адреса) и длины записи в таблице страниц (системная константа) определяется адрес нужной записи в таблице,
2. из этой записи извлекается номер физической страницы,
3. к номеру физической страницы присоединяется смещение (младшие разряды виртуального адреса).

Использование в пункте 3 того факта, что размер страницы равен степени 2, позволяет применить операцию конкатенации (присоединения) вместо более длительной операции сложения, что уменьшает время получения физического адреса и повышает производительность компьютера.

На производительность системы со страничной организацией памяти влияют временные затраты, связанные с обработкой страничных прерываний и преобразованием виртуального адреса в физический. При часто возникающих страничных прерываниях система может тратить большую часть времени впустую, на свопинг страниц. Чтобы уменьшить частоту страничных прерываний, следовало бы увеличивать размер страницы. Кроме того, увеличение размера страницы уменьшает размер таблицы страниц, а значит уменьшает затраты памяти. С другой стороны, если страница велика, значит велика и фиктивная область в последней виртуальной странице каждой программы. В среднем на каждой программе теряется половина объема страницы, что в сумме при большой странице может составить существенную величину. Время преобразования виртуального адреса в физический в значительной степени определяется временем доступа к таблице страниц. В связи с этим таблицы страниц стремятся размещать в "быстрых" запоминающих устройствах. Это может быть, например, набор специальных регистров или память, используемая для уменьшения времени доступа ассоциативный поиск и кэширование данных.

4.4.4. Свопинг

Рассмотрим зависимость коэффициента загрузки процессора в зависимости от числа одновременно выполняемых процессов и доли времени, проводимого этими процессами в состоянии ожидания ввода-вывода. На рисунке 4.10 показан график этой зависимости.

Из представленного графика видно, что для загрузки процессора на 90% достаточно всего трех счетных задач. Однако для того, чтобы обеспечить такую же загрузку интерактивными задачами, выполняющими интенсивный ввод-вывод, потребуются десятки таких задач. Необходимым условием для выполнения задачи является загрузка ее в оперативную память, объем которой ограничен. В этих условиях был предложен метод организации вычислительного процесса, называемый свопингом. В соответствии с этим методом некоторые процессы (обычно находящиеся в состоянии ожидания) временно

выгружаются на диск. Планировщик операционной системы не исключает их из своего рассмотрения, и при наступлении условий активизации некоторого процесса, находящегося в области свопинга на диске, этот процесс перемещается в оперативную память. Если свободного места в оперативной памяти не хватает, то выгружается другой процесс.

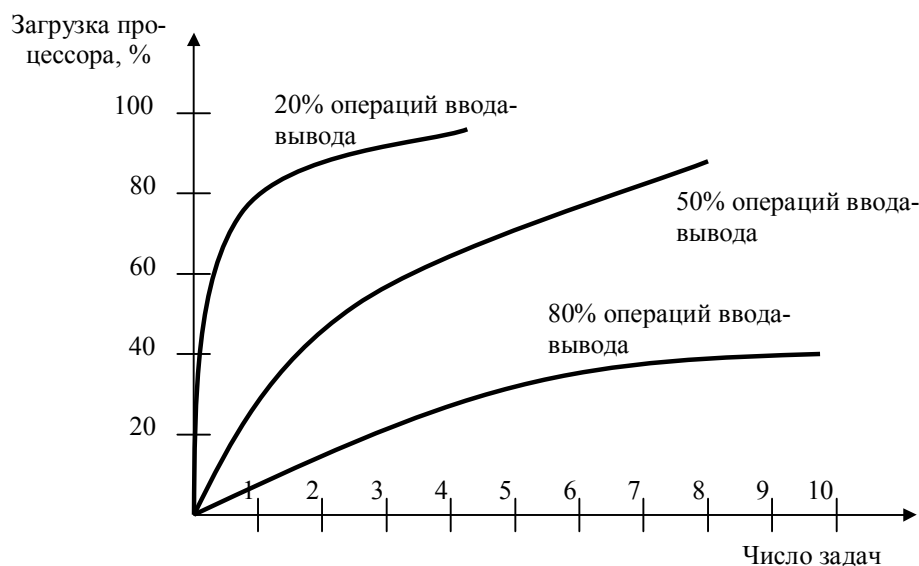


Рис. 4.10. График зависимости загрузки процессора от числа задач и интенсивности ввода-вывода

При свопинге, в отличие от рассмотренных ранее методов реализации виртуальной памяти, процесс перемещается между памятью и диском целиком, то есть в течение некоторого времени процесс может полностью отсутствовать в оперативной памяти. Существуют различные алгоритмы выбора процессов на загрузку и выгрузку, а также различные способы выделения оперативной и дисковой памяти загружаемому процессу.

4.5. Иерархия запоминающих устройств. Принцип кэширования данных

Память вычислительной машины представляет собой иерархию запоминающих устройств (внутренние регистры процессора, различные типы сверхоперативной и оперативной памяти, диски, ленты), отличающихся средним временем доступа и стоимостью хранения данных в расчете на один бит. Эта иерархия проиллюстрирована рисунком 4.11. Пользователю хотелось бы иметь и недорогую и быструю память.

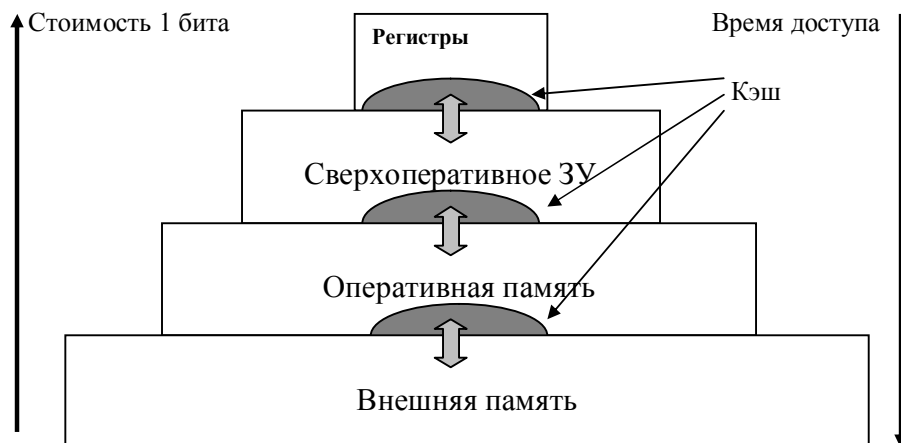


Рис. 4.11. Иерархия запоминающих устройств

Кэш-память представляет некоторое компромиссное решение этой проблемы. *Кэш-память* - это способ организации совместного функционирования двух типов запоминающих устройств, отличающихся временем доступа и стоимостью хранения данных, который позволяет уменьшить среднее время доступа к данным за счет динамического копирования в "быстрое" ЗУ наиболее часто используемой информации из "медленного" ЗУ.

Кэш-памятью часто называют не только способ организации работы двух типов запоминающих устройств, но и одно из устройств – «быстрое» ЗУ. Оно стоит дороже и, как правило, имеет сравнительно небольшой объем. Важно, что механизм кэш-памяти является прозрачным для пользователя, который не должен сообщать никакой информации об интенсивности использования данных и не должен никак участвовать в перемещении данных из ЗУ одного типа в ЗУ другого типа, все это делается автоматически системными средствами.

Рассмотрим частный случай использования кэш-памяти для уменьшения среднего времени доступа к данным, хранящимся в оперативной памяти. Для этого между процессором и оперативной памятью помещается быстрое ЗУ, называемое просто кэш-памятью. Содержимое кэш-памяти представляет собой совокупность записей обо всех загруженных в нее элементах данных. Каждая запись об элементе данных включает в себя адрес, который этот элемент данных имеет в оперативной памяти, и управляющую информацию: признак модификации и признак обращения к данным за некоторый последний период времени.

В системах, оснащенных кэш-памятью, каждый запрос к оперативной памяти выполняется в соответствии со следующим алгоритмом:

1. Просматривается содержимое кэш-памяти с целью определения, не находятся ли нужные данные в кэш-памяти; кэш-память не является адресуемой, поэтому поиск нужных данных осуществляется по содержимому - значению поля «адрес в оперативной памяти», взятому из запроса.
2. Если данные обнаруживаются в кэш-памяти, то они считываются из нее, и результат передается в процессор.
3. Если нужных данных нет, то они вместе со своим адресом копируются из оперативной памяти в кэш-память, и результат выполнения запроса передается в процессор. При копировании данных может оказаться, что в кэш-памяти нет свободного места, тогда выбираются данные, к которым в последний период было меньше всего обращений, для вытеснения из кэш-памяти. Если вытесняемые данные были модифицированы за время нахождения в кэш-памяти, то они переписываются в оперативную память. Если же эти данные не были модифицированы, то их место в кэш-памяти объявляется свободным.

Механизм работы с кэш-памятью показан на рисунке 4.12.

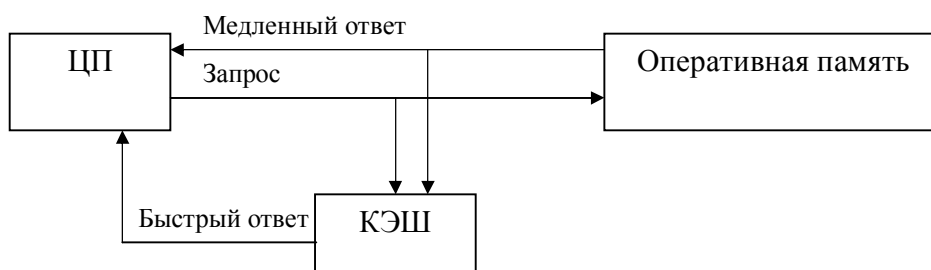


Рис. 4.12. Механизм работы с кэш-памятью

На практике в кэш-память считывается не один элемент данных, к которому произошло обращение, а целый блок данных, это увеличивает вероятность так называемого "попадания в кэш", то есть нахождения нужных данных в кэш-памяти.

Покажем, как среднее время доступа к данным зависит от вероятности попадания в кэш. Пусть имеется основное запоминающее устройство со средним временем доступа к данным t_1 и кэш-память, имеющая время доступа t_2 , очевидно, что $t_2 < t_1$. Обозначим через t среднее время доступа к данным в системе с кэш-памятью, а через p -вероятность попадания в кэш. По формуле полной вероятности имеем:

$$t = t_1(1 - p) + t_2(p)$$

Из нее видно, что среднее время доступа к данным в системе с кэш-памятью линейно зависит от вероятности попадания в кэш и изменяется от

среднего времени доступа в основное ЗУ (при $p=0$) до среднего времени доступа непосредственно в кэш-память (при $p=1$).

В реальных системах вероятность попадания в кэш составляет примерно 0,9. Высокое значение вероятности нахождения данных в кэш-памяти связано с наличием у данных объективных свойств: пространственной и временной локальности.

- *Пространственная локальность*. Если произошло обращение по некоторому адресу, то с высокой степенью вероятности в ближайшее время произойдет обращение к соседним адресам.

- *Временная локальность*. Если произошло обращение по некоторому адресу, то следующее обращение по этому же адресу с большой вероятностью произойдет в ближайшее время.

Все предыдущие рассуждения справедливы и для других пар запоминающих устройств, например, для оперативной памяти и внешней памяти. В этом случае уменьшается среднее время доступа к данным, расположенным на диске, и роль кэш-памяти выполняет буфер в оперативной памяти.

4.6. Аппаратные средства процессора x86 для управления памятью

Как уже было сказано ранее, эффективная работа механизмов распределения памяти и управления этим ресурсом невозможна только за счет программных средств. Необходимо, чтобы отдельные механизмы были реализованы на аппаратном уровне.

4.6.1. Реальный и защищенный режимы работы процессора

Оригинальный процессор i8086 предназначался для работы в однопрограммном режиме и не имел никаких механизмов защиты памяти. Однако к тому времени, как разработчики осознали необходимость включения в микропроцессор средств поддержки мультипрограммных вычислений, было создано много различных программных продуктов. Для того, чтобы обеспечить совместимость этих программ с новым «железом», была реализована возможность работы процессоров в двух режимах: *реальном* (real mode) и *защищенном* (protected mode). Реальный режим работы совпадает с единственным режимом работы первых 16-битовых процессоров. В защищенном режиме включаются возможности по аппаратной защите параллельных вычислений, основанные на особенностях 32-разрядной архитектуры. Рассмотрим особенности работы двух режимов применительно к доступу к оперативной памяти.

Напомним, что в реальном режиме память имеет сегментную организацию, и физический (эффективный) адрес получается из сегментного адреса и смещения в этом сегменте по формуле

$$\text{segment} * 16 + \text{offset}$$

Для хранения адресов сегментов используются специальные регистры CS, DS, ES, SS. В частности, адрес команды, которая будет выполнена следующей, хранится в паре регистров CS:IP. Сегментная организация памяти в реальном режиме работы процессора обусловлена несоответствием между длиной шины адреса (20 бит) и 16-битовым форматом хранимых данных.

При переходе на 32-битовую архитектуру данных и адресов необходимость в такой организации адресов отпадает, и понятие сегмента используется для организации работы в мультипрограммном режиме.

В защищенном режиме работы включаются в работу новые регистры, появившиеся в процессорах 286 и 386:

- EAX, EBX, ECX, EDX, EBP, ESP, ESI, EDI, EIP, EFLAGS, которые являются 32-битными расширениями соответствующих регистров процессора 8086;
- 16-битные *селекторы сегментов* CS, SS, DS, ES, FS, GS, первые четыре из них ранее использовались как сегментные регистры;
- 16-битные регистр-указатель задачи TR и регистр-указатель на *локальную таблицу сегментов* текущей задачи LDTR;
- 48-битный регистр GDTR на глобальную таблицу GDT, содержащую как дескрипторы общих сегментов, так и специальные системные дескрипторы.

При каждом из регистров CS, SS, DS, ES, FS, GS, TR, LDTR имеются скрытые от программиста 64-битовые теньевые регистры, в которые загружаются дескрипторы соответствующих сегментов.

4.6.2. Сегментный способ организации распределения памяти

Для организации эффективной и надежной работы вычислительной системы желательно выполнение следующих требований:

- чтобы у каждого вычислительного процесса было свое собственное адресное пространство, которое не пересекается с адресными пространствами других задач;
- чтобы существовало общее (разделяемое) адресное пространство, используемое для обмена данными и разделения кода.

Поэтому в процессорах x86 реализован сегментный способ организации памяти (помимо него, может быть задействована и страничная адресация).

Сегменты подразделяются на *глобальные*, к которым может обращаться любая задача, и *локальные*, доступ к которым имеет только одна из загруженных задач.

Неоднократно упоминавшийся ранее *дескриптор сегмента* представляет собой 64-битное описание сегмента, содержащее следующую информацию:

- признак нахождения сегмента в оперативной памяти;
- адрес начала сегмента (или информация о нахождении на внешних устройствах, если сегмент выгружен из ОП);
- длину сегмента (20 бит);
- уровень привилегий сегмента;
- некоторую другую информацию.

Дескрипторы глобальных сегментов и специальные системные дескрипторы образуют таблицу глобальных дескрипторов GDT. Эта таблица формируется на этапе запуска операционной системы, и ее адрес помещается в регистр GDTR.

Дескрипторы локальных сегментов для каждой задачи образуют свою таблицу LDT. Адрес этой таблицы помещается в регистр LDTR.

Инициализация сегмента LDTR происходит следующим образом. Для каждой запущенной задачи строится *сегмент состояния задачи* (TSS, task state segment). Дескриптор каждого TSS хранится в регистре GDT, а одно из полей TSS содержит содержимое регистра LDTR. При переключении между задачами смещение дескриптора TSS помещается в регистр TR, а содержимое поля LDTR – в одноименный регистр. Этот механизм проиллюстрирован на рисунке 4.13.

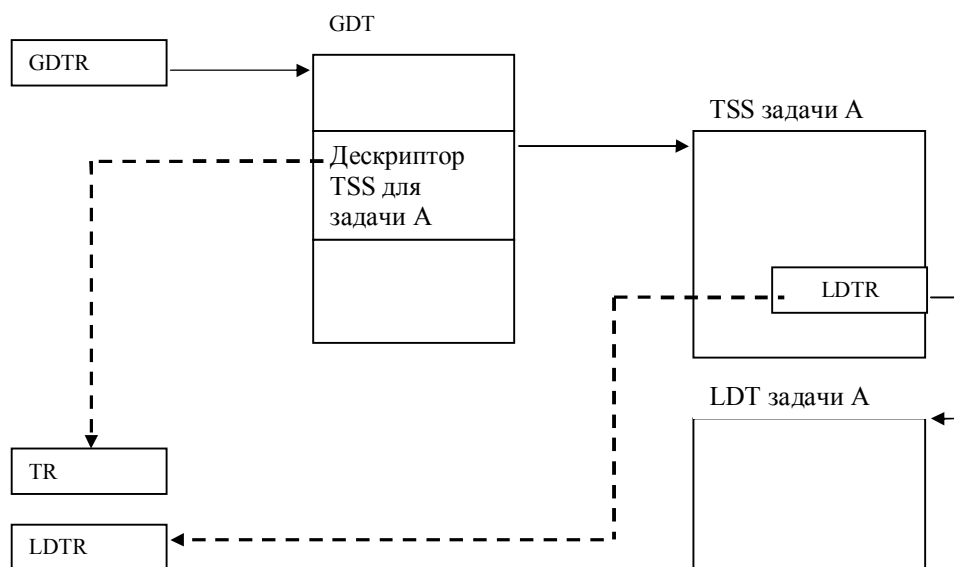


Рис. 4.13. Инициализация регистров при переключении на задачу А

Содержимое сегментных регистров (называемых в защищенном режиме селекторами сегментов) интерпретируется согласно следующей таблице:

15												3	2	1	0
Index (номер дескриптора)													TI RPL		

Здесь бит TI (table index) определяет, к какому из адресных пространств (глобальному или локальному) принадлежит запрашиваемый сегмент. Биты RPL (request privilege level) определяют уровень привилегий запрашиваемого сегмента. Наконец, поле Index определяет номер запрашиваемого сегмента, т.е. его смещение в соответствующей таблице дескрипторов.

При изменении содержимого любого из селекторов сегментов происходит перемещение соответствующего дескриптора в теневой регистр, связанный с этим селектором. Это сделано для ускорения доступа к памяти и реализовано следующим образом: вначале анализируется бит TI и определяется, к какому пространству адресов принадлежит данный сегмент. Затем происходит поиск дескриптора в таблицах GDT или LDT и загрузка найденного дескриптора в соответствующий теневой сегмент.

Формирование физического адреса на основании содержимого адресного регистра и селектора сегмента выполняется следующим образом. На основе анализа длины сегмента, хранящегося в дескрипторе, и смещения, задаваемого в адресном регистре, определяется ситуация «выход за границы сегмента». Затем базовый адрес начала сегмента складывается со смещением для определения т.н. линейного адреса. Если не включен механизм страничной трансляции адресов, то линейный адрес совпадает с физическим.

4.6.3. Страничный способ организации памяти

Если задача разбита на большое количество небольших по размеру сегментов, то рассмотренный выше способ организации памяти приводит к медленной работе (переключения между сегментами часты, а каждое такое переключение приводит к перезагрузке теневого регистра). Поэтому имеет смысл задуматься о реализации страничного механизма доступа к памяти.

При реализации страничного доступа к памяти разработчики процессора x86 приняли следующие решения:

- длина страницы составляет 4 Кбайта;
- для доступа к страницам используются *дескрипторы страниц*, содержащие 20-битовый адрес начала страницы, а также поля, описывающие страницу;
- дескрипторы страниц объединяются в двухуровневые таблицы. На верхнем уровне находится таблица PDE (page directory entry), на нижнем – таблица PTE (page table entry). Адрес PDE хранится в специальном управляющем регистре CR3, а адреса таблиц PTE – в элементах таблицы PDE.

В этом случае 32-битовый адрес делится на 3 части: смещение в таблице PDE (10 бит), смещение в соответствующей таблице PTE (также 10 бит) и,

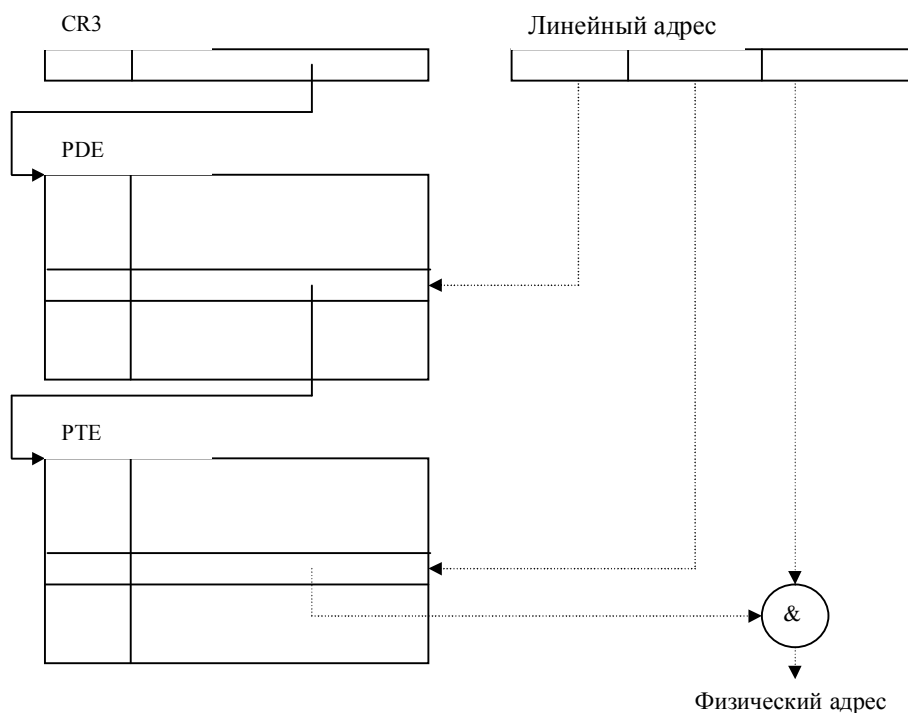


Рис. 4.14. Трансляция линейного адреса в физический

наконец, смещение в странице. Механизм соответствующей двойной трансляции адресов показан на рисунке 4.14.

Итак, микропроцессор позволяет иметь для каждой задачи одну таблицу PDE и несколько таблиц PTE. Значение адреса PDE хранится в TSS и загружается в регистр CR3, когда соответствующая задача становится активной. Это позволяет отказаться от сегментной адресации и считать, что вся задача состоит из единственного сегмента, состоящего из нескольких страниц. Этот подход, получивший название «плоской памяти», реализован в настоящее время в большинстве современных ОС.

Режим виртуальных машин. Для исполнения приложений, требующих реального режима работы процессора, введен специальный режим эмуляции 32-разрядных приложений. Этот режим называется режимом виртуального процессора или просто виртуальным режимом. Для организации виртуального режима используется специальный режим преобразования 16-битных адресов реального режима в 32-битные физические адреса. Это позволяет организовать параллельное выполнение нескольких DOS-приложений, де еще и совместно с 32-битными задачами.

4.6.4. Механизмы защиты памяти

При рассмотрении механизмов защиты памяти следует учесть два аспекта:

- защиты кодов и данных одного приложения от воздействия другого приложения;
- защиты операционной системы от воздействия запущенных ранее приложений.

Первый аспект защиты реализуется по-разному, в зависимости от того, какая модель памяти используется. При использовании сегментной модели организации памяти обращение к локальным сегментам не приведет к каким-либо проблемам (ибо у каждой задачи они свои). Если же объявить глобальные сегменты как системные, то задача свелась к защите системных областей, что будет рассмотрено позднее.

При реализации плоской модели памяти и страничной ее организации каждой задаче выделяется свое обособленное виртуальное адресное пространство, так что одно приложение влиять на другое не может.

Рассмотрим теперь вопросы организации защиты операционной системы от воздействия запущенных приложений. Для того, чтобы запретить пользовательским задачам модифицировать области памяти, принадлежащие самой ОС, необходимо иметь специальные средства. Для явного разграничения системных и пользовательских сегментов необходимо ввести несколько уровней (режимов) работы. В большинстве системных процессоров имеются по крайней мере 2 режима работы: *пользовательский* режим и режим *супервизора*. В последнем режиме программа может выполнять все действия и иметь доступ по любым адресам. Работа в пользовательском режиме должна быть ограничена. Например, в этом режиме недопустимо выполнение отдельных команд (ввода/вывода и др.) или обращение по определенным адресам. Можно сказать, что различные режимы работы процессора имеют различные уровни привилегий.

В процессорах x86 имеются не два, а четыре различных уровня привилегий. Эти уровни привилегий иногда называются *кольцами защиты*. Наиболее привилегированные программы выполняются в кольце защиты 0, а самый низкий уровень (кольцо защиты 3) отведен для прикладных программ. Промежуточные уровни привилегий введены для большей свободы прикладных программистов. Таким образом, режим супервизора соответствует выполнению программ в кольце защиты 0.

Каждый дескриптор сегмента имеет в своем составе двухбитовое поле DPL – уровень привилегии связанного с ним сегмента. При обращении к сегменту его селектор (т.е. регистр CS, DS и т.д.) содержит поле RPL – запрашиваемый уровень привилегий. DPL выполняемого сегмента кода имеет свое название - CPL.

В пределах одной задачи могут использоваться сегменты с различными уровнями привилегий. Механизм проверки привилегий работает в ситуациях, которые можно назвать межсегментными обращениями. Это – изменение сегментов данных или стека, а также межсегментные передачи управления в случае прерываний и использования команд JMP, CALL, INT и т.д. В межсегментных обращениях участвуют два сегмента: сегмент кода, из которого идет обращение (и соответственно поле CPL) и целевой сегмент, к которому мы обращаемся (с полями RPL и DPL).

При доступе к сегментам данных проверяется соотношение

$$CPL \leq \max(RPL, DPL)$$

Иными словами, прикладные программы не могут напрямую обращаться к системным данным, а обратный процесс возможен (например, в результате выполнения операции ввода/вывода заполняется буфер прикладной программы). Нарушение этого соотношения влечет ошибку защиты памяти.

При доступе к сегменту стека необходимо, чтобы

$$CPL = RPL = DPL$$

Соответственно, мы имеем 4 сегмента стека, каждый из которых соответствует своему кольцу защиты.

Правила для передачи управления, т.е. когда осуществляется межсегментная передача управления с одного кольца защиты на другой, несколько сложнее. Передача управления с высоко привилегированного сегмента на менее привилегированный должна контролироваться дополнительно (например, описанием соответствующей функции как callback). И обратно, нельзя просто так давать прикладным задачам возможность выполнить высоко привилегированный код. Для реализации возможностей передачи управления в сегменты с иным уровнем привилегий введен специальный механизм *шлюзования*, детальное рассмотрение которого выходит за рамки этой книги.

4.7. Распределение оперативной памяти в различных операционных системах

4.7.1. Распределение оперативной памяти в MS-DOS

Как известно, MS-DOS является однопрограммной операционной системой. Конечно, в ней можно организовать создание и выполнение резидентных программ, но в целом она предназначена для выполнения только одного вычислительного процесса. Поэтому распределение памяти построено по наиболее простой схеме – непрерывное распределение с переменными границами разделов.

Как известно, 16-разрядный процессор 8086 мог напрямую адресовать память до 1 Мбайта (точнее, до адреса FFFF:FFFF). В последующих ПК было

принято решение поддерживать совместимость с первыми, а для доступа к памяти свыше 1 Мбайта использовать механизм EMS, рассмотренный нами ранее. Поэтому рассмотрим распределение памяти в рамках 1 Мбайта.

Начальный адрес	Длина	Описание
00000	1 Кб	IVT (таблица векторов прерываний)
00400	512 байт	глобальные переменные BIOS и DOS
00600	35-60 Кб	Память, занятая модулями операционной системы (обслуживающие функции, рабочие и управляющие области, устанавливаемые драйверы). Резидентная часть интерпретатора команд COMMAND.COM
	≈ 580 Кб	Область памяти для выполнения программ пользователя. В конце этой области может располагаться транзитная часть COMMAND.COM
A0000	384 Кб	Системная область (UMA – upper memory area). В ней, в частности, располагается видеопамять при работе в текстовом режиме и режиме CGA, окна EMS и т.д.
100000-10FFFF	64Кб-16 байт	HMA (high memory area). при наличии драйвера HIMEM.SYS здесь можно расположить некоторые системные файлы MS-DOS, освобождая тем самым место в пределах первого мегабайта.

4.7.2. Распределение оперативной памяти в ОС Windows 95/98/ME

С точки зрения базовой архитектуры указанные в заголовке операционные системы являются 32-разрядными, многопоточными ОС с вытесняющей многозадачностью.

В этих системах применяется плоская модель памяти и страничный механизм ее распределения. Напомним, что при плоской модели все возможные сегменты, которые может использовать программист, совпадают друг с другом и имеют максимально возможный размер. Таким образом, сегментный механизм распределения памяти, описанный в предыдущих параграфах, остается практически незадействованным.

Все виртуальное адресное пространство размером в 4 Гбайта делится на следующие фрагменты:

- первые 64 Кбайта содержат значения системных переменных и недоступны для 32-разрядных программ (что дает возможность перехватывать неверные указатели). Однако 16-разрядные приложения могут записывать туда свои данные;

- область памяти до 4 Мбайт отображается в виртуальное адресное пространство каждой прикладной программы и используется совместно всеми процессами. Это сделано для обеспечения совместимости с резидентными программами и некоторыми 16-разрядными программами, входящими в состав Windows. С точки зрения надежности это – не самое лучшее решение;

- область памяти от 4 Мбайт + 1 до 2 Гбайт используется под «личные» адресные пространства 32-разрядных прикладных программ. За счет переключения каталогов страниц в момент переключения между процессами личные адресные пространства недоступны для других параллельно выполняющихся процессов;

- начиная с отметки 2 Гбайта, в памяти располагается системный код Windows. В области памяти от 2 до 3 Гбайт находятся системные DLL и DLL, совместно используемые несколькими приложениями, а также 16-разрядные прикладные программы Windows, которые разделяют одно и то же адресное пространство и в принципе могут испортить друг друга. Собственно ядро Windows, включающее подсистему управления виртуальными машинами, модули файловой системы и виртуальные драйверы, располагается в памяти по адресам свыше 3 Гбайт и работает в кольце защиты 0. Описываемая область памяти совместно используется всеми 32-разрядными приложениями. Такая организация позволяет обслуживать вызовы функций API непосредственно в адресном пространстве прикладной программы, однако за это приходится расплачиваться снижением надежности.

4.7.3. Распределение оперативной памяти в Windows NT

Заметим сразу, что различные модификации этих операционных систем (NT Server, NT Workstation) практически не отличаются по принципам распределения памяти, поэтому мы не будем различать эти модификации.

ОС Windows NT также использует плоскую модель памяти, однако схема распределения виртуального адресного пространства в ней сильно отличается от аналогичного распределения в Windows 95/98.

Во-первых, все системные программные модули находятся, как и прикладные, в своих собственных адресных пространствах, и доступ к ним со стороны прикладных программ невозможен. Ядро системы и несколько драйверов работают в нулевом кольце защиты.

Во-вторых, те программные модули, которые выступают как серверные процессы по отношению к прикладным программам-клиентам, также функ-

ционируют в своем собственном адресном пространстве, невидимом для прикладных программ.

В-третьих, прикладные программы полностью изолированы друг от друга, для обмена информацией между собой они должны использовать специальные системные механизмы (например, clipboard, OLE, DDE и т.д.).

Логическое распределение памяти при использовании Windows NT подчиняется следующим правилам:

- первые 64 Кбайта используются для системных целей и полностью недоступны для прикладных программ;

- память от 64 Кбайт до 2 Гбайт выделяется под «личные» адресные пространства прикладных программ. В верхней части каждой 2-гигабайтной области прикладной программы размещен код системных DLL кольца 3. Эти DLL выполняют перенаправление системных вызовов в совершенно изолированное адресное пространство (в том же интервале), где содержится собственно системный код. Этот код, называемый сервер-процессом, проверяет значения переданных параметров, исполняет запрошенную функцию и передает результаты обратно в адресное пространство прикладной программы;

- между отметками 2 и 4 Гбайта расположены системные компоненты кольца защиты 0 (ядро системы, планировщик потоков, менеджер памяти и т.д.). Использование физических схем кольцевой защиты процессора делает низкоуровневый код невидимым и недоступным для программ прикладного уровня.

В отличие от Windows 95/98, для 16-разрядных приложений Windows NT реализует сеансы WOW (Windows on Windows), и дает возможность выполнять их как совместно в разделяемом адресном пространстве, так и в отдельных пространствах. Аналогично происходит работа и с приложениями DOS.

Глава 5. Управление внешними устройствами и организация ввода-вывода

С самого начала развития вычислительных систем организация взаимодействия с внешними устройствами и ввода-вывода информации считалась одной из наиболее сложных и трудоемких задач, требующих очень высокой квалификации. Поэтому код, позволяющий осуществлять операции ввода-вывода, стали оформлять в виде системных библиотечных процедур; затем этот код был перенесен из систем программирования в операционные системы. Можно даже сказать, что системы управления вводом-выводом – это то зерно, из которого в настоящее время выросло все дерево операционных систем. В настоящее время управление вводом-выводом – это одна из основных функций любой операционной системы.

В рамках данной главы вначале рассмотрим общие концепции организации ввода-вывода, а затем более подробно рассмотрим организацию наиболее распространенных файловых систем.

5.1. Физическая организация устройств ввода-вывода

Устройства ввода-вывода делятся на два типа: *блок-ориентированные* устройства и *байт-ориентированные* устройства. Блок-ориентированные устройства хранят информацию в блоках фиксированного размера, каждый из которых имеет свой собственный адрес. Самое распространенное блок-ориентированное устройство - диск. Байт-ориентированные устройства не адресуемы и не позволяют производить операцию поиска, они генерируют или потребляют последовательность байтов. Примерами являются терминалы, строчные принтеры, сетевые адаптеры. Однако некоторые внешние устройства не относятся ни к одному классу, например, часы, которые, с одной стороны, не адресуемы, а с другой стороны, не порождают потока байтов. Это устройство только выдает сигнал прерывания в некоторые моменты времени.

Внешнее устройство обычно состоит из механического и электронного компонента. Электронный компонент называется контроллером устройства или адаптером. Механический компонент представляет собственно устройство. Некоторые контроллеры могут управлять несколькими устройствами. Если интерфейс между контроллером и устройством стандартизован, то независимые производители могут выпускать совместимые как контроллеры, так и устройства.

Операционная система обычно имеет дело не с устройством, а с контроллером. Контроллер, как правило, выполняет простые функции, например,

преобразует поток бит в блоки, состоящие из байт, и осуществляют контроль и исправление ошибок. Каждый контроллер имеет несколько регистров, которые используются для взаимодействия с центральным процессором. В некоторых компьютерах эти регистры являются частью физического адресного пространства. В таких компьютерах нет специальных операций ввода-вывода. В других компьютерах адреса регистров ввода-вывода, называемых часто портами, образуют собственное адресное пространство за счет введения специальных операций ввода-вывода (например, команд IN и OUT в процессорах i86).

Операционная система выполняет ввод-вывод, записывая команды в регистры контроллера. Например, контроллер гибкого диска IBM PC принимает 15 команд, таких как READ, WRITE, SEEK, FORMAT и т.д. Когда команда принята, процессор оставляет контроллер и занимается другой работой. При завершении команды контроллер организует прерывание для того, чтобы передать управление процессором операционной системе, которая должна проверить результаты операции. Процессор получает результаты и статус устройства, читая информацию из регистров контроллера.

5.2. Основные понятия и концепции организации ввода-вывода

5.2.1. Классификация устройств ввода-вывода

ОС должна поддерживать огромное число разнообразных устройств ввода-вывода, которые могут быть подключены к компьютеру. Эти устройства можно классифицировать по различным критериям. Первый и наиболее очевидный критерий – по функциональному назначению (принтеры, дисковые накопители, дисплеи и т.д.). На этом уровне мы абстрагируемся от конкретных физических характеристик внешнего устройства (его производителя, модели, емкости, быстродействия и т.д.) и рассматриваем общие принципы работы всех устройств данного типа.

Рассмотрим сейчас другие критерии, по которым можно классифицировать устройства ввода-вывода, поскольку они очень важны при проектировании и реализации подсистем ввода-вывода.

1. Различают устройства с *прямым* и *последовательным* доступом. Первые поддерживают доступ к любому элементу данных без доступа к предыдущим элементам; вторые требуют, чтобы данные были записаны или прочитаны строго в определенном порядке. Классическим примером устройств, допускающих прямой доступ, являются дисковые накопители, а устройств с последовательным доступом – магнитные ленты.

2. Устройства ввода /вывода делятся на *разделяемые* и *неразделяемые*. Первые могут быть одновременно предоставлены нескольким параллельно

выполняющимся задачам, вторые должны быть закреплены только за одной из таких задач. Очевидно, что последовательные устройства могут быть только неразделяемыми, хотя обратное неверно: трудно представить себе операцию форматирования диска совмещенной с какими-то другими обращениями к этому же диску!

3. Информация, передаваемая на внешние устройства или считываемая с них, может быть *структурированной* (она может быть разбита на порции, каждая из которых имеет определенный смысл) или *неструктурированной*. В первом случае говорят, что устройство ввода-вывода поддерживает *файловую структуру*, и каждую функционально обособленную единицу информации будем называть файлом.

4. Некоторые устройства могут сами инициировать ввод-вывод информации, подчиняясь внешним сигналам (например, клавиатура или мышь), другие работают только по командам от программного обеспечения.

5.2.2. Функциональная организация систем ввода-вывода

Система ввода-вывода должна выполнять следующие функции:

- организовывать эффективное распределение устройств ввода-вывода между несколькими параллельно выполняющимися процессами;
- осуществлять взаимодействие с внешними устройствами на физическом уровне;
- учитывать различную скорость работы центрального процессора и внешних устройств;
- предоставить прикладным программистам удобный и эффективный интерфейс виртуальных устройств ввода-вывода, избавив их от необходимости учитывать специфику работы каждого конкретного устройства.

Для эффективной реализации этих функций необходимо, чтобы все команды ввода-вывода на низком уровне могли выполняться только средствами ОС, а не пользовательскими программами. Для обеспечения этого принципа в большинстве процессоров даже вводятся два режима: обычный и привилегированный, причем операции ввода-вывода выполняются *супервизором ввода-вывода* только в привилегированном режиме.

Ввод-вывод может выполняться в двух основных режимах: *режиме обмена с опросом готовности устройства* и *режиме обмена с прерываниями*.

В обоих этих режимах супервизор ввода-вывода выполняет следующие действия:

- получает запрос на ввод-вывод от прикладных программ или других модулей операционной системы, проверяет этот запрос на корректность;

- планирует ввод-вывод, т.е. определяет очередность предоставления внешних устройств задачам, затребовавшим их. Запрос на ввод-вывод либо ставится в очередь, либо тут же выполняется;
- при выполнении запроса супервизор передает управление драйверу соответствующего устройства.

Дальнейшие действия по обслуживанию запроса зависят от одного из двух упомянутых режимов. В случае опроса готовности драйвер, обслуживающий внешнее устройство, время от времени посылает ему запрос, чтобы узнать о состоянии выполнения операции ввода-вывода. Этот режим наиболее прост в реализации, но приводит к значительным потерям быстродействия, т.к. внешнее устройство работает гораздо медленнее центрального процессора, а посылать новую команду, не дождавшись сигнала о выполнении предыдущей, бессмысленно. Гораздо выгоднее, выдав команду ввода-вывода, на время забыть о соответствующем устройстве и перейти к выполнению других задач. Внешнее устройство, завершив свою работу, инициирует прерывание, которое возобновит работу соответствующего драйвера. Для того, чтобы не потерять связь с устройством, можно установить величину тайм-аута, т.е. задать время, в течение которого устройство должно вызвать прерывание. Если этого не произойдет, то связь с устройством считается разорванной.

После завершения обработки запроса супервизор ввода-вывода передает соответствующее сообщение программе, инициировавшей этот запрос.

5.2.3. Спулинг

Как известно, многие устройства, и прежде всего устройства с последовательным доступом, не допускают совместного использования. Такие устройства могут быть только закрепленными, т.е. предоставленными некоторому процессу на все время жизни этого процесса. Однако это приводит к тому, что многие вычислительные процессы (например, использующие в своей работе принтер) просто не смогут выполняться параллельно. Для организации использования многими параллельно выполняющимися задачами устройств ввода-вывода, которые не могут быть разделяемыми, используется принцип *спулинга* (SPOOL – сокращение термина simultaneous peripheral operation on-line). Суть спулинга заключается в том, что устройство передается в монопольное распоряжение специальному процессу - спулеру. Этот процесс, в свою очередь, принимает заявки на обслуживаемое устройство от других задач, формирует из этих запросов очередь и работает с этой очередью. Таким образом, мы можем работать с виртуальным устройством, которое уже является разделяемым. Классическим примером спулера является диспетчер печати Windows.

5.2.4. Асинхронный ввод-вывод

Задача, выдавшая запрос на операцию ввода-вывода, переводится супервизором в состояние ожидания до тех пор, пока заказанная операция не завершится. Эта ситуация соответствует *синхронному* вводу/выводу. Однако, если выводимые или читаемые порции данных сравнительно невелики, а между различными операциями проходит достаточно много времени, то имеет смысл отказаться от синхронности. Действительно, время обработки запроса на ввод-вывод не растет пропорционально объему передаваемой информации, т.к. много времени затрачивается на подготовку и завершение ввода-вывода. (Кстати, спулинг является одним из примеров асинхронного ввода-вывода!)

Простейшим вариантом асинхронного вывода является т.н. *буферизация*. При ее использовании данные из приложения передаются не непосредственно на внешнее устройство, а в специальный системный буфер. С точки зрения приложения, запрос на вывод считается завершенным, и оно может продолжать свою работу. Реальное обращение к внешнему устройству выполняется тогда, когда заполнится буфер (или завершится приложение, которое этот буфер заполняло). Если использовать несколько буферов (как минимум, два), то можно распараллелить работу по заполнению одного буфера и очистке другого.

Другим способом реализации асинхронного ввода-вывода является *кэширование*, которое используется в основном при работе с дисками. Суть этого метода заключается в том, что при запросе на чтение данных читается не только запрошенная информация, но и данные, расположенные рядом. Вся прочитанная информация помещается в специальный буфер – дисковый кэш. Здесь используется эвристический принцип «данные, расположенные рядом, используются совместно». При последующих чтениях в первую очередь проверяется, не находится ли запрошенная информация в кэше, и только в случае ее отсутствия производится обращение к диску. Аналогично идет и запись данных: в случае включения механизма «отложенной записи» (lazy write) данные вначале помещаются в кэш, а записываются на внешние устройства спустя некоторое время.

Кэш может быть организован как в оперативной памяти, так и на самом винчестере (т.н. read ahead buffer).

5.3. Логическая организация данных

5.3.1. Общие положения

Как уже было сказано ранее, некоторые внешние устройства дают возможность структурировать данные, так что ОС могут выполнять ряд стандартных действий над данными (таких, например, как создание, уничтожение или переименование). В рамках настоящего раздела под *файлом* будем пони-

мать именованную совокупность данных, представляющую с точки зрения операционной системы единое целое.

Файловая система - это часть операционной системы, назначение которой состоит в том, чтобы обеспечить пользователю удобный интерфейс при работе с данными, хранящимися на диске, и обеспечить совместное использование файлов несколькими пользователями и процессами.

Ранее уже говорилось о том, что понятие «файловая система» весьма многозначно. В частности, под ним понимается:

- совокупность всех файлов на диске;
- принципы организации данных на внешних носителях на логическом уровне;
- реализация этих принципов на физическом уровне;
- программное обеспечение для реализации этих принципов.

В этом параграфе будут рассмотрены два первых аспекта файловых систем.

5.3.2. Логическая организация данных в ОС UNIX

Хорошо известно, как логически организованы данные в ОС от Microsoft: логический диск, обозначаемый буквой, структура каталогов, правила именования файлов и их атрибуты. Поэтому мы не будем подробно останавливаться на этом, а перейдем к рассмотрению принципов логической организации данных в операционных системах семейства UNIX.

С логической точки зрения файлы в UNIX организованы в виде единой древовидной структуры: / обозначает корневой каталог, /usr/guest/letter1 – файл (или каталог) letter1 в каталоге guest, который в свою очередь находится в каталоге usr корневого каталога.

Каждый каталог и файл файловой системы имеет уникальное полное имя (в ОС UNIX это имя принято называть full pathname - имя, задающее полный путь, поскольку оно действительно задает полный путь от корня файловой системы через цепочку каталогов к соответствующему каталогу или файлу; мы будем использовать термин «полное имя», поскольку для термина pathname отсутствует благозвучный русский аналог). Каталог, являющийся корнем файловой системы (корневой каталог), в любой файловой системе имеет предопределенное имя /. Полное имя файла, например, /bin/sh означает, что в корневом каталоге должно содержаться имя каталога bin, а в каталоге bin должно содержаться имя файла sh. Коротким или относительным именем файла (relative pathname) называется имя (возможно, составное), задающее путь к файлу от текущего рабочего каталога. В каждом каталоге содержатся два специальных имени: имя «.», именуемое сам этот каталог, и имя «..», именуемое родительский каталог данного каталога, т.е. каталог, непосредственно предшествующий данному в иерархии каталогов.

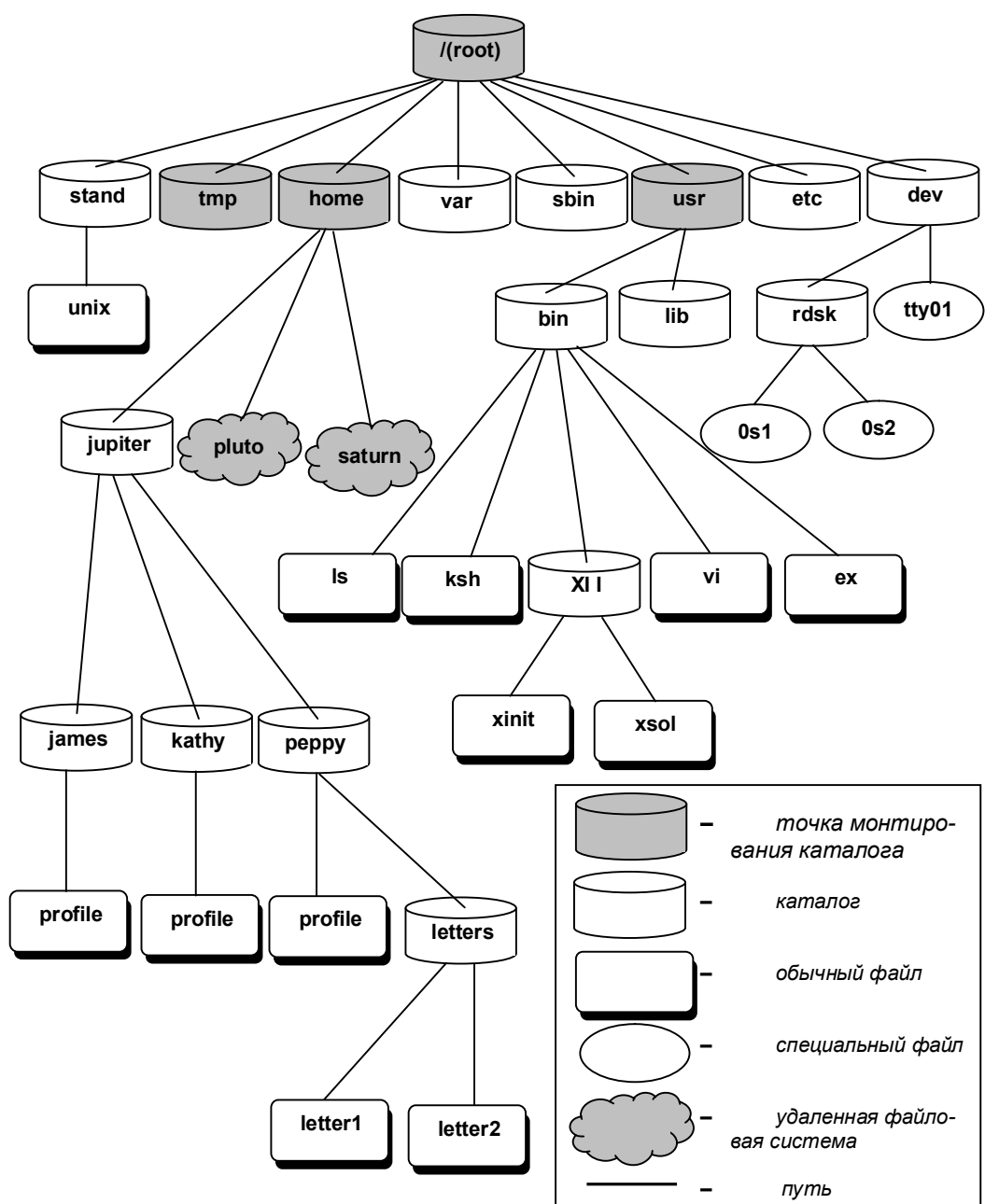


Рис. 5.1. Структура каталогов файловой системы UNIX

Как можно заметить, организация данных по сравнению с ОС фирмы Microsoft имеет как сходные черты, так и отличия. Одно из этих отличий – отсутствие понятия логического диска (drive). Действительно, для несъемных

дисков (да и для CD-ROM) использование логических имен создает определенные неудобства для пользователя. С другой стороны, каталог `/dev` содержит специальные файлы для обращения к конкретным внешним устройствам (например, `/dev/fd0` – для обращения к дискете). специальная команда *монтирования* устройств позволяет встроить файловую систему этих устройств в общую структуру данных. Так, команда

```
mount /dev/fd0 /floppy
```

дает возможность обращаться к содержимому данной дискеты через каталог `/floppy`.

В общем случае механизм монтирования заключается в формировании единого дерева файловой системы из различных поддеревьев, возможно, находящихся на различных физических устройствах. Для этого в структуру файловой системы встраиваются специальные каталоги – точки монтирования (mount points), вместо которых и интегрируется присоединяемая файловая система.

На рисунке 5.1 приведен пример иерархии файловой системы UNIX.

Другие отличия в логической организации данных UNIX не столь очевидны, и о них стоит рассказать отдельно.

Во-первых, файлы имеют собственные идентификаторы, как правило, не используемые, а имена в каталогах содержат *ссылки* на эти файлы. Таким образом, обращение к одному и тому же файлу может быть выполнено из различных каталогов.

Второе отличие заключается в ограничениях на доступ к файлу. ОС UNIX изначально проектировалась как многопользовательская, многотерминальная система. Поэтому все пользователи должны быть зарегистрированы перед началом работы. Пользователи могут объединяться в группы, так что один и тот же пользователь может принадлежать к разным группам. Наконец, среди пользователей выделяется один – *суперпользователь* (с именем superuser или root), который владеет практически неограниченными правами.

В качестве атрибутов каждого файла UNIX указываются:

- владелец (пользователь, создавший файл);
- группа (та группа, к которой относился владелец файла при его создании).

При попытке доступа к файлу различают три категории пользователей: владелец (owner), члены группы, к которой относится файл (group) и прочие пользователи (other). Режим доступа для каждой из этих категорий указывается отдельно.

Что касается видов доступа к файлу, то для файла можно указать разрешение на выполнение следующих операций: чтение (“r”), модификация (“w”)

и выполнение (“x”). Таким образом, режим доступа к файлу может быть задан следующей комбинацией (первые три символа относятся к владельцу, вторые – к членам группы, третьи – к прочим пользователям, знак «минус» означает запрет на соответствующий тип доступа):

rwxr-x---

Это означает, что владелец файла имеет полный доступ к нему, членам группы запрещено изменять файл, а для прочих пользователей этот файл недоступен.

5.4. Физическая организация данных на магнитных дисках

5.4.1. Общие требования к организации данных

При рассмотрении вопроса о физической организации данных надо учитывать следующие соображения:

1) Как известно, единицей обмена информацией между диском и портом компьютера на физическом уровне является *сектор*. Объем сектора в принципе может быть изменен, но большинство ОС работает с секторами размером в 512 байт. Вся совокупность физических секторов на винчестере представляет собой его неформатированную емкость.

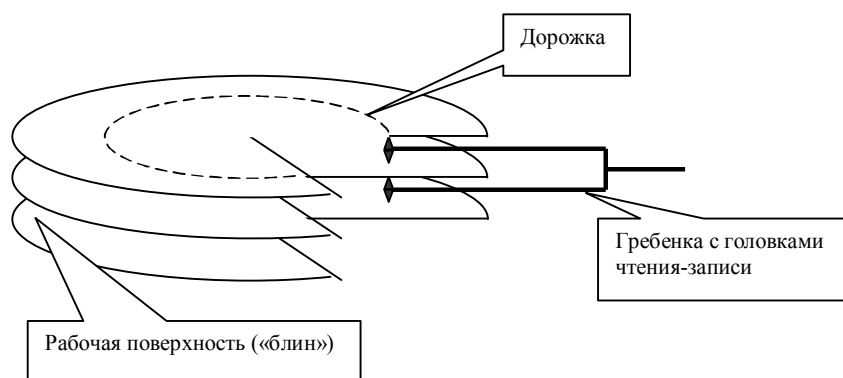


Рис. 5.2. Схематическое устройство магнитного диска

2) Информация на диске записывается на магнитных поверхностях – «блинах», причем один диск содержит несколько рабочих поверхностей (см. рисунок 5.2.)². Между поверхностями диска с помощью шагового двигателя перемещается гребенка с несколькими головками чтения-записи. Информация, считываемая (или записываемая) одной головкой при неподвижной гре-

² В современных винчестерах механизм размещения данных на физическом уровне более сложен, но детальное рассмотрение такого механизма выходит за рамки этой книги.

бенке, составляет одну *дорожку*. Соответственно, *цилиндром* будем называть совокупность информации, считываемой всеми головками при неподвижной гребенке. На уровне контроллера диска адрес для доступа к данным задается в виде триады [c-t-s], где c – номер цилиндра, t – номер дорожки на цилиндре и s – номер сектора на дорожке. Очевидно, что доступ к данным, расположенным в пределах одного цилиндра, происходит существенно быстрее, нежели к данным того же объема, но разбросанным по разным цилиндрам. Таким образом, желательно, чтобы информация, соответствующая одному файлу располагалась на одном или нескольких смежных цилиндрах, или, по крайней мере, в смежных секторах.

3) С другой стороны, необходимо обеспечить возможность произвольного изменения размера файлов, их удаления, выделения места под вновь создаваемые файлы. При этом освободившееся при удалении или уменьшении размера файлов место должно быть доступно для повторного распределения. Заметим, что это требование вступает в противоречие с только что высказанным тезисом о смежном расположении файла.

4) Определение операционной системы, которая должна быть загружена, должно выполняться автоматически. Кроме того, в настоящее время все чаще требуется, чтобы с одного диска запускались несколько различных ОС (т.е. чтобы выполнялась *мультизагрузка*).

Вначале рассмотрим размещение данных на диске в целом, а затем, в качестве примеров, организацию файловой системы FAT (с различными модификациями).

5.4.2. Разбиение диска на разделы

Для того, чтобы можно было загрузить с диска любую операционную систему, требуется, чтобы были приняты специальные соглашения о структуре диска, общие для всех ОС. Расположение информации о структуре диска и простейшей, не зависящей от операционной системы программы (non-system bootstrap, первичный загрузчик), с помощью которой выполняется загрузка конкретной системы, очевидно: это начальные сектора диска³.

Согласно спецификациям, сложившимся де-факто и ставшими в настоящее время стандартными, жесткий диск при подготовке к работе разбивается на *главную загрузочную запись* (master boot record, MBR) и несколько *разделов* (partitions), которые в дальнейшем могут быть использованы одной либо различными ОС. На каждом разделе может быть организована своя файловая система. Однако для организации единственной файловой системы требуется, чтобы диск содержал, по крайней мере, один раздел. Дальнейшее изложение будет вестись для т.н. спецификаций DOS.

³ В случае мультизагрузки первичный загрузчик также организует диалог с пользователем для определения той ОС, которую требуется загрузить.

MBR располагается по физическому адресу [0-0-1] и содержит:

- первичный загрузчик;
- таблицу описания разделов диска (partition table), которая содержит информацию о разделах диска.

Разделы диска могут быть объявлены как *первичные* (primary) и *расширенные* (extended). Максимальное число разделов – 4, при этом на диске должно присутствовать не менее одного первичного раздела. Если первичных разделов несколько, то только один из них может быть объявлен *активным*. Именно загрузчику ОС, расположенному в активном разделе, передается управление первичным загрузчиком (исключая случаи мультизагрузки).

Помимо первичных, на жестком диске может быть расположен один расширенный раздел, разбитый, в свою очередь, на несколько логических дисков. Перед каждым диском записывается т.н. *расширенная таблица разделов*, структура которой аналогична структуре partition table. Как правило, диск, разбитый на разделы программой **fdisk** от Microsoft, содержит один первичный раздел, отведенный для диска C:, и (необязательно) один расширенный раздел, на котором располагаются диски D:, E: и т. д.

Если в компьютере физически установлено несколько винчестеров⁴, то имена логическим дискам присваиваются в следующем порядке: вначале активные первичные разделы, а затем логические диски расширенных разделов. Такой принцип именования логических дисков не совсем удобен: при временном подключении второго винчестера (например, для переноса большого объема данных) обозначения «родных» дисков сдвигаются, что приводит к неправильной работе многих программ.

Структура логического диска подчиняется правилам конкретной файловой системы. Что касается дискет, их структура аналогична структуре одного логического диска винчестера.

5.4.3. Файловая система FAT

При проектировании этой системы в качестве основополагающего критерия был выбран принцип простоты выделения и освобождения места под файлы. При этом принципом хранения информации, соответствующей одному файлу, в смежных участках, приходится пожертвовать.

В файловой системе FAT все логическое дисковое пространство разбивается на следующие области:

- загрузочную запись, содержащую один сектор;
- несколько *зарезервированных* секторов;
- две копии *таблицы размещения файлов* (file allocation table, FAT)
- корневого директория диска (root directory);

⁴ Напомним, что по распространенному интерфейсу IDE к компьютеру может быть присоединено до 4 устройств (жестких дисков или CD-ROM'ов).

- области данных, разбитой на *кластеры*.

Загрузочная область содержит собственно программу начальной загрузки, а также *блок параметров диска*, содержащий информацию о размерах остальных областей диска.

Кластер представляет собой один или несколько смежных секторов в области данных и является единицей выделения дискового пространства под файл или директорию, за исключением корневого директория (в файловой системе FAT32 корневой директорий также размещается в выделяемом дисковом пространстве). Номера кластеров начинаются с 2.

Все директории (каталоги) состоят из элементов, каждый из которых соответствует одному файлу или поддиректорию. Таким образом, нет никаких различий в распределении памяти между поддиректорией и любым другим файлом.

Для случая, когда имена файлов формируются по правилам MS-DOS: имя файла содержит не более восьми символов, а расширение – не более трех (так называемый *формат 8.3*), каждый элемент директория содержит следующие поля:

- имя и расширение файла;
- атрибуты файла (volume, directory, archive, system, hidden, read-only);
- дата и время последней модификации файла;
- дата и время создания файла (только в случае FAT32);
- размер файла;
- номер первого кластера, соответствующего данным файла.

В случае длинных имен файлов в описанном формате хранится т.н. *имя MS-DOS*, представляющее собой псевдоним файла в формате 8.3, под которым файлы с длинными именами видны из DOS-приложений. Само длинное имя также записывается в директории, но формат его совершенно другой. В частности, один элемент директория может содержать до 13 символов длинного имени в формате Unicode, так что длинное имя файла помещается в нескольких элементах директория. Специальная комбинация атрибутов (volume + directory), бессмысленная с точки зрения DOS, делает такие элементы директория невидимыми для DOS-приложений. Такая организация хранения длинных имен, конечно же, не оптимальна, но она предложена для совместимости со старыми системами (причем совместимость достигается в обе стороны, т. е. не только новые приложения видят старые данные, но и старые приложения могут ограниченно работать с файлами, имеющими длинные имена).

Перейдем сейчас к рассмотрению собственно FAT. Эта область также состоит из элементов длиной в 12, 16 или 32 бита, каждый из которых соответствует одному кластеру файла. Элементы FAT могут содержать следующие значения:

- 0: соответствующий кластер свободен;
- 0FFFFh: кластер является последним для некоторого файла;
- 0FFF7h: кластер содержит сбойные участки; запись данных в этот кластер запрещена;
- 2 – 0FFF6h: номер следующего кластера, занятого файлом, которому выделен соответствующий кластер.

Таким образом, информация, содержащаяся в элементе каталога и в FAT, представляет собой набор списков номеров кластеров. Каждому файлу соответствует свой список, причем начало этого списка хранится в директории, а продолжение - в FAT. На рисунке 5.3 проиллюстрирован случай распределения данных по кластерам в случае записи двух файлов в корневой директории.

		0	4	7	FILE1		3			
6	8	E	E	0				FILE1	FILE1	FILE2
0	0	0	0	0				FILE2	FILE1	FILE2
0	0	0	0	0	FILE2		5			

Начальные элементы FAT (E – признак конца файла)

Элементы корневой директории (несущественные поля заштрихованы)

Распределение файлов по кластерам (два первых кластера не используются)

Рис. 5.3. Пример распределения файлов по кластерам

Различают следующие модификации файловой системы FAT:

- FAT12. Размер элемента FAT, как ясно из названия, равен 12 битам. Размер кластера не превосходит 4 Кбайт. Самая ранняя версия файловой системы, которая позволяла работать с логическими дисками до 30 Мбайт. В настоящее время используется лишь для дискет;
- FAT16. Увеличение размера элемента FAT позволяет увеличить число кластеров. Кроме того, сняты ограничения на размер кластера, так что этот размер растет с объемом логического диска;
- VFAT (virtual FAT) представляет собой модификацию FAT16, предназначенную для выполнения файлового ввода-вывода в защищенном режиме. С выходом Windows 95 в VFAT появилась возможность поддержки длинных имен;
- FAT32. Введение 32-битовых элементов FAT приводит к уменьшению размера кластера. Так, для современных винчестеров, даже не разбитых на логические диски, размер кластера в FAT32 составляет 4 Кбайта. Кроме

того, FAT32 позволяет хранить корневой директорий в виде цепочки кластеров, подобно всем другим каталогам. Это снимает ограничения на его размер в 512 элементов, принятый в более ранних версиях. С другой стороны, размер самой таблицы FAT существенно увеличивается, так что на дисках размером менее 500 Мбайт рекомендуется обходиться ранними версиями этой файловой системы.

В заключение обсудим сильные и слабые стороны файловой системы FAT. К положительным качествам следует отнести:

- простоту реализации;
- относительно малый объем, отводимый под служебные области;
- возможность легко изменять размер файлов и структуру директориев.

Недостатками этой системы является:

- сравнительно медленный поиск данных за счет того, что используются списковые структуры, которые к тому же неотсортированы;
- фрагментированность файлов. С течением времени процент фрагментированных файлов растет, что замедляет доступ к данным и вынуждает время от времени выполнять специальные программы дефрагментации;
- из-за большого размера кластеров, особенно при использовании FAT16, возрастают объемы выделенного, но не используемого дискового пространства. Считается, что в среднем потери составляют 0.5 кластера на файл, т.к. половина последнего кластера не заполняется. При объеме логического диска свыше 511 Мбайт размер кластера составляет 16 Кбайт, что в реальности приводит к потерям около 20 процентов дискового пространства;
- отсутствие механизмов ограничения доступа к данным;
- слабая защищенность от сбоев системы. В частности, при аварийном завершении работы системы могут появиться потерянные цепочки кластеров и скрещивающиеся файлы. Время от времени следует запускать программы проверки диска для устранения этих ошибок.

5.4.4. Файловая система NTFS

Файловая система HPFS (High Performance File System) была создана IBM и Microsoft для системы OS/2 и является предшественницей NTFS. Предполагалось, что эта система будет быстрой, поддерживающей длинные имена файлов, небольшие размеры кластеров, не имеющей проблем с дефрагментацией и поддерживающей больше атрибутов. HPFS, являясь предшественницей NTFS, является по сути дела NTFS без атрибутов безопасности. Вместо того, чтобы хранить цепочки кластеров в связанном списке, HPFS сохраняет всю информацию в сортированном В-дереве, что значительно ускоряет поиск файлов. Вместо того, чтобы хранить таблицы директориев и другие дескрипторы в начале диска, HPFS размещает их по всему диску через определенные

интервалы, поэтому для того, чтобы прочитать таблицы, головки диска должны добраться до ближайшей таблицы, а не до начала диска.

Файловая система NTFS (New Technology File System) обеспечивает такое сочетание производительности, надежности и эффективности, которое невозможно получить с помощью FAT. Основными целями разработки NTFS являлись:

- обеспечение скоростного выполнения стандартных операций над файлами, таких как чтение, запись, поиск;
- предоставление дополнительных возможностей, включая восстановление поврежденной файловой системы на чрезвычайно больших дисках.

Файловая система NTFS является стандартной файловой системой для Windows NT и других операционных систем, основанных на этой технологии (Windows 2000, Windows XP). Эти же операционные системы могут работать и с различными модификациями файловой системы FAT. С другой стороны, системы Windows линейки 9x (т.е. Windows 95/98/ME) не поддерживают работу с NTFS. Поэтому, если использовать на одном компьютере несколько систем из различных семейств Windows, например, Windows 98 и Windows XP, то загрузочный том нельзя форматировать в NTFS.

NTFS обладает характеристиками защищенности, поддерживая контроль доступа к данным и привилегии владельца, играющие исключительно важную роль в обеспечении целостности важных данных. Папки и файлы NTFS могут иметь назначенные им права доступа вне зависимости от того, являются ли они разделяемыми или нет. NTFS - единственная файловая система в Windows, которая позволяет назначать права доступа к различным файлам. Устанавливая пользователям определенные разрешения для файлов и каталогов, пользователь может защищать конфиденциальную информацию от несанкционированного доступа. Разрешения пользователя на доступ к объектам файловой системы работают по принципу дополнения. Это означает, что действующие разрешения образуются из всех прямых или косвенных разрешений, назначенных пользователю для данного объекта с помощью логической функции «или». Например, если пользователь имеет разрешение для каталога на чтение, а косвенно через членство в группах ему дано право на запись, то в результате пользователь сможет читать информацию в файлах каталога и записывать в них данные. Одним из свойств файловой системы NTFS является возможность введения квот. Это свойство необходимо системным администраторам. После установки квот пользователь может хранить на томе ограниченный объем данных, в то время как на этом диске может оставаться свободное пространство. Если пользователь превысит выданную ему квоту, в журнал событий будет внесена соответствующая запись. Среди новшеств файловой системы NTFS 5 следует отметить точки монтирования, механизм работы которых сходен с аналогичным механизмом для UNIX, опи-

санным в п. 5.3.2. Пользователь может определить различные, несвязанные с собой папки и даже диски в системе как одну виртуальную папку или диск. Это имеет большое значение для описания в одном месте разнородной информации, находящейся в системе. Файлы и папки, созданные таким образом, имеют уникальный идентификационный номер, что гарантирует их правильное нахождение в системе, даже если они были перенесены.

5.4.5. Физическая организация данных в файловой системе NTFS

Каждый файл на томе NTFS представлен записью в специальном файле, называемом *главной файловой таблицей* (MFT — master file table). NTFS резервирует первые 16 записей таблицы для специальной информации. Так, первая запись этой таблицы описывает непосредственно главную файловую таблицу, за ней следует *зеркальная запись* (mirror record) MFT. Если первая запись MFT разрушена, то NTFS читает вторую запись для отыскания зеркального файла MFT, первая запись которого идентична первой записи MFT. Местоположения сегментов данных MFT и зеркального файла MFT записаны в секторе начальной загрузки. Дубликат сектора начальной загрузки находится в логическом центре диска. Третья запись MFT — *файл регистрации* (log file), который используется для восстановления файлов.

Семнадцатая и последующие записи главной файловой таблицы используются собственно файлами и каталогами (также рассматриваются как файлы NTFS) на томе.

Упрощенная структура MFT представлена на рисунке 5.4.

Главная файловая таблица отводит определенное количество пространства для каждой записи файла. Атрибуты файла записываются в распределенное пространство MFT. Небольшие файлы и каталоги (обычно до 1500 байт или меньше) могут полностью содержаться внутри главной файловой таблицы. Подобный подход обеспечивает очень быстрый доступ к файлам. Рассмотрим, например, файловую систему FAT, которая использует таблицу размещения файлов, в которой перечисляются имена и адрес каждого файла. Элементы каталога FAT содержат индекс в таблице размещения файла. В случае если необходимо просмотреть содержимое файла, FAT сначала читает таблицу размещения файлов и убеждается в существовании файла. Далее FAT восстанавливает файл, ища цепочку распределенных блоков, относящихся к этому файлу. В NTFS поиск файла производится только для непосредственного его использования. Записи каталога помещены внутри главной файловой таблицы так же, как записи файла. Вместо данных каталоги содержат индексную информацию. Небольшие записи каталогов находятся полностью внутри структуры MFT. Большие каталоги организованы в В-дереве, имея записи с указателями на внешние *кластеры*, содержащие элементы каталога, которые не могли быть записаны внутри структуры MFT.

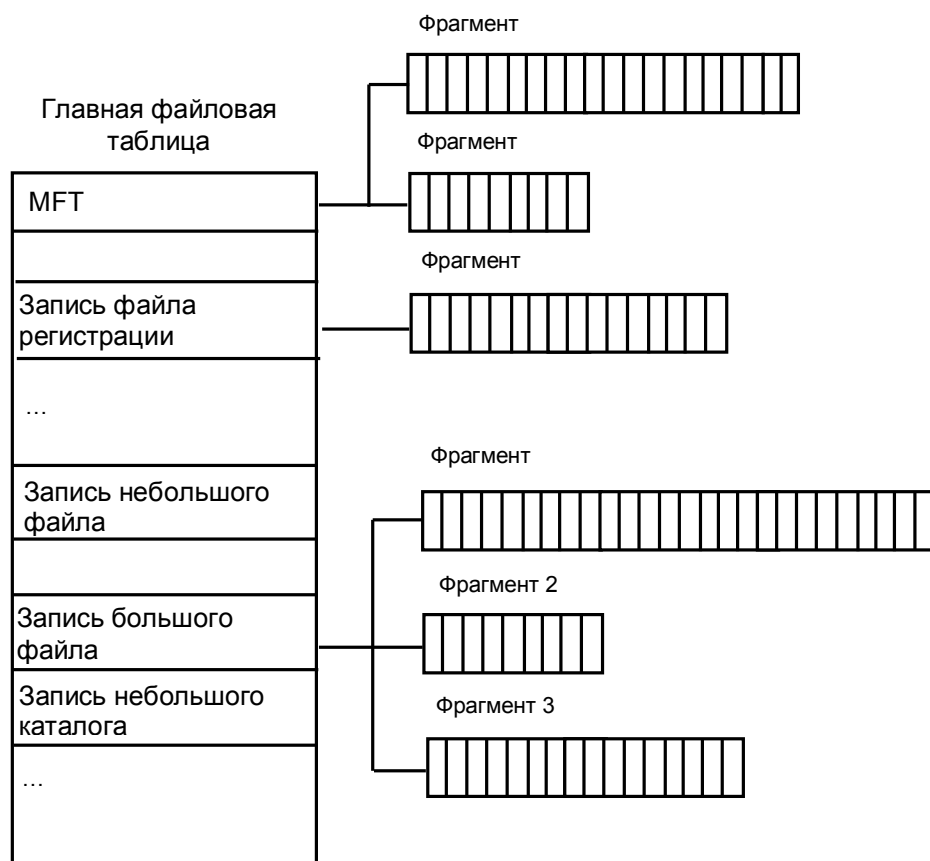


Рис. 5.4. Организация главной файловой таблицы NTFS.

5.4.6. Сравнительный анализ быстродействия FAT и NTFS

Как уже указывалось, фундаментальное свойство любой файловой системы, влияющее на быстродействие всех дисковых операций - структура организации и хранения информации, т.е. то, как, собственно, устроена сама файловая система. Любая файловая система хранит данные файлов в неких объемах - секторах, которые используются аппаратурой и драйвером как самая маленькая единица полезной информации диска. Если файл хранится в сжатом или закодированном виде - как это возможно, к примеру, в системе NTFS - то, конечно, на восстановление или расшифровку информации тратится время и ресурсы процессора. В остальных случаях чтение и запись самих данных файла осуществляется с одинаковой скоростью, какую файловую систему вы бы не использовали.

Остановимся на основных процессах, которые осуществляются системой для доступа к файлам:

Поиск данных файла

Выяснение того, в каких областях диска хранится тот или иной фрагмент файла - процесс, который имеет принципиально разное воплощение в различных файловых системах. Это лишь поиск информации о местоположении файла - доступ к самим данным, фрагментированы они или нет, здесь уже не рассматривается, так как этот процесс совершенно одинаков для всех систем. Речь идет о тех «лишних» действиях, которые приходится выполнять системе перед доступом к реальным данным файлов.

Этот параметр влияет на скорость навигации по файлу (доступ к произвольному фрагменту файла). Он показывает, насколько сильно сама файловая система страдает от фрагментации файлов.

NTFS способна обеспечить быстрый поиск фрагментов, если вся информация хранится в нескольких очень компактных записях (типичный размер - несколько килобайт). Если же файл очень сильно фрагментирован (содержит большое число фрагментов), NTFS придется использовать много записей MFT, что часто заставит хранить их в разных местах. Лишние движения головок при поиске этих данных, в таком случае, приведут к сильному замедлению процесса поиска данных о местоположении файла.

FAT32, из-за большого объема самой таблицы размещения будет испытывать огромные трудности, если фрагменты файла разбросаны по всему диску. Дело в том, что таблица размещения файлов (FAT) представляет собой мини-образ диска, куда включен каждый его кластер. Для доступа к фрагменту файла в системе FAT16 и FAT32 приходится обращаться к соответствующей частичке FAT. Если файл, к примеру, расположен в трех фрагментах - в начале диска, в середине, и в конце - то нам придется обратиться к фрагменту FAT также в его начале, в середине и в конце. В системе FAT16, где максимальный размер области FAT составляет 128 Кбайт, это не составит проблемы - вся область FAT просто хранится в памяти, или же считывается с диска целиком за один проход и буферизируется. FAT32 же, напротив, имеет типичный размер области FAT порядка сотен килобайт, а на больших дисках - даже несколько мегабайт. Если файл расположен в разных частях диска - это вынуждает систему совершать движения головок винчестера столько раз, сколько групп фрагментов в разных областях имеет файл, а это очень и очень сильно замедляет процесс поиска фрагментов файла.

Вывод: абсолютным лидером по этому показателю является FAT16, т.к. эта файловая система никогда не заставит систему делать лишние дисковые операции. Затем идет NTFS - эта система также не требует чтения лишней информации, по крайней мере, до того момента, пока файл имеет разумное число фрагментов. FAT32 испытывает огромные трудности, вплоть до чтения

лишних сотен килобайт из области FAT, если файл разбросан по разным областям диска. Работа с большими по размеру файлами на FAT32 в любом случае сопряжена с огромными трудностями. Системе необходимо понять, в каком месте на диске расположен тот или иной фрагмент файла, Эту операцию можно выполнить лишь изучив всю последовательность кластеров файла с самого начала, обрабатывая за один раз один кластер (через каждые 4 Кбайт файла в типичной системе). Если файл фрагментирован, но лежит компактной кучей фрагментов - FAT32 всё же не испытывает больших трудностей, так как физический доступ к области FAT будет также компактен и буферизован.

Поиск свободного места

Данная операция производится в том случае, если файл нужно создать с нуля или скопировать на диск. Поиск места под физические данные файла зависит от того, как хранится информация о занятых участках диска.

Этот параметр влияет на скорость создания файлов, особенно больших, сохранение или создание в реальном времени больших мультимедийных файлов, копирование больших объемов информации, т.д. Этот параметр показывает, насколько быстро система сможет найти место для записи на диск новых данных, и какие операции ей придется для этого проделать.

Для определения того, свободен ли данный кластер или нет, системы на основе FAT должны просмотреть одну запись FAT, соответствующую этому кластеру. Размер одной записи FAT16 составляет 16 бит, одной записи FAT32 - 32 бита. Для поиска свободного места на диске может потребоваться просмотреть почти всего FAT - это 128 Кбайт (максимум) для FAT16 и до нескольких мегабайт - в FAT32. Для того, чтобы не превращать поиск свободного места в катастрофу (особенно для FAT32), операционной системе приходится идти на различные ухищрения.

NTFS имеет битовую карту свободного места, где одному кластеру соответствует 1 бит. Для поиска свободного места на диске приходится оценивать объемы в десятки раз меньшие, чем в системах FAT и FAT32.

Вывод: NTFS имеет наиболее эффективную систему нахождения свободного места. Стоит отметить, что действовать «в лоб» на FAT16 или FAT32 очень медленно, поэтому для нахождения свободного места в этих системах применяются различные методы оптимизации, в результате чего и там достигается приемлемая скорость. Одно можно сказать наверняка: поиск свободного места при работе в DOS-приложениях с использованием FAT32 – катастрофический по скорости процесс, поскольку в этом случае никакая оптимизация невозможна.

Работа с каталогами и файлами

Каждая файловая система выполняет элементарные операции с файлами - доступ, удаление, создание, перемещение и т.д. Скорость работы этих опера-

ций зависит от принципов организации хранения данных об отдельных файлах и от устройства структур каталогов.

Этот параметр влияет на скорость осуществления любых операций с файлом, в том числе - на скорость любой операции доступа к файлу, особенно - в каталогах с большим числом файлов (тысячи).

FAT16 и FAT32 имеют очень компактные каталоги, размер каждой записи которых предельно мал. Более того, из-за сложившейся исторически системы хранения длинных имен файлов (более 11 символов), в каталогах систем FAT используется не очень эффективная и на первый взгляд неудачная, но зато очень экономная структура хранения этих самих длинных имен файлов. Работа с каталогами FAT производится достаточно быстро, так как в подавляющем числе случаев каталог (файл данных каталога) не фрагментирован и находится на диске в одном месте.

Единственная проблема, которая может существенно понизить скорость работы каталогов FAT - большое количество файлов в одном каталоге (порядка тысячи или более). Система хранения данных - линейный массив - не позволяет организовать эффективный поиск файлов в таком каталоге, и для нахождения файла приходится перебирать большой объем данных, в среднем - половину каталога.

NTFS использует гораздо более эффективный способ адресации - бинарное дерево. Эта организация позволяет эффективно работать с каталогами любого размера - каталогам NTFS не страшно увеличение количества файлов в одном каталоге и до десятков тысяч. С другой стороны, каталог NTFS представляет собой гораздо менее компактную структуру, нежели каталог FAT - это связано с гораздо большим размером одной записи каталога. Данное обстоятельство приводит к тому, что каталоги на томе NTFS в подавляющем числе случаев сильно фрагментированы. Размер типичного каталога FAT укладывается в один кластер, тогда как сотня файлов в каталоге на NTFS уже приводит к размеру файла каталога, превышающему типичный размер одного кластера. Это, в свою очередь, почти гарантирует фрагментацию файла каталога, что, к сожалению, довольно часто сводит на нет все преимущества гораздо более эффективной организации самих данных.

Вывод: структура каталогов на NTFS теоретически гораздо эффективнее, но при размере каталога в несколько сотен файлов это практически не имеет значения. Фрагментация каталогов NTFS, однако, уверенно наступает уже при таком размере каталога. Для малых и средних каталогов NTFS имеет на практике меньшее быстроедействие. Преимущества каталогов NTFS становятся реальными только в том случае, если в одно каталоге присутствуют тысячи файлов - в этом случае быстроедействие компенсирует фрагментацию самого каталога и трудности с физическим обращением к данным (тем более, что эти трудности возникают только при первом обращении к каталогу - далее его содержимое помещается в кэш). Напряженная работа с каталогами, содержа-

щими порядка тысячи и более файлов, проходит на NTFS буквально в несколько раз быстрее, а иногда выигрыш в скорости по сравнению с FAT и FAT32 достигает десятков раз.

Следует отметить, что на практике основной фактор, от которого зависит быстродействие файловой системы - это объем оперативной памяти компьютера. Системы с памятью 64-96 Мбайт - некий рубеж, на котором быстродействие NTFS и FAT32 примерно эквивалентно. Основное преимущество NTFS с точки зрения быстродействия заключается в том, что этой системе безразличны такие параметры, как сложность каталогов (число файлов в одном каталоге), размер диска, фрагментация и т.д. В системах FAT же, напротив, каждый из этих факторов приведет к существенному снижению скорости работы. Только в сложных высокопроизводительных системах - например, на графических станциях или просто на серьезных офисных компьютерах с тысячами документов, или, тем более, на файл-серверах – преимущества структуры NTFS смогут дать реальный выигрыш быстродействия.

Подведем итоги сравнительного анализа исследуемых файловых систем.

NTFS	FAT, FAT32
<i>Ограничения на размеры тома</i>	
Минимальный размер тома составляет приблизительно 10 Мб. С помощью NTFS нельзя форматировать дискеты. На практике рекомендуется создавать тома, размеры которых не превышают 2 Тб.	FAT поддерживает различные размеры томов - от объема дискет и до 4 Гб. FAT32 поддерживает тома объемом от 512 Мб до 2 Тб. Работая под управлением Windows XP, для FAT32 можно отформатировать тома, размер которых не превышает 32 Гб.
<i>Ограничения на размеры файлов</i>	
Теоретически размер файла может составлять 16 экзбайт.	FAT поддерживает файлы размером не более 2 Гб. FAT32 поддерживает файлы размером не более 4 Гб.
<i>Преимущества файловой системы</i>	
Фрагментация файлов не имеет практически никаких последствий для самой файловой системы - работа фрагментированной системы ухудшается только с точки зрения доступа к самим данным файлов.	Для эффективной работы требуется не-много оперативной памяти.

NTFS	FAT, FAT32
Сложность структуры каталогов и число файлов в одном каталоге также не чинит препятствий быстродействию.	Быстрая работа с малыми и средними каталогами.
Быстрый доступ к произвольному фрагменту файла (например, редактирование больших мультимедийных файлов).	Диск совершает в среднем меньшее, в сравнении с NTFS, количество движений головок.
Очень быстрый доступ к маленьким файлам (несколько сотен байт) - весь файл находится в том же месте, где и системные данные (запись MFT).	Эффективная работа на медленных дисках.
<i>Недостатки файловой системы</i>	
Существенные требования к памяти системы (64 Мбайт – абсолютный минимум, лучше иметь больше).	Катастрофическая потеря быстродействия с увеличением фрагментации, особенно для больших дисков (только FAT32).
Медленные диски и контроллеры сильно снижают быстродействие NTFS.	Сложности с произвольным доступом к большим (скажем, 10% и более от размера диска) файлам.
Работа с каталогами средних размеров затруднена тем, что они почти всегда фрагментированы.	Очень медленная работа с каталогами, содержащими большое количество файлов.
Диск, долго работающий в заполненном на 80% - 90% состоянии, будет показывать крайне низкое быстродействие.	

5.4.7. Файловые системы ОС Linux

Операционная система Linux поддерживает несколько типов файловых систем. Основоположителем всех файловых систем, поддерживаемых операционной системой Linux, была система UFS (UNIX File System), разработанная для операционной системы UNIX.

У большей части файловых систем UNIX имеется сходная физическая структура, а их некоторые особенности очень мало различаются. Основными понятиями являются: суперблок, индексный дескриптор (inode), блок данных, блок каталога и косвенный блок.

В суперблоке содержится информация о файловой системе в целом, например, ее размер, максимальное число файлов в разделе, объем свободного пространства и содержится информация о том, где искать незанятые участки (точная информация зависит от типа файловой системы). Информация, хра-

нимая в суперблоке, используется для организации доступа к остальным данным на диске. При запуске ОС суперблок считывается в память и все изменения файловой системы вначале находят отображение в копии суперблока, находящейся в ОП, и записываются на диск только периодически. Это позволяет повысить производительность системы, так как многие пользователи и процессы постоянно обновляют файлы. С другой стороны, при выключении системы суперблок обязательно должен быть записан на диск, что не позволяет выключать компьютер простым выключением питания. В противном случае, при следующей загрузке информация, записанная в суперблоке, окажется не соответствующей реальному состоянию файловой системы.

В индексном дескрипторе хранится вся информация о файле, кроме его имени. В частности, в этом дескрипторе хранятся номера нескольких блоков данных, которые используются для хранения самого файла. В *inode* есть место только для нескольких номеров блоков данных, однако, если требуется большее количество, то пространство для указателей на блоки данных динамически выделяется. Такие блоки называются косвенными. Для того, чтобы найти блок данных, нужно сначала найти его номер в косвенном блоке.

Имя файла хранится в блоке каталога, вместе с номером дескриптора. Каждая запись каталога содержит имя файла и номер индексного дескриптора соответствующего файла.

Как уже было сказано ранее в п. 5.3.2, файловая система UNIX (а операционная система Linux поддерживает соглашения UNIX относительно логической организации файловой системы), в отличие от файловых систем DOS и Windows, является единым деревом. Корень этого дерева — каталог, называемый *root*, и обозначаемый символом */*. Части дерева файловой системы могут физически располагаться в разных разделах разных дисков или вообще на других компьютерах — для пользователя это прозрачно. Процесс присоединения файловой системы раздела к дереву называется монтированием, удаление — размонтированием.

Проблема нарушения целостности файловой системы при некорректном завершении работы в большей или меньшей мере характерна для всех ОС семейства UNIX, и поэтому достаточно давно в них разрабатываются так называемые *журналируемые* файловые системы. Журнал представляет собой файл дисковых операций, в котором, в отличие от обычного *log*-файла, фиксируются не выполненные, а только предстоящие манипуляции с файлами, вследствие чего оказывается возможным самовосстановление целостности файловой системы после сбоя. Следует подчеркнуть, что журналирование направлено на обеспечение целостности файловой системы, но ни в коем случае не гарантирует сохранность пользовательских данных внутри файлов. Так, не следует ожидать, что журналирование восстановит изменения документа, загруженного в текстовый редактор, если они не были сохранены перед сбоем.

Более того, в большинстве журналируемых файловых систем фиксируются грядущие операции только над метаданными изменяемых файлов. Обычно этого достаточно для сохранения целостности файловой системы и, уж во всяком случае, предотвращения долговременных проверок этой системы, однако не предотвращает потери данных в аварийных ситуациях.

В некоторых из файловых систем возможно распространение журналирования и на область данных файла. Однако, как всегда, повышение надежности за счет этого оплачивается снижением быстродействия.

Отметим, что использование журналируемой файловой системы не делает использование программ проверки файловой системы полностью устаревшими. Ошибки в программном и аппаратном обеспечении, которые разрушают случайные блоки файловой системы, как правило, не исправляются за счет использования журнала.

Исторически первой файловой системой для Linux была система *minix*. Она имеет массу недостатков: размера раздела жесткого диска ограничен 64 мегабайтами, а длина имени файла - 30 символами и т.д. Она продолжает использоваться для дискет и RAM-дисков. Еще одной из ранних версий файловой системы для Linux является *ExtFS* (Extended File System), которая представляет собой расширение файловой системы *minix*. В настоящее время эта система заменена на более совершенные (например, на *Ext2FS*) и не используется из-за несовместимости с поздними версиями.

Текущие версии ядра Linux поддерживают в качестве нативных четыре журналируемые файловые системы: *ReiserFS*, *Ext2FS* и *Ext3FS*, специфичные для этой ОС, *XFS* и *JFS* - результаты переноса в Linux файловых систем, разработанных первоначально для рабочих станций под ОС *Irix* (SGI) и *AIX* (IBM), соответственно

Рассмотрим более подробно файловые системы Linux, используемые в настоящее время.

Ext2FS

Ext2FS (Second Extended File System) заменила собой *ExtFS*. В «новой» файловой системе были исправлены некоторые проблемы, а также убраны ограничения. По своей концепции она была предназначена для развития, обеспечивая при этом большую ошибкоустойчивость и хорошую производительность.

По способу организации хранения данных - это типичная представительница файловых систем UNIX. Отличительная особенность - наличие нескольких копий суперблока, что повышает надежность хранения данных. Кроме того, для *Ext2FS* характерен очень эффективный механизм кэширования дисковых операций, что обеспечивает замечательное их быстродействие. Однако при этом следует отметить относительно слабую устойчивость при аварий-

ном завершении работы (например, из-за мертвого зависания или отказа питания).

Ext3FS

Ext3FS (Third Extended File System) является наследником файловой системы *Ext2FS*. Она предлагает технологию журналирования файловой системы, сохраняя при этом структуру *Ext2FS*, что обеспечивает превосходную совместимость. В отличие от *ReiserFS*, которая будет рассмотрена ниже, *Ext3FS* - не более чем журналируемая надстройка над классической *Ext2FS*. Она сохраняет полную совместимость со своей прародительницей, в том числе и на уровне утилит обслуживания. Основным преимуществом *Ext3FS* является максимальная надежность: она является единственной системой из рассматриваемых, в которой возможно журналирование операций не только с метаданными, но и с данными файлов. В *Ext3FS* предусмотрено три режима работы - полное журналирование (full data journalizing), журналирование с отложенной записью (writeback), а также используемое по умолчанию последовательное журналирование (ordered).

Режим полного журналирования распространяется и на метаданные, и на данные файлов. Все их изменения сначала пишутся в файл журнала и только после этого фиксируются на диске. В случае аварийного отказа журнал можно повторно перечитать, приведя данные и метаданные в непротиворечивое состояние. Этот механизм практически гарантирует от потерь данных, однако является наиболее медленным.

В режиме отложенной записи, напротив, в файл журнала записываются только изменения метаданных файлов, подобно всем рассматриваемым файловым системам. То есть никакой гарантии сохранности данных он не предоставляет, однако обеспечивает наибольшее (в рамках *Ext3FS*) быстродействие.

В последовательном режиме физически журналируются только метаданные файлов, однако связанные с ними блоки данных логически группируются в единый модуль, называемый транзакцией. Эти блоки сохраняются перед записью на диск новых метаданных, что способствует сохранности данных, хотя и не может ее полностью гарантировать. Последовательный режим журналирования влечет меньшие накладные расходы по сравнению с полным журналированием, обеспечивая тем самым промежуточный уровень быстродействия.

ReiserFS

ReiserFS - журналируемая файловая система. Она подобна *Ext3FS*, но их внутренняя структура радикально отличается. В *ReiserFS* используется концепция бинарных деревьев, позаимствованная из программного обеспечения баз данных. В *ReiserFS* осуществляется журналирование только операций над

метаданными файлов, что, при определенном снижении надежности, обеспечивает высокую производительность: на большинстве типичных пользовательских задач она лишь незначительно уступает Ext2FS. Более того, на операции копирования большого количества мелких файлов ReiserFS существенно ее опережает.

Кроме этого, ReiserFS обладает возможностью оптимизации дискового пространства, занимаемого мелкими (менее одного блока) файлами. Вторая особенность ReiserFS - то, что т.н. хвосты файлов, то есть их конечные части, меньшие по размеру, чем один блок, могут быть подвергнуты упаковке. Этот режим, называемый *тейлингом* (tailing), включается по умолчанию при создании ReiserFS, обеспечивая в среднем около 5% экономии дискового пространства. Однако это несколько снижает быстродействие, и потому режим тейлинга можно отменить при монтировании файловой системы. Однако упаковка хвостов автоматически восстанавливается после перекомпиляции ядра.

Более серьезная проблема с совместимостью состоит в том, что распространенные загрузчики Linux (и lilo, и GRUB - хотя и по разным причинам) часто не способны загрузить ядро Linux с раздела ReiserFS, оптимизированного в режиме тейлинга. А поскольку этот режим обладает свойством самовосстановления, пользователь может столкнуться с тем, что после перемонтировки ядра система просто откажется загружаться.

XFS

Файловая система XFS, в отличие от молодых ReiserFS и Ext3FS, развивается для фирмой SGI на протяжении почти десяти лет - впервые она появилась в версии Irix 5.3, вышедшей в 1994 году. В Linux она была импортирована лишь относительно недавно. Особенности XFS являются:

- механизм allocation group, то есть деление единого дискового раздела на несколько равных областей, имеющих собственные списки inodes, и несколько свободных блоков для распараллеливания дисковых операций;
- логическое журналирование только изменений метаданных, но с довольно частым сбросом их на диск для минимизации возможных потерь при сбоях;
- механизм delayed allocation - выделение дискового пространства при записи файлов не во время журналирования, а при фактическом сбросе их на диск, что, вместе с повышением производительности, предотвращает фрагментацию дискового раздела;
- списки контроля доступа (ACL, Access Control List) и расширенные атрибуты файлов (extended attributes).

XFS предстает собой очень сбалансированную файловую систему: она почти столь же надежна, как Ext3FS, и не очень уступает ReiserFS в быстродействии на большинстве файловых операций.

Таким образом, каждая из четырех рассмотренных файловых систем имеет свою особенность. Выбор файловой системы должен определяться задачами пользователя и характером преобладающих данных. Кроме этого, если максимально разгрузить корневую файловую систему, выделив в логические тома отдельные ее ветви, возможно и комбинированное использование файловых систем. Иными словами, возможен подбор для каждого из дисковых разделов наиболее подходящей к его содержанию управляющей структуры.

Так, Ext2FS - наиболее подходящий выбор для загрузочного раздела (а при использовании в качестве загрузчика GRUB - это почти обязательное требование). Кроме того, Ext2FS вполне подходит для таких ветвей, как /tmp или /var. Для первой ветви устойчивость к сбоям не критична. Для второй же определяющим требованием является быстродействие. Ext2FS можно применить и для корневой файловой системы - ведь при дробном разбиении диска в корне остается минимум редко изменяемых компонентов.

С другой стороны, корень - наиболее критичный в отношении устойчивости элемент файловой системы. И потому оптимальным для него представляется файловая система Ext3FS, как наиболее устоявшаяся. Кроме того, в экстремальных ситуациях она может быть без проблем смонтирована как Ext2FS. Для разделов типа /usr и /usr/local Ext3fs также видится вполне подходящим вариантом.

Наиболее важная часть файловой системы настольной машины с точки зрения пользователя - его пользовательские данные, то есть каталог /home (ибо систему можно переустановить, а вот потеря данных может быть невозможной). Однако это - и наиболее изменяемая ее часть, что предъявляет высокие требования к быстродействию файловых операций. И поэтому Ext3FS - не лучшее решение для каталога /home, более целесообразно разместить здесь какую-либо из "быстрых" журналируемых файловых систем, ReiserFS или XFS. Выбор между ними определяется предпочтениями пользователя и характером хранимых данных. Быстродействие XFS при работе с файлами большого размера делает ее предпочтительной, если речь идет об обработке изображений, мультимедийных данных, картографической информации и т.д. Преимущества ReiserFS отражаются при работе с файлами малого размера (менее блока файловой системы). XFS ориентирована на работу с мультимедийными приложениями с их огромными потоками данных. Для нее также не отмечалось проблем с совместимостью.

Глава 6. Сетевые операционные системы

В мире существует множество различных операционных систем специального назначения. В рамках данной главы более подробно остановимся на рассмотрении сетевых операционных систем.

Заметим, что на свойства операционной системы непосредственное влияние оказывают аппаратные средства, на которые она ориентирована. По типу аппаратуры различают операционные системы персональных компьютеров, мини-компьютеров, мэйнфреймов, кластеров и сетей ЭВМ. Среди перечисленных типов компьютеров могут встречаться как однопроцессорные варианты, так и многопроцессорные. В любом случае специфика аппаратных средств, как правило, отражается на специфике операционных систем.

6.1. Структура сетевой операционной системы

Сетевая ОС имеет в своем составе средства передачи сообщений между компьютерами по линиям связи, которые совершенно не нужны в автономной ОС. На основе этих сообщений сетевая ОС поддерживает разделение ресурсов компьютера между удаленными пользователями, подключенными к сети. Для поддержания функций передачи сообщений сетевые ОС содержат специальные программные компоненты, реализующие популярные коммуникационные протоколы (например, такие, как IP или IPX).

Сетевая операционная система составляет основу любой вычислительной сети. Каждый компьютер в сети в значительной степени автономен, поэтому под *сетевой операционной системой* в широком смысле понимается совокупность операционных систем отдельных компьютеров, взаимодействующих с целью обмена сообщениями и разделения ресурсов по единым правилам - протоколам. В узком смысле сетевая ОС - это операционная система отдельного компьютера, обеспечивающая ему возможность работать в сети.

В сетевой операционной системе отдельной машины можно выделить несколько частей, показанных на рисунке 6.1:

- Средства управления локальными ресурсами компьютера: функции распределения оперативной памяти между процессами, планирования и диспетчеризации процессов, управления процессорами в мультипроцессорных машинах, управления периферийными устройствами и другие функции управления ресурсами локальных ОС.

- Средства предоставления собственных ресурсов и услуг в общее пользование - серверная часть ОС (сервер). Эти средства обеспечивают, например, блокировку файлов и записей, что необходимо для их совместного использования; ведение справочников имен сетевых ресурсов; обработку за-

просов удаленного доступа к собственной файловой системе и базе данных; управление очередями запросов удаленных пользователей к своим периферийным устройствам.

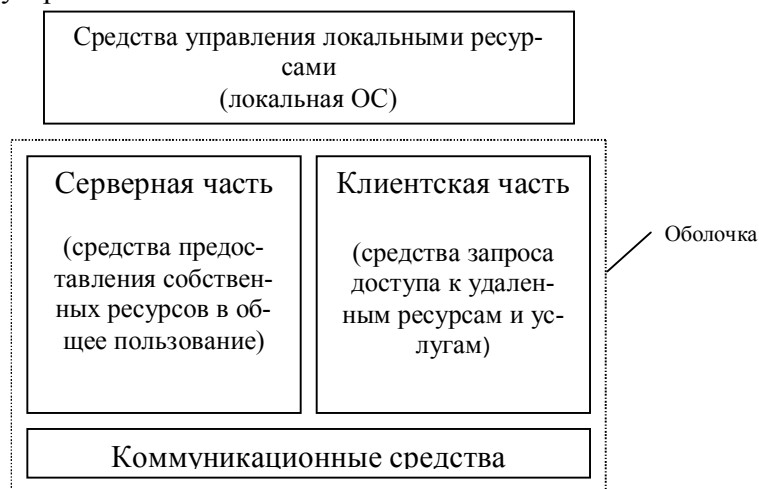


Рис. 6.1. Структура сетевой ОС

- Средства запроса доступа к удаленным ресурсам и услугам и их использования - клиентская часть ОС (*редиректор*). Эта часть выполняет распознавание и перенаправление в сеть запросов к удаленным ресурсам от приложений и пользователей, при этом запрос поступает от приложения в локальной форме, а передается в сеть в другой форме, соответствующей требованиям сервера. Клиентская часть также осуществляет прием ответов от серверов и преобразование их в локальный формат, так что для приложения выполнение локальных и удаленных запросов неразличимо.

- Коммуникационные средства ОС, с помощью которых происходит обмен сообщениями в сети. Эта часть обеспечивает адресацию и буферизацию сообщений, выбор маршрута передачи сообщения по сети, надежность передачи и т.п., то есть является средством транспортировки сообщений.

В зависимости от функций, возлагаемых на конкретный компьютер, в его операционной системе может отсутствовать либо клиентская, либо серверная части.

На рисунке 6.2 представлено взаимодействие сетевых компонентов. Компьютер 1 выполняет роль «чистого» клиента, а компьютер 2 - роль «чистого» сервера. Соответственно на первой машине отсутствует серверная часть, а на второй - клиентская. Отдельно показан компонент клиентской части - редиректор. Именно редиректор перехватывает все запросы, поступающие от приложений, и анализирует их. Если выдан запрос к ресурсу данного компьюте-

ра, то он переадресовывается соответствующей подсистеме локальной ОС, если же это запрос к удаленному ресурсу, то он переправляется в сеть. При этом клиентская часть преобразует запрос из локальной формы в сетевой формат и передает его транспортной подсистеме, которая отвечает за доставку сообщений указанному серверу.

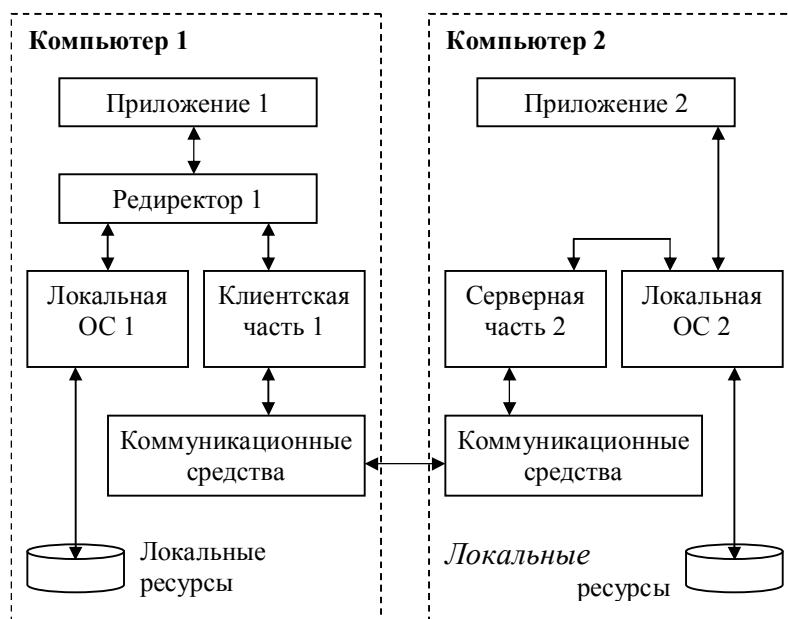


Рис. 6.2. Взаимодействие компонентов операционной системы при взаимодействии компьютеров

Серверная часть операционной системы компьютера 2 принимает запрос, преобразует его и передает для выполнения своей локальной ОС. После того, как результат получен, сервер обращается к транспортной подсистеме и направляет ответ клиенту, выдавшему запрос. Клиентская часть преобразует результат в соответствующий формат и адресует его тому приложению, которое выдало запрос.

На практике сложилось несколько подходов к построению сетевых операционных систем, что иллюстрирует рисунок 6.3. Первые сетевые ОС представляли собой совокупность существующей локальной ОС и надстроенной над ней *сетевой оболочки*. При этом в локальную ОС встраивался минимум сетевых функций, необходимых для работы сетевой оболочки, которая выполняла основные сетевые функции. Примером такого подхода является использование на каждой машине сети операционной системы MS DOS (у которой начиная с ее третьей версии появились такие встроенные функции, как блокировка файлов и записей, необходимые для совместного доступа к фай-

лам). Принцип построения сетевых ОС в виде сетевой оболочки над локальной ОС используется и в современных ОС (например, Personal Ware). Однако более эффективным представляется путь разработки операционных систем, изначально предназначенных для работы в сети. Сетевые функции у ОС такого типа глубоко *встроены* в основные модули системы, что обеспечивает их логическую стройность, простоту эксплуатации и модификации, а также высокую производительность.

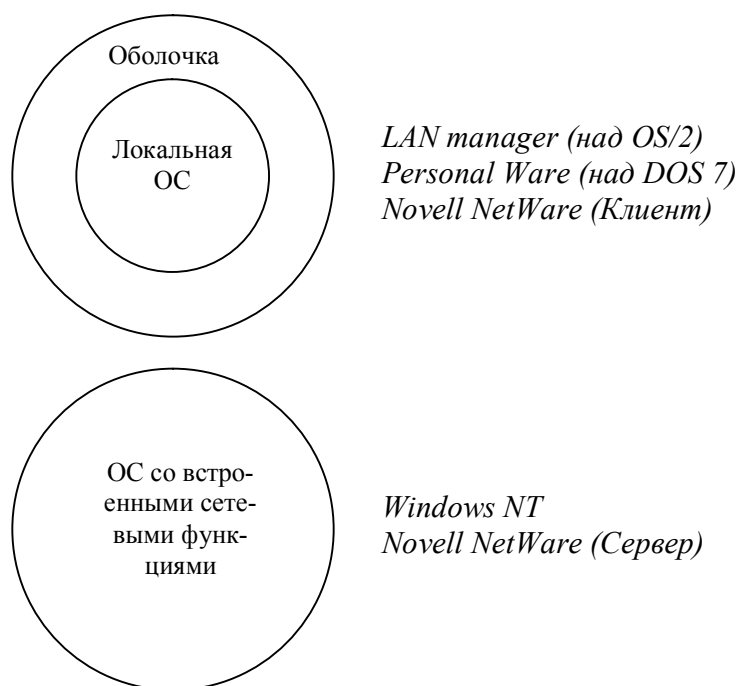


Рис. 6.3. Варианты построения сетевых ОС

Специфика ОС проявляется и в том, каким образом она реализует сетевые функции: распознавание и перенаправление в сеть запросов к удаленным ресурсам, передача сообщений по сети, выполнение удаленных запросов. При реализации сетевых функций возникает комплекс задач, связанных с распределенным характером хранения и обработки данных в сети: ведение справочной информации о доступных в сети ресурсах и серверах, адресация взаимодействующих процессов, обеспечение прозрачности доступа, тиражирование данных, согласование копий, поддержка безопасности данных.

6.2. Одноранговые сетевые ОС и ОС с выделенными серверами

В зависимости от того, как распределены функции между компьютерами сети, сетевые операционные системы, а следовательно, и сети делятся на два класса: одноранговые и двухранговые. Двухранговые сетевые ОС чаще называют сетями с выделенными серверами. Работа одноранговых и двухранговых сетей показана на рисунке 6.4

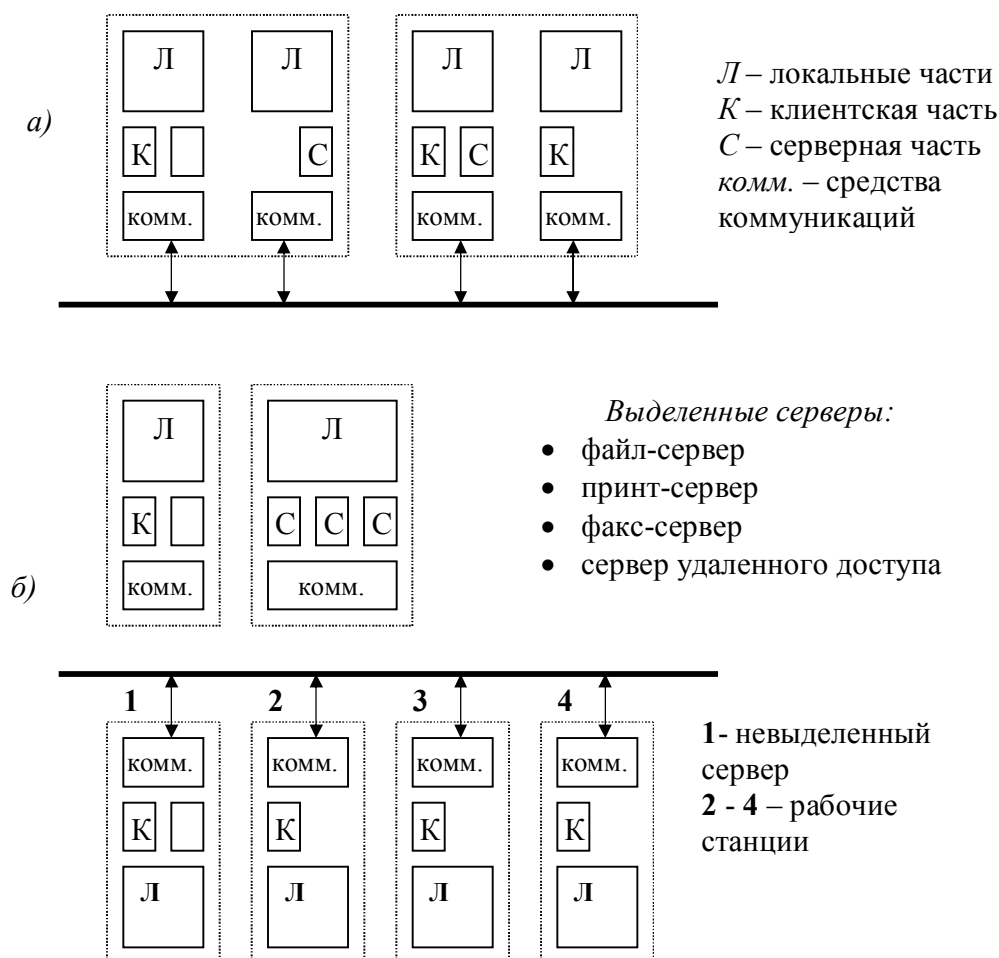


Рис. 6.4. Схема одноранговой (а) и двухранговой (б) сети.

Если компьютер предоставляет свои ресурсы другим пользователям сети, то он играет роль сервера. При этом компьютер, обращающийся к ресурсам другой машины, является клиентом. Компьютер, работающий в сети, может выполнять функции либо клиента, либо сервера, либо совмещать обе эти функции.

Если выполнение каких-либо серверных функций является основным назначением компьютера (например, предоставление файлов в общее пользование всем остальным пользователям сети или организация совместного использования факса, или предоставление всем пользователям сети возможности запуска на данном компьютере своих приложений), то такой компьютер называется выделенным сервером. Заметим, что на выделенных серверах желательно устанавливать ОС, оптимизированные для выполнения тех или иных серверных функций. Поэтому в сетях с выделенными серверами чаще всего используются сетевые операционные системы, в состав которых входит нескольких вариантов ОС, отличающихся возможностями серверных частей. Например, сетевая ОС Novell NetWare имеет серверный вариант, оптимизированный для работы в качестве файл-сервера, а также варианты оболочек для рабочих станций с различными локальными ОС, причем эти оболочки выполняют исключительно функции клиента. Другим примером ОС, ориентированной на построение сети с выделенным сервером, является операционная система Windows NT. В отличие от NetWare, оба варианта данной сетевой ОС - Windows NT Server (для выделенного сервера) и Windows NT Workstation (для рабочей станции) - могут поддерживать функции и клиента и сервера. Но серверный вариант Windows NT имеет больше возможностей для предоставления ресурсов своего компьютера другим пользователям сети, так как может выполнять более широкий набор функций, поддерживает большее количество одновременных соединений с клиентами, реализует централизованное управление сетью, имеет более развитые средства защиты.

Выделенный сервер не принято использовать в качестве компьютера для выполнения текущих задач, не связанных с его основным назначением, так как это может уменьшить производительность его работы как сервера. В связи с такими соображениями в ОС Novell NetWare на серверной части возможность выполнения обычных прикладных программ вообще не предусмотрена, то есть сервер не содержит клиентской части, а на рабочих станциях отсутствуют серверные компоненты. Однако в других сетевых ОС функционирование на выделенном сервере клиентской части вполне возможно. Например, под управлением Windows NT Server могут запускаться обычные программы локального пользователя, которые могут потребовать выполнения клиентских функций ОС при появлении запросов к ресурсам других компьютеров сети. При этом рабочие станции, на которых установлена ОС Windows NT Workstation, могут выполнять функции невыделенного сервера. Несмотря на то, что в сети с выделенным сервером все компьютеры в общем случае могут выполнять одновременно роли и сервера, и клиента, эта сеть функционально не симметрична: аппаратно и программно в ней реализованы два типа компьютеров - одни, в большей степени ориентированные на выполнение серверных функций и работающие под управлением специализированных серверных ОС, а другие - в основном выполняющие клиентские функции и рабо-

тающие под управлением соответствующего этому назначению варианта ОС. Функциональная несимметричность, как правило, вызывает и несимметричность аппаратуры - для выделенных серверов используются более мощные компьютеры с большими объемами оперативной и внешней памяти. Таким образом, функциональная несимметричность в сетях с выделенным сервером сопровождается несимметричностью операционных систем (специализация ОС) и аппаратной несимметричностью (специализация компьютеров).

В одноранговых сетях все компьютеры равны в правах доступа к ресурсам друг друга. Каждый пользователь может объявить какой-либо ресурс своего компьютера разделяемым, после чего другие пользователи могут его эксплуатировать. В таких сетях на всех компьютерах устанавливается одна и та же ОС, которая предоставляет всем компьютерам в сети *потенциально* равные возможности. Одноранговые сети могут быть построены, например, на базе ОС Windows NT Workstation. В одноранговых сетях также может возникнуть функциональная несимметричность: одни пользователи не желают разделять свои ресурсы с другими, и в таком случае их компьютеры выполняют роль клиента; за другими компьютерами администратор закрепил только функции по организации совместного использования ресурсов, а значит они являются серверами; в третьем случае, когда локальный пользователь не возражает против использования его ресурсов и сам не исключает возможности обращения к другим компьютерам, ОС, устанавливаемая на его компьютере, должна включать и серверную, и клиентскую части. При этом в одноранговых сетях отсутствует специализация ОС в зависимости от преобладающей функциональной направленности - клиента или сервера.

Глава 7. Проблемы безопасности операционных систем

Особую важность для операционных систем приобретают вопросы безопасности данных. С одной стороны, в компьютерной сети объективно существует больше возможностей для несанкционированного доступа - из-за децентрализации данных и большой распределенности «законных» точек доступа, из-за большого числа пользователей, благонадежность которых трудно установить, а также из-за большого числа возможных точек несанкционированного подключения к сети. С другой стороны, корпоративные приложения работают с данными, которые имеют большое значение для успешной работы организации в целом. И для защиты таких данных в корпоративных сетях наряду с различными аппаратными средствами используется весь спектр средств защиты, предоставляемый операционной системой: избирательные или мандатные права доступа, сложные процедуры аутентификации пользователей, программная шифрация.

Говоря об информационной безопасности, иногда различают два термина *protection* и *security*. Термин *security* используется для рассмотрения проблемы в целом, а *protection* для рассмотрения специфичных механизмов ОС, сохраняющих информацию в компьютере.

7.1. Классификация угроз

Все то, что представляет собой опасность для нормальной работы системы (в данном случае операционной системы и программ, которые работают под ее управлением) принято называть *угрозами*. Следует различать *неумышленные* и *умышленные* угрозы.

Неумышленные угрозы связаны со следующими факторами:

- ошибками оборудования или математического обеспечения (сбои процессора, питания, нечитаемые дискеты, ошибки в коммуникациях, ошибки в программах);
- ошибками человека (некорректный ввод, неправильная монтировка дисков или лент, запуск неправильных программ, потеря дисков или лент);
- форс-мажорными обстоятельствами.

Умышленные угрозы, в отличие от случайных, преследуют цель нанесения ущерба пользователям ОС и, в свою очередь, подразделяются на *активные* и *пассивные*. Пассивная угроза состоит в несанкционированном доступе к информации без изменения состояния системы, активная – в несанкционированном изменении системы. Для описания умышленных угроз часто используется термин «атака» или «нападение».

Целью атак является:

- получение несанкционированного доступа, т.е. нарушение секретности или конфиденциальности;
- выдача себя за другого пользователя, с целью снятия с себя ответственности или использования его полномочий или привилегий;
- отказ от факта формирования или получения информации;
- утверждение о получении некоторой информации от другого пользователя, хотя в действительности она сформирована самим нарушителем;
- утверждение о получении или посылке информации в определенный момент времени, хотя информация посылалась в другое время или не посылалась вообще.
- незаконное расширение своих или изменение чужих полномочий;
- модификация (нарушение целостности) программного обеспечения, как правило, внесением в него новых функций, в том числе распространение или внедрение в систему вирусов;
- изучение того, кто к какой информации имеет доступ, путем анализа трафика каналов связи, структуры базы данных, структуры программного обеспечения.

Можно выделить следующие типы угроз:

- незаконное проникновение под видом легального пользователя;
- нарушение функционирования системы с помощью программ-вирусов или программ-червей;
- нелегальные действия легального пользователя.

Следует отметить, что типизация угроз не слишком строгая. Так, А.Таненбаум приводит список наиболее успешных атак на ОС:

- попытки чтения страниц памяти, дисков и лент, которые сохранили информацию предыдущего пользователя;
- попытки выполнения нелегальных системных вызовов, или системных вызовов с нелегальными параметрами,
- внедрение программы, которая выводит на экран приглашение к регистрации пользователя. Многие легальные пользователи, видя такое, начинают пытаться входить в систему, и их попытки могут протоколироваться (например, вариант троянского коня);
- попытки торпедировать программу проверки входа в систему путем многократного нажатия клавиш del, esc и т.д. В некоторых системах проверочная программа погибает, и вход в систему становится возможным;
- подкуп персонала;
- использование закладных элементов (дыр), специально оставленных дизайнерами системы.

7.2. Классы информационной безопасности

Существует ряд основополагающих документов, в которых регламентированы основные подходы к проблеме информационной безопасности. Среди них можно выделить:

- документы Республики Беларусь (например, Закон Республики Беларусь «О научно-технической информации» от 5 мая 1999 года);
- документы Гостехкомиссии при Президенте Российской Федерации;
- «оранжевая» (по цвету обложки) книга министерства обороны США;
- гармонизированные критерии европейских стран;
- рекомендации X.800 по защите распределенных систем.

Эти основополагающие документы открыли путь к ранжированию информационных систем по степени надежности. Так, «оранжевая» книга определяет иерархию 4 уровней безопасности, каждый последующий из которых строже предыдущего:

D: минимальная безопасность;

C: дискреционная защита;

B: мандатная защита;

A: верифицируемая защита.

Уровни **C** и **B** подразделяются на классы (**C1, C2, B1, B2, B3**): чем больше номер, тем больше степень безопасности. Так, уровень **C2** обеспечивает большую безопасность, чем **C1**. По мере перехода от уровня **D** до **A** к надежности систем предъявляются все более жесткие требования.

Например, для большинства версий UNIX стандартным является уровень безопасности **C2**: управляемая защита доступа с требованием возможностей ревизии, защиты паролем, других средств контроля владения и использования ресурсов, строгого тестирования и документирования. Возможно повышение уровня защиты (например, до уровня **B1** или выше) за счет дополнительных компонент, приобретаемых отдельно.

Чтобы система в результате процедуры сертификации могла быть отнесена к некоторому классу, ее политика безопасности и гарантированность должны удовлетворять оговоренным требованиям.

В качестве примера рассмотрим требования класса **C2**, которому удовлетворяют некоторые популярные ОС (Windows NT, отдельные реализации UNIX и др.):

- Каждый пользователь должен быть идентифицирован уникальным входным именем и паролем для входа в систему. Система должна быть в состоянии использовать эти уникальные идентификаторы, чтобы следить за действиями пользователя.

- Операционная система должна защищать объекты от повторного использования.
- Владелец ресурса (например, файла) должен иметь возможность контролировать доступ к этому ресурсу.
- Системный администратор должен иметь возможность учета всех событий, относящихся к безопасности.
- Система должна защищать себя от внешнего влияния или навязывания, такого как модификация загруженной системы или системных файлов, хранимых на диске.

7.3. Анализ некоторых ОС с точки зрения их защищенности

Операционная система должна способствовать реализации мер безопасности или прямо поддерживать их. Примеры подобных решений в рамках аппаратуры и операционной системы - разделение команд по уровням привилегированности, защита различных процессов от взаимного влияния за счет выделения каждому своего виртуального пространства, особая защита ядра ОС, контроль над повторным использованием объекта. Большое значение имеет структура файловой системы. Например, в ОС с дискреционным контролем доступа каждый файл должен храниться вместе с дискреционным списком прав доступа к нему, а при выполнении, например, копирования файла вместе с телом файла должны быть автоматически скопированы все его атрибуты.

В принципе, меры безопасности не обязательно должны быть заранее встроены в ОС - достаточно принципиальной возможности дополнительной установки защитных продуктов. Так, «ненадежная» система MS-DOS может быть улучшена за счет средств проверки паролей доступа к компьютеру и/или жесткому диску, за счет борьбы с вирусами путем отслеживания попыток записи в загрузочный сектор CMOS-средствами и т.п. Тем не менее, по-настоящему надежная система должна изначально проектироваться с акцентом на механизмы безопасности.

Среди архитектурных решений, с точки зрения информационной безопасности, целесообразно выделить следующие:

- деление аппаратных и системных функций по уровням привилегированности и контроль обмена информацией между уровнями;
- защита различных процессов от взаимного влияния за счет механизма виртуальной памяти;
- наличие средств управления доступом;
- структурированность системы, явное выделение надежной вычислительной базы, обеспечение компактности этой базы;

- следование принципу минимизации привилегий - каждому компоненту дается ровно столько привилегий, сколько необходимо для выполнения им своих функций;
- сегментация (в частности, сегментация адресного пространства процессов) как средство повышения надежности компонентов.

Рассмотрим некоторые известные операционные системы с точки зрения их защищенности.

MS-DOS

Операционная система MS-DOS функционирует в реальном режиме (real-mode) процессора i80x86. В ней невозможно выполнение требования, касающегося изоляции программных модулей (отсутствует аппаратная защита памяти). Уязвимым местом для защиты является также файловая система FAT, не предполагающая в файлах наличие атрибутов, связанных с разграничением доступа к ним. Таким образом, MS-DOS, не будучи защищенной, находится на самом нижнем уровне в иерархии защищенных ОС.

UNIX

В операционной системе UNIX существует несколько уязвимых с точки зрения безопасности мест, вытекающих из самой природы UNIX и открывающих двери для нападения. Однако хорошее системное администрирование может ограничить эту уязвимость. Существуют противоречивые сведения относительно защищенности UNIX. В UNIX изначально были заложены идентификация пользователей и разграничение доступа. Как оказалось, средства защиты данных в UNIX могут быть доработаны, и сегодня можно утверждать, что многие клоны UNIX по всем параметрам соответствуют классу безопасности **C2**.

Обычно, говоря о защищенности UNIX, рассматривают защищенность автоматизированных систем, одним из компонентов которых является UNIX-сервер. Безопасность такой системы увязывается с защитой глобальных и локальных сетей, безопасностью удаленных сервисов и аутентификацией в сетевой конфигурации. На системном уровне важно наличие средств идентификации и аудита.

В UNIX существует список именованных пользователей, в соответствии с которым может быть построена система разграничения доступа. Все пользователи, которым разрешена работа в системе, учитываются в файле пользователей `/etc/passwd`. Группы пользователей учитываются в файле `/etc/group`. Каждому пользователю назначается целочисленный идентификатор и пароль. Когда пользователь входит в систему и предъявляет свое имя (процедура login), отыскивается запись в учетном файле `/etc/passwd`. В этой записи имеются такие поля, как имя пользователя, имя группы, к которой принадлежит

данный пользователь, целочисленный идентификатор пользователя, целочисленный идентификатор группы, зашифрованный пароль пользователя.

Разработчики ОС UNIX считают, что информация, нуждающаяся в защите, находится главным образом в файлах. Как уже указывалось ранее, в главе 5, по отношению к конкретному файлу все пользователи делятся на три категории: владелец файла, члены группы владельца и прочие пользователи. Для каждой из этих категорий режим доступа определяет права на операции с файлом, а именно: право на чтение; право на запись; право на выполнение (для каталогов - право на поиск).

Таким образом, безопасность ОС UNIX может быть доведена до соответствия классу C2. Однако разработка на ее основе автоматизированных систем более высокого класса защищенности может быть сопряжена с большими трудозатратами.

Windows NT/2000

С момента выхода версии 3.1 осенью 1993 года в Windows NT гарантировалось соответствие уровню безопасности C2. В 1999 году сертифицирована версия NT 4 с Service pack 6a с использованием файловой системы NTFS в автономной и сетевой конфигурации. Следует заметить, что этот уровень безопасности не подразумевает защиту информации, передаваемой по сети, и не гарантирует защищенности от физического доступа. Компоненты защиты NT частично встроены в ядро, а частично реализуются подсистемой защиты. Подсистема защиты регистрирует правила контроля доступа и контролирует учетную информацию. Кроме того, Windows NT имеет определенные встроенные средства, например, поддержку резервных копий данных и управление источниками бесперебойного питания, которые не требуются «оранжевой» книгой, но в целом повышают общий уровень безопасности.

Windows NT - ОС, которая была спроектирована для поддержки разнообразных защитных механизмов: от минимальных до удовлетворяющих требованиям C2. Система защиты ОС Windows NT состоит из следующих компонентов:

- процедуры регистрации (Logon Processes), которые обрабатывают запросы пользователей на вход в систему. Они включают в себя начальную интерактивную процедуру, которая отображает начальный диалог с пользователем на экране и удаленные процедуры входа, которые позволяют удаленным пользователям получить доступ с рабочей станции сети к серверным процессам Windows NT.
- подсистемы локальной авторизации (Local Security Authority, LSA), которая гарантирует, что пользователь имеет разрешение на доступ в систему. Эта компонента - центральная для системы защиты Windows NT. Она порождает маркеры доступа, управляет локальной политикой безо-

пасности и предоставляет интерактивным пользователям аутентификационные услуги. LSA также контролирует политику аудита и ведет журнал, в котором сохраняются аудитные сообщения, порождаемые диспетчером доступа.

- менеджера учета (Security Account Manager, SAM), который управляет базой данных учета пользователей. Эта база данных содержит информацию обо всех пользователях и группах пользователей. SAM предоставляет услуги по легализации пользователей, которые используются в LSA.
- диспетчера доступа (Security Reference Monitor, SRM), который проверяет, имеет ли пользователь право на доступ к объекту и на выполнение тех действий, которые он пытается совершить с объектом. Эта компонента проводит в жизнь легализацию доступа и политику аудита, определяемые LSA. Она предоставляет услуги для программ супервизорного и пользовательского режимов для того, чтобы гарантировать, что пользователи и процессы, осуществляющие попытки доступа к объекту, имеют необходимые права. Эта компонента также порождает аудитные сообщения, когда это необходимо.

Ключевая цель системы защиты Windows NT - мониторинг и контроль того, кто и к каким объектам осуществляет доступ. Система защиты хранит информацию, относящуюся к безопасности для каждого пользователя, группы пользователей и объекта. Она может идентифицировать попытки доступа, которые производятся прямо пользователем или непрямо программой или другим процессом, инициированным пользователем. Windows NT также отслеживает и контролирует доступ к тем объектам, которые пользователь может видеть посредством пользовательского интерфейса (такие как файлы и принтеры), а также к объектам, которые пользователь не может видеть (таким, как именованные каналы).

7.4. Основные понятия криптографии

Многие службы информационной безопасности, например, контроль входа в систему, разграничение доступа к ресурсам, обеспечение безопасного хранения данных, опираются на использование криптографических алгоритмов. Поэтому мы сочли уместным в рамках настоящей книги кратко изложить основные понятия, связанные с шифрованием данных.

К настоящему времени сложилась определенная терминология, которой в дальнейшем мы будем придерживаться. Криптология, как наука о шифровании, подразделяется на две части: криптография и криптоанализ.

Криптограф пытается найти методы обеспечения секретности и/или подлинности сообщений. Криптоаналитик пытается решить обратную задачу - раскрывать шифры или подделывать кодированные сигналы, чтобы они воспринимались как подлинные. На практике трудно сказать, чья работа сложнее, но в «узких кругах» широко распространено мнение, что только хороший криптоаналитик, имеющий большой опыт в «раскалывании» «чужих» шифров, может разработать хороший, т.е. устойчивый к различным видам атак новый шифр.

Современная криптография включает в себя следующие крупные разделы:

- симметричные криптосистемы;
- криптосистемы с открытым ключом;
- системы электронной подписи;
- управление ключами.

Основными направлениями использования криптографических методов являются передача конфиденциальной информации по каналам связи (например, электронная почта), установление подлинности передаваемых сообщений, хранение информации (документов, баз данных) на носителях в зашифрованном виде.

Шифрование – это процесс изменения цифрового сообщения из открытого текста (plain text) в зашифрованный текст (cipher text) таким образом, чтобы его могли прочитать только те стороны, для которых он предназначен, либо для проверки подлинности отправителя (аутентификация), либо для гарантии того, что отправитель действительно послал данное сообщение. В алгоритмах шифрования предусматривается наличие *секретного ключа* и принято правило Кирхгофа: стойкость шифра должна определяться только секретностью ключа.

Процесс шифрования данных часто иллюстрируют с помощью функции $f(x) = y$. Другими словами, если мы применяем функцию f к значению x , тогда получаем значение y . Хотя подобное сравнение выглядит довольно простым, оно точно описывает процесс шифрования данных. Все виды шифрования состоят из трех фундаментальных элементов: нечто *тайное*, что нуждается в защите, *алгоритм*, обычно являющийся математической операцией и *ключ*, который делает все эти вещи возможными.

Строго говоря, шифр - это множество всех возможных криптографических преобразований (их число равно числу всех возможных ключей), взаимно однозначно отображающих множество всех открытых текстов в множество всех зашифрованных текстов (шифротекстов) и обратно. Принято считать, что шифры делятся на *блочные* и *поточковые*. Если в первом случае открытый текст разбивается на блоки фиксированной длины, с которыми в дальнейшем оперирует шифр, то во втором открытый текст рассматривается как битовый поток и работает шифр с битами.

Для того чтобы две стороны, участвующие в передаче данных могли шифровать и расшифровывать данные, они должны иметь соглашение по поводу двух важных фрагментов информации - используемого алгоритма и ключа.

7.4.1. Ключи, используемые для шифрования данных

Стойкость шифрования основана на стойкости ключей, используемых для шифрования данных. Стойкость ключа определяется двумя характеристиками – длиной и непредсказуемостью (случайностью). Чем длиннее и случайнее ключ, тем сложнее его вычислить. Многие пакеты программного обеспечения объявляют, что они поддерживают 40-битные или 128-битные технологии шифрования. Это утверждение соотносится с длиной ключа, используемого в процессах шифрования. Шифрование должно обеспечиваться невозможность извлечения данных без знания ключа. Кроме того, не должно существовать способов вычисления ключа из зашифрованных данных.

Существует два основных способа, с помощью которых взламываются системы шифрования – *грубая сила* (brute force) и *статистический анализ*. Атаки типа brute force являются самыми простыми по реализации, но требуют больших затрат времени. Они сводятся к попыткам перебрать все возможные ключи шифрования до тех пор, пока данные не будут расшифрованы. По существу, это - отгадывание значения ключа шифрования.

Атаки, основанные на статистическом анализе, используют недостатки в реализации и использовании ключей. Эти недостатки могут быть связаны с генерацией, хранением или передачей ключей. Другая форма статистического анализа заключается в поиске закономерностей в зашифрованных данных. Если этот документ был зашифрован, тогда решение кроется в сравнении шифрованного текста с известной алфавитной статистикой. Например, хорошо известно, что наиболее часто используемой буквой в английском алфавите является буква “е”. Пытаясь сопоставить наиболее часто встречаемые символы, появляющиеся в шифрованном тексте, с наиболее часто встречаемыми буквами алфавита, злоумышленник формирует основу для статистического анализа.

Числа инициализации – это числа, используемые для того, чтобы удостовериться в истинной произвольности при генерировании ключей. Во многих случаях знание алгоритма генерирования случайного числа может привести к тому, что злоумышленник определяет случайные числа, а затем и сам ключ. Число инициализации является переменной, вводимой в систему генерирования случайных чисел таким образом, что даже детальное знание алгоритма генерирования недостаточно для определения случайных значений без знания значения числа инициализации.

Привязки применяются к шифрованному тексту. Добавление привязок приводит к трансформации шифрованного текста еще один раз и является очень недорогой операцией, обычно использующей алгоритмы логических

операций XOR или AND. Эта операция маскирует закономерности в зашифрованном тексте, делая статистический анализ невозможным до тех пор, пока функция привязки не будет отменена. Если злоумышленник не знает, что имело место добавление привязок, которыми были снабжены кусочки зашифрованного текста и значение привязок, тогда попытки расшифровки текста становятся более трудновыполнимыми.

Когда расшифровка уже выполнена, для завершения процесса потребуется логическое решение. Важно заметить, что расшифровка никогда не может провалиться, поскольку функция $f(x) = y$ вычисляется однозначно. Успех расшифровки основан на значении y , являющемся результатом выполнения функции $f(x)$.

Криптоаналитические системы часто используют очень сложные системы логического сравнения закономерностей, пытаясь определить, когда расшифровка является успешной, используя представления о том, как *должен* выглядеть незашифрованный текст. Кроме прочего, можно еще больше усложнить процесс расшифровки, используя идею *времени жизни* (lifetime). Попытки расшифровки могут провалиться, потому что не успели расшифровать секретные данные до того, как они стали неверными. В реальной жизни системы расшифровки имеют дело с секундами или минутами для определения секретных данных или ключа, поскольку многие реализации шифрования повторно генерируют ключи после того, как истек определенный период времени или было передано определенное количество данных. Чтобы еще больше усложнить систему, обмен новыми ключами может проводиться с использованием специального ключа, отличного от ключа, используемого для данных.

7.4.2. Алгоритмы шифрования: шифрование с симметричным ключом

Как уже отмечалось, алгоритм шифрования является математической функцией $f(x)$, принимающей значение x и возвращающей результат y . Современные функции шифрования являются очень сложными. Некоторые из алгоритмов являются общедоступными, в то время как другие – конфиденциальны. Примерами наиболее часто используемых доступных алгоритмов являются RC4 и различные варианты DES (DES, 3DES, DESx). Для того чтобы алгоритм шифрования стал обоснованно строгим, он:

- не должен допускать определения скрываемых данных без знания ключа;
- не должен допускать вычисления ключа на основе анализа зашифрованных данных.

Шифрование с симметричным ключом основано на концепции *закрытого ключа* (private key). Имеется один ключ, который используется как для шифрования, так и для расшифровки сообщения. При этом подходе существует

две стороны, поддерживающие ключ, являющийся для них общим и известный только им. Этот ключ обеспечивает аутентификацию и конфиденциальность.

Симметричный алгоритм существует в двух основных форматах – *потока* (stream) и *блока* (block). Потокое шифрование не шифрует данные напрямую при помощи ключа. Напротив, оно использует ключ, как основу для генерирования ключевого потока (keystream). Генерирование ключевого потока является ресурсоемкой операцией и приводит к существенной загрузке процессора при потоковом шифровании. Для каждого бита незашифрованного текста существует один соответствующий бит в ключевом потоке. Незашифрованный текст шифруется бит за битом с помощью таких операций как XOR (исключающее ИЛИ), вычисление которых не требует особых затрат. Протокол RC4 (Rivest cipher 4) является наиболее распространенным протоколом потокового шифрования, поскольку является шифрованием по умолчанию, используемой для технологии, называемой Microsoft Point to Point Encryption (MPPE). MPPE имеет множество применений на платформе Windows. Блочное шифрование с другой стороны напрямую использует ключ для шифрования данных. Незашифрованные данные делятся на блоки, размер которых напрямую связан с размером ключа. Каждый блок затем шифруется с помощью ключа для получения шифрованного текста. DES (Data Encryption Standard). Наиболее популярные алгоритмы шифрования с секретным ключом: DES, TripleDES, ГОСТ и др.

Другим применением симметричных методов шифрования является шифрование с помощью *односторонней* функции, называемой также хеш-функцией или дайджест-функцией. Нарушение целостности данных может произойти при несанкционированной модификации. В технических терминах, несанкционированная модификация - это изменение, выходящее за пределы нормальных операций компьютера. Например, изменение электронного адреса в ходе его передачи или переполнение буфера, вызывающее изменение данных в оперативной памяти. И хотя невозможно предотвратить эти изменения, существует возможность зарегистрировать их. Это достигается с помощью функции, называемой *односторонней хэш-функцией* (one-way hash). Односторонняя хеш-функция концептуально похожа на функцию *четности* (parity). Вычисление хеш-функции осуществляется перед тем, как они передаются или сохраняются, и результаты вычисления сохраняются или передаются вместе с данными. Перед повторным использованием данных, вычисление осуществляется еще раз и новые результаты сравниваются с сохраненными. Если результаты совпадают, данные считаются не изменившимися и, следовательно, целостность данных была сохранена. Алгоритмы хеширования похожи на алгоритмы шифрования, но между ними также существует четкое различие. В то время как секретный ключ используется для того, чтобы сде-

лать результат алгоритма шифрования *непредсказуемым*, алгоритм хеширования должен быть на 100% *предсказуемым*. Если он не будет предсказуемым, то его нельзя будет *воспроизвести* и, следовательно, результат окажется непроверяемым. Алгоритм хеширования работает, принимая значение документа в качестве входного параметра. Затем алгоритм сортирует документ множество раз, перемещая данные из документа при каждой итерации. Результатом является строка фиксированной длины, полученная из этого документа. Хеш-алгоритмы очень чувствительны к изменениям, и поэтому изменение даже одного бита в исходном документе приведет к изменению около 50% битов в хеше. Кроме того, *невозможно* извлечь сам документ из хеша. Это происходит потому, что хэш является неполным документом, поскольку в нем отсутствуют кусочки данных.

Двумя наиболее часто используемыми алгоритмами хеширования являются MD5 (Message Digest 5) и SHA-1 (Secure Hash Algorithm-1). MD5 производит 128-битный хэш из любого потока данных, в то время как SHA-1 производит 160-битный хэш. Чем длиннее хэш, тем большую безопасность он обеспечивает и поэтому SHA-1 - чаще используемый алгоритм.

Процесс хеширования может быть представлен в следующем виде:

$$\text{Хэш}\{\text{Данные}\} = y$$

Затем данные передаются в виде *Данные+y*.

Когда данные получены, значение *y* извлекается и целостность определяется следующим образом:

if $\text{Хэш}\{\text{Полученные Данные}\} = y$ **then** данные верны

Хеширование может привести к еще одной крупной уязвимости в обеспечении безопасности. Существует возможность перехвата пакета *Данные+y*, изменения данных, повторного вычисления значения *y* и передачи измененных данных назначенному получателю. Этот получатель не будет иметь возможности убедиться, что полученные им данные фактически являются отправленными данными. Поэтому, для того, чтобы хеширование обеспечивало эффективность технологии целостности данных, сам хэш должен быть защищен.

Итак, применение хэш-функции к шифруемым данным позволяет сформировать небольшой дайджест из нескольких байт, по которому невозможно восстановить исходный текст. Получатель сообщения может проверить целостность данных, сравнивая полученный вместе с сообщением дайджест с повторно вычисленным при помощи той же односторонней функции дайджестом. Эта техника активно используется для контроля входа в систему. Например, пароли пользователей хранятся на диске в зашифрованном односторонней функцией виде.

7.4.3. Шифрование с открытым ключом

В системах шифрования с *открытым* или *асимметричным* ключом (public/ asymmetric key) используется два ключа. Один из ключей, называемый открытым или несекретным, используется для шифрования сообщений, которые могут быть расшифрованы только с помощью секретного ключа, имеющегося у получателя, для которого предназначено сообщение. Возможна и обратная ситуация: для шифрования сообщения может использоваться секретный ключ и если сообщение можно расшифровать с помощью открытого ключа, то подлинность отправителя будет гарантирована (система электронной подписи). Этот принцип изобретен Уитфилдом Диффи и Мартином Хеллманом в 1976 г.

Шифрование с открытым ключом основано на следующих четырех правилах:

1. Каждый участник переговоров имеет закрытый ключ, неизвестный другим участникам.
2. Каждый участник имеет открытый ключ, который сообщается другим участникам.
3. Закрытый и открытый ключи связаны между собой математически.
4. Невозможно определить один ключ, зная другой.

Схема работы механизма шифрования с асимметричным ключом показана на рисунке 7.1.

Фактически *закрытый ключ* – это не единственное число, а набор из нескольких чисел, являющихся результатом сложного вычисления с использованием случайных исходных чисел. Во многих случаях это означает, что ключам не разрешается покидать компьютер, на котором они были сгенерированы. Поскольку закрытый ключ известен только одной стороне - он обеспечивает подлинность. Управление ключами и, в частности, хранение ключей играют жизненную роль в обеспечении безопасности закрытого ключа. Асимметричное шифрование является эффективным только в том случае, если существует уверенность, что закрытый ключ доступен только назначенному пользователю. В Windows и других операционных системах, закрытые ключи хранятся как часть профиля пользователя, защищенного его паролем. Более безопасные реализации хранения закрытых ключей основаны на использовании смарт-карт или биометрическом доступе к ключам.

Открытый ключ, напротив, доступен каждому. Поскольку открытый ключ математически связан только с одним закрытым ключом - только эти два ключа взаимосвязаны. Операции, выполняемые с использованием некоторого закрытого ключа, связаны только с одним открытым ключом.

Правила шифрования с использованием открытого ключа следующие:

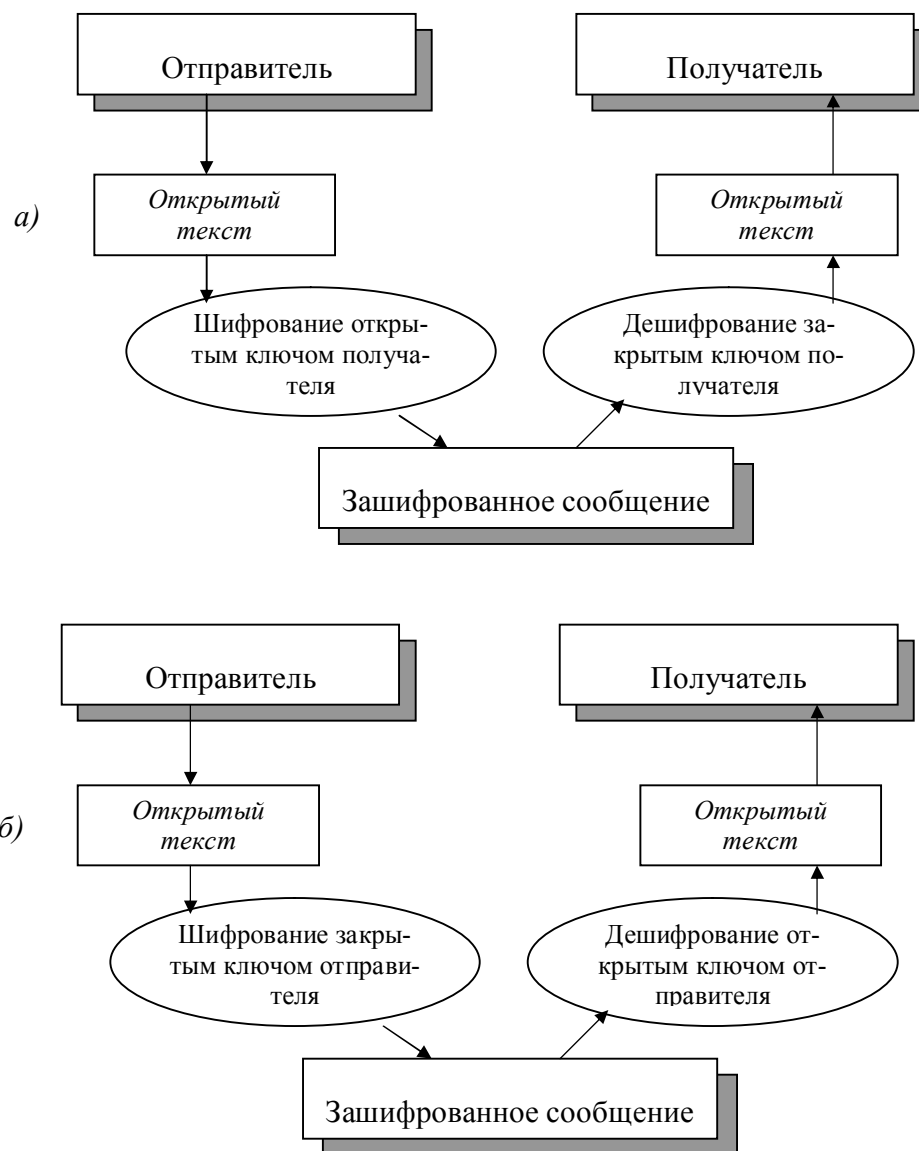


Рис. 7.1. Передача сообщений с открытыми ключами: а) – передача конфиденциального сообщения; б) – проверка электронной подписи.

$$Ш\{\text{открытый ключ, данные}\} = P\{\text{закрытый ключ, данные}\}$$

$$Ш\{\text{закрытый ключ, данные}\} = P\{\text{открытый ключ, данные}\}$$

Иными словами, данные, зашифрованные с помощью ключа одного типа, не могут быть расшифрованы с помощью этого же ключа. Вместо этого они должны расшифровываться с помощью соответствующего ему ключа.

Рассмотрим пример работы системы с открытыми ключами. Пусть, например, фирма С объявляет о тендере на покупку какого-либо товара, на который могут откликнуться несколько продавцов. Поскольку в момент объявления тендера неизвестно, кто из поставщиков представит свои предложения, фирма С публикует открытый ключ для работы с тендером. Все предложения продавцов шифруются этим открытым ключом и, следовательно могут быть прочитаны только объявителем тендера, но не конкурентами.

В процессе проведения тендера может возникнуть необходимость в конфиденциальных переговорах между фирмой С и некоторыми продавцами. Поэтому каждый участник тендера вместе с предложениями сообщает объявителю собственный открытый ключ. Фирма С шифрует конфиденциальные сообщения, посылаемые одному из продавцов, его открытым ключом, сопровождая это сообщение своей электронной подписью (зашифрованной, естественно, закрытым ключом фирмы С). Такой способ шифрования гарантирует, что сообщение будет прочитано только получателем, а также дает возможность получателю проверить подлинность отправителя. Сообщения от одного из продавцов для объявителя тендера шифруются аналогичным образом.

Наконец, сообщения о результатах тендера шифруются закрытым ключом фирмы С и, следовательно, могут быть прочитаны всеми участниками. При этом гарантируется, что сообщение отправлено объявителем тендера, а не подделано недобросовестными конкурентами.

Шифрование с открытым ключом является одним из средств безопасности веб-сервера, работающего по протоколу SSL. Рассмотрим пример шифрования с открытым ключом, в котором используются два дополнительных ключа, *закрытый* и *открытый*. Шифрование выполняется следующим образом:

- браузер пользователя устанавливает защищенную связь по протоколу HTTPS с веб-сервером;
- браузер пользователя и сервер вступают в диалог, чтобы определить уровень шифрования, который должен использоваться для защиты подключений;
- веб-сервер отправляет браузеру свой открытый ключ;
- браузер шифрует сведения, используемые при создании ключа сеанса, с помощью открытого ключа, и отправляет их на сервер;
- с помощью закрытого ключа сервер расшифровывает сообщение, создает ключ сеанса, шифрует его с помощью открытого ключа и отправляет обозревателю;
- ключ сеанса используется как сервером, так и браузером для шифрования и расшифровывания передаваемых данных.

Использование открытых ключей снимает проблему обмена и хранения ключей, свойственную системам с симметричными ключами. Открытые ключи могут храниться публично, и каждый может послать зашифрованное открытым ключом сообщение владельцу ключа. Тогда как расшифровать это сообщение может только владелец открытого ключа, и никто другой, при помощи своего секретного ключа. Однако алгоритмы с симметричным ключом более эффективны, поэтому во многих криптографических системах используются оба метода.

Широко известная система PGP (Pretty Good Privacy) представляет собой систему шифрования с двумя ключами: асимметричным (открытым) и обычным симметричным алгоритмом с секретным ключом. С первого взгляда PGP ведет себя как обычный алгоритм с открытым ключом. Однако это не так. В криптосистемах с открытым ключом генерируется, с помощью специального математического алгоритма, два ключа - один открытый, и один закрытый. Сообщения шифруются с помощью одного ключа, а дешифруются с помощью другого. Например, вы делаете свой открытый ключ доступным другим, чтобы вам могли присылать сообщения, зашифрованные этим ключом, и дешифруете их с помощью секретного ключа. То есть дешифровать сообщение с помощью открытого ключа невозможно. PGP использует для шифрования алгоритм с открытым ключом RSA в паре с обычным методом шифрования IDEA. В методе IDEA используется алгоритм с одним ключом для шифрования и дешифрования. PGP шифрует текст с помощью ключа IDEA, а сам ключ IDEA шифруется открытым ключом адресата методом RSA. Адресат, приняв сообщение, дешифрует ключ IDEA (своим секретным ключом), а далее ключом IDEA расшифровывает сам текст сообщения.

7.4.4. Алгоритм RSA: пример несимметричного шифрования

Среди несимметричных алгоритмов наиболее известен алгоритм RSA, предложенный Рональдом Ривестом, Ади Шамиром и Леонардом Эдмандом. Рассмотрим его далее более подробно.

Задача, положенная в основу данного метода, состоит в том, чтобы найти такую функцию $y=f(x)$, чтобы получение обратной функции $x=f^{-1}(y)$, было бы в общем случае NP-полной задачей. Однако, если знать некоторую секретную информацию, то сделать это существенно проще. Такие функции также называют односторонними функциями с потайным ходом. Например, получить произведение двух чисел $n=p*q$ просто, а разложить n на множители, если p и q достаточно большие простые числа, сложно.

Первым шагом в алгоритме RSA является генерация ключей. Для этого необходимо:

1. Выбрать два очень больших простых числа p и q .
2. Вычислить произведение $n=p*q$.

3. Выбрать большое случайное число d , не имеющее общих сомножителей с числом $(p-1)*(q-1)$.
4. Определить число e так, чтобы выполнялось следующее условие $(e*d) \bmod ((p-1)*(q-1))=1$.

Тогда открытым ключом будет совокупность чисел e и n , а секретным ключом - чисел d и n .

Теперь, чтобы зашифровать данные по известному ключу $\{e,n\}$, необходимо сделать следующее:

1. Разбить шифруемый текст на блоки, где i -й блок может быть представлен в виде числа $m(i)=0,1,2,..., n-1$. Их должно быть не более $n-1$.
2. Зашифровать текст, рассматриваемый как последовательность чисел $m(i)$ по формуле $c(i)=(m(i)e) \bmod n$.

Чтобы расшифровать эти данные, используя секретный ключ $\{d,n\}$, необходимо вычислить: $m(i) = (c(i)^d) \bmod n$. В результате будет получено множество чисел $m(i)$, которые представляют собой часть исходного текста.

В качестве примера зашифруем и расшифруем сообщение «АБВ», которое представим, как последовательность чисел 123 (в диапазоне 0-32)

- Выбираем $p=5$ и $q=11$ (числа на самом деле должны быть большими).
- Находим $n=5*11=55$
- Определяем $(p-1)*(q-1) = 40$. Тогда d будет равно, например 7.
- Выберем e , исходя из $(e*7) \bmod 40=1$. Например, $e=3$.

Теперь зашифруем сообщение, используя открытый ключ $\{3,55\}$

- $C1 = (13) \bmod 55 = 1$
- $C2 = (23) \bmod 55 = 8$
- $C3 = (33) \bmod 55 = 27$

Теперь расшифруем эти данные, используя закрытый ключ $\{7,55\}$.

- $M1 = (17) \bmod 55 = 1$
- $M2 = (87) \bmod 55 = 2097152 \bmod 55 = 2$
- $M3 = (277) \bmod 55 = 10460353203 \bmod 55 = 3$

В результате применения алгоритма все данные будут расшифрованы.

7.4.5. Сравнение методов шифрования

Итак, мы рассмотрели три основных метода шифрования – симметричное шифрование, асимметричное шифрование и хеширование. Каждый из этих методов может быть применен во всех современных системах обеспечения безопасности. Краткие результаты рассмотрения приведены в следующей таблице.

Метод шифрования	Основное применение	Преимущества	Недостатки
Симметричное шифрование	Групповое шифрование больших объемов данных	Скорость и безопасность	Обмен ключами
Асимметричное шифрование	Аутентификация сторон	Безопасность – отсутствие обмена ключами	Очень медленный, большие накладные расходы
Хеширование	Обеспечение целостности данных	Стабильная технология, высокая степень безопасности	Хэш может стать предметом подмены, что делает его применение бесполезным

Следует отметить, что ни одна из этих технологий в одиночку не является полностью безопасной и не может обеспечить целостного решения для безопасного процесса передачи информации. Для уменьшения риска, связанного с каждой из этих технологий, мы должны использовать их вместе. Решение вопросов безопасности операционных систем обусловлено их архитектурными особенностями и связано с правильной организацией аутентификации, авторизации и аудита.

7.5. Шифрование хранимых данных

В предыдущих параграфах данной главы обсуждались аспекты обеспечения безопасности при обмене сообщениями. Однако проблема защиты информации этим не ограничивается. Необходимо исключить также несанкционированный доступ к хранящимся на внешних носителях данным.

На персональном компьютере операционную систему можно загрузить не с жесткого, а с гибкого диска. Это позволяет обойти проблемы, связанные с отказом жесткого диска и разрушением загрузочных разделов. Однако, поскольку с помощью гибкого диска можно загружать различные операционные системы, любой пользователь, получивший физический доступ к компьютеру, может обойти встроенную систему управления доступом файловой системы NTFS и с помощью определенных инструментов прочесть информацию жесткого диска. Многие конфигурации оборудования позволяют применять пароли, регулирующие доступ при загрузке. Однако такие средства не имеют

широкого распространения. Кроме того, если на компьютере работает несколько пользователей, подобный подход не дает хороших результатов, да и сама защита с помощью пароля недостаточно надежна. Вот типичные примеры несанкционированного доступа к данным:

- хищение переносного компьютера;
- неограниченный доступ: компьютер оставлен в рабочем состоянии, и за ним никто не наблюдает. Любой пользователь может подойти к такому компьютеру и получить доступ к конфиденциальной информации.

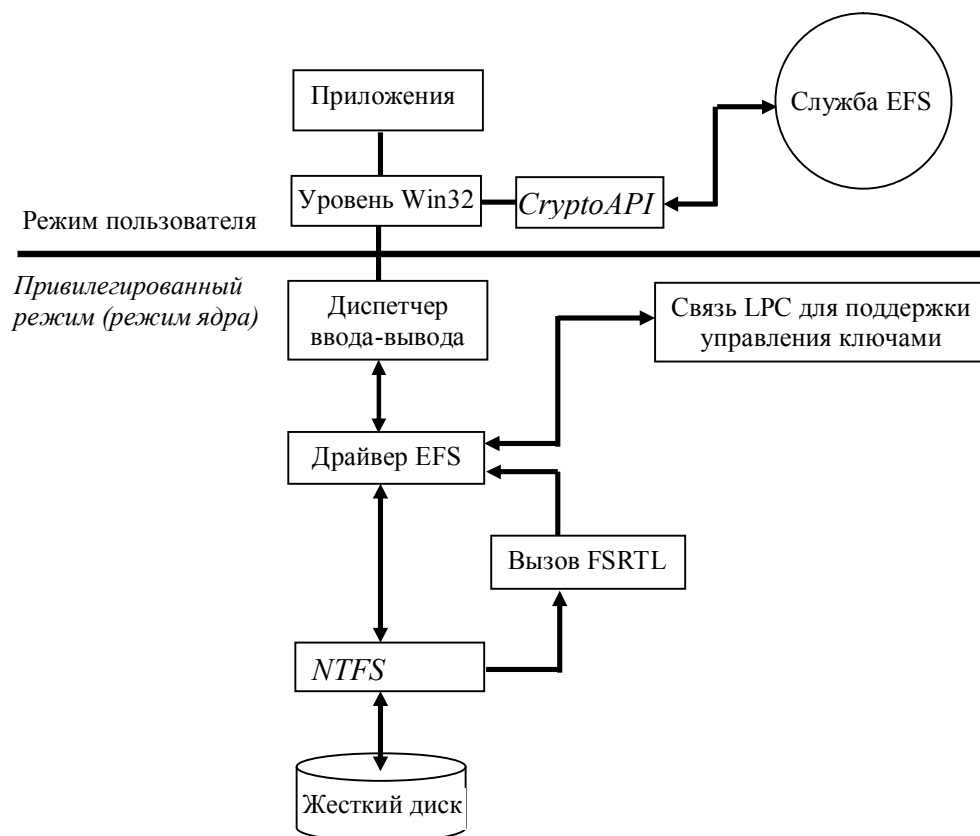


Рис. 7.2. Взаимодействие компонент EFS

Основной целью создания системы безопасности является защита конфиденциальной информации, которая обычно находится в незащищенных файлах на жестком диске, от несанкционированного доступа. Доступ к данным можно ограничить с помощью средств NTFS. Такой подход обеспечивает хорошую степень защиты, если:

- единственной загружаемой операционной системой является Windows, построенная на NT-технологии;

- жесткий диск не может быть физически удален из компьютера;
- и, наконец, данные находятся в разделе NTFS.

Если кто-либо захочет получить доступ к данным, он может осуществить свое желание, получив физический доступ к компьютеру или жесткому диску этого компьютера.

Единственный надежный способ защиты информации - шифрующая файловая система. На рынке программного обеспечения существует целый набор продуктов, обеспечивающих шифрование данных с помощью образованного от пароля ключа на уровне приложений. Однако такой подход имеет ряд ограничений:

- ручное шифрование и дешифрование;
- утечка информации из временных файлов и файлов подкачки;
- слабая криптостойкость ключей.

Все вышеперечисленные проблемы позволяет решить *шифрующая файловая система* (Encrypting File System, EFS), реализованная в Windows 2000. Далее коротко рассмотрим технологию шифрования, место шифрования в операционной системе.

EFS содержит следующие компоненты операционной системы Windows 2000 (см. рисунок 7.2):

- *Драйвер EFS*. Драйвер EFS является надстройкой над файловой системой NTFS. Он обменивается данными со службой EFS, т.е. запрашивает ключи шифрования, наборы DDF (Data Decryption Field) и DRF (Data Recovery Field), а также обменивается с другими службами управления ключами. Полученную информацию драйвер EFS передает библиотеке реального времени файловой системы EFS (File System Run Time Library, FSRTL), которая выполняет различные операции, характерные для файловой системы;
- *Библиотека реального времени файловой системы EFS*. FSRTL - это модуль, находящийся внутри драйвера EFS, реализующий вызовы NTFS, выполняющие такие операции, как чтение, запись и открытие зашифрованных файлов и каталогов, а также операции, связанные с шифрованием, дешифрованием и восстановлением файлов при их чтении или записи на диск. Хотя драйверы EFS и FSRTL реализованы в виде одного компонента, они никогда не обмениваются данными напрямую. Для передачи сообщений друг другу они используют механизм вызовов NTFS, предназначенный для управления файлами. Это гарантирует, что вся работа с файлами происходит при непосредственном участии NTFS. С помощью механизма управления файлами операции записи значений атрибутов EFS (DDF и DRF) реализованы как обычная модификация атрибутов файла. Передача ключа шифрования выполняется так, чтобы он мог быть установлен в контексте открытого файла.

Затем контекст файла используется для автоматического выполнения операций шифрования и дешифрования при записи и чтении информации файла;

- *Служба EFS.* Служба EFS (EFS Service) является частью системы безопасности операционной системы. Для обмена данными с драйвером EFS она использует порт связи LPC, существующий между локальным администратором безопасности (Local Security Authority, LSA) и монитором безопасности, работающим в привилегированном режиме. Она также поддерживает набор API для Win32;
- *Набор функций API для Win32.* Этот набор интерфейсов прикладного программирования позволяет выполнять шифрование файлов, дешифрование и восстановление зашифрованных файлов, а также их импорт и экспорт (без предварительного дешифрования). Эти API поддерживаются стандартным системным модулем DLL (advapi32).

EFS основана на шифровании с открытым ключом и использует все возможности архитектуры CryptoAPI в Windows 2000. Каждый файл шифруется с помощью случайно сгенерированного ключа, зависящего от пары открытого и личного, закрытого ключей пользователя. Подобный подход в значительной степени затрудняет осуществление большого набора атак, основанных на криптоанализе. При криптозащите файлов может быть применен любой алгоритм симметричного шифрования.

В EFS для шифрования и дешифрования информации используются открытые ключи. Данные зашифровываются с помощью симметричного алгоритма с применением *ключа шифрования файла* (File Encryption Key, FEK). FEK - это сгенерированный случайным образом ключ, имеющий определенную длину. В свою очередь, FEK шифруется с помощью одного или *нескольких* открытых ключей, предназначенных для криптозащиты ключа. В этом случае создается *список* зашифрованных ключей FEK, что позволяет организовать доступ к файлу со стороны нескольких пользователей. Для шифрования набора FEK используется открытая часть пары ключей каждого пользователя. Список зашифрованных ключей FEK хранится вместе с зашифрованным файлом в специальном атрибуте EFS, называемом *полем дешифрования данных* (Data Decryption Field, DDF). Информация, требуемая для дешифрования, привязывается к самому файлу. Секретная часть ключа пользователя используется при дешифровании FEK. Она хранится в безопасном месте, например на смарт-карте или другом устройстве, обладающем высокой степенью защищенности.

Шифрование данных производится в следующем порядке:

- Незашифрованный файл пользователя шифруется с помощью сгенерированного случайным образом ключа шифрования файла, FEK;

- FEK шифруется с помощью открытой части пары ключей пользователя и помещается в поле дешифрования данных, DDF;
- FEK шифруется с помощью открытой части ключа восстановления и помещается в поле восстановления данных, DRF.

Дешифрование данных производится следующим образом:

- Из DDF извлекается зашифрованный FEK и дешифруется с помощью секретной части ключа пользователя;
- Зашифрованный файл пользователя дешифруется с помощью FEK, полученного на предыдущем этапе.

При работе с большими файлами дешифруются только отдельные блоки, что значительно ускоряет выполнение операций чтения

Как уже говорилось, EFS тесно взаимодействует с NTFS. Временные файлы, создаваемые приложениями, наследуют атрибуты оригинальных файлов (если файлы находятся в разделе NTFS). Вместе с файлом шифруются также и его временные копии. EFS находится в ядре Windows 2000 и использует для хранения ключей специальный пул, не выгружаемый на жесткий диск. Поэтому ключи никогда не попадают в файл подкачки.

ЛИТЕРАТУРА

1. Таненбаум А. Современные операционные системы. – СПб.: Издательский дом «Питер», 2004.
2. Ахо В., Хопкрофт Д., Ульман Д. Структуры данных и алгоритмы. - М.: Издательский дом «Вильямс», 2001.
3. Блэк У. Интернет: протоколы безопасности. Учебный курс. – СПб.: Издательский дом «Питер», 2001.
4. Дейтел Г. Введение в операционные системы. - М.: «Мир», 1987.
5. Коффрон Дж. Технические средства микропроцессорных систем. - М.: «Мир», 1983.
6. Олифер В.Г., Олифер Н.А.. Сетевые операционные системы. - СПб.: Издательский дом «Питер», 2003.
7. Робачевский А. Операционная система UNIX. - СПб.: «BNV-Петербург», 1999.
8. Цикритис Д., Бернштейн Ф. Операционные системы. М.: «Мир», 1977.
9. Хевиленд К., Грэй Д., Салама Б. Системное программирование в UNIX. – М.: «ДМК Пресс», 2000.
10. Столлинкс В. Операционные системы. – М.: Издательский дом «Вильямс», 2002.
11. Костромин В.А. Самоучитель Linux для пользователя. - СПб.: «BNV-Петербург», 2003.
12. Митчел М., Оулдем Д., Самьюэл А. – Программирование для Linux. Профессиональный подход. - М.: Издательский дом «Вильямс», 2003.
13. Рихтер Дж. - Windows для профессионалов: создание эффективных WIN32 приложений с учетом специфики 64-х разрядной версии Windows. - СПб.: Издательский дом «Питер», 2001.
14. Вильямс А. Системное программирование в Windows 2000 для профессионалов. - СПб.: Издательский дом «Питер», 2001.
15. Холзнер С. Visual C++ :учебный курс. - СПб.: Издательский дом «Питер», 2000.

16. Липский В. - Комбинаторика для программистов. - М.: «Мир», 1988.
17. Иртегов Д.В. – Введение в операционные системы. - СПб.: «БХВ-Петербург», 2002.
18. Кирх О. - Linux для профессионалов. Руководство администратора сети. - СПб.: Издательский дом «Питер», 2000.
19. Гостехкомиссия России. Автоматизированные системы. Защита от несанкционированного доступа к информации. Классификация автоматизированных систем и требования по защите информации. - Руководящий документ - М.: 1992.
20. Гостехкомиссия России. Средства вычислительной техники. Защита от несанкционированного доступа к информации. Показатели защищенности от НСД к информации. - Руководящий документ. - М.: 1992.
21. R.L. Rivest, A. Shamir, and L.M. Adleman, "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems," Communications of the ACM, v. 21, No. 2, Feb 1978, pp. 120-126.

Ресурсы Internet

1. <http://www.opennet.ru>
2. <http://www.linux.yaroslavl.ru>
3. <http://www.linux.ru>
4. <http://www.linuxrsp.ru>
5. <http://www.linuxcenter.ru>
6. <http://www.citforum.kture.edu.ua>
7. <http://www.osp.ru>
8. <http://www.softerra.ru>
9. <http://www.citforum.ru>
10. <http://www.mtu-net.ru>
11. <http://www.woweb.ru>
12. <http://www.lowlevel.ru>
13. <http://www.sdteam.com>
14. <http://www.parallel.ru>
15. <http://www.sparc.spb.su>
16. <http://www.uspu.ru>
17. <http://www.ipm.kstu.ru>
18. <http://www.ergeal.ru>
19. <http://www.computers.refportal.ru>
20. <http://www.helpis.nm.ru>

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	1
ГЛАВА 1. ОСНОВНЫЕ ПОНЯТИЯ.....	5
1.1. Определение и структура операционных систем.....	5
1.2. Классификация операционных систем	7
1.3. Эволюция операционных систем.....	12
1.4. Основные функции операционных систем	15
ГЛАВА 2. ПРОЦЕССЫ И РЕСУРСЫ.....	17
2.1. Основные определения.....	17
2.2. Прерывания	25
2.3. Алгоритмы планирования процессов.....	28
2.4. Вытесняющие и невытесняющие алгоритмы планирования	30
2.5. Взаимодействующие процессы.....	32
2.6. Треды.....	34
2.7. Управление процессами и тредами в операционной системе UNIX	38
ГЛАВА 3. ВЗАИМОДЕЙСТВИЕ И СИНХРОНИЗАЦИЯ ВЫЧИСЛИТЕЛЬНЫХ ПРОЦЕССОВ	41
3.1. Взаимодействие процессов, состояние гонки и взаимоисключения	41
3.2. Критические ресурсы и критические секции.....	43
3.3. Реализация взаимного исключения для конкурирующих процессов	50
3.4. Взаимодействие сотрудничающих процессов.....	61
3.5. Тупики	66
ГЛАВА 4. УПРАВЛЕНИЕ ЦЕНТРАЛЬНЫМ ПРОЦЕССОРОМ И ПАМЯТЬЮ	76
4.1. Общие положения	76
4.2. Управление временем центрального процессора	78
4.3. Основные концепции управления памятью	80
4.4. Механизмы распределения памяти в мультипрограммных системах	87
4.5. Иерархия запоминающих устройств. Принцип кэширования данных... ..	95
4.6. Аппаратные средства процессора x86 для управления памятью	98
4.7. Распределение оперативной памяти в различных операционных системах.....	104
ГЛАВА 5. УПРАВЛЕНИЕ ВНЕШНИМИ УСТРОЙСТВАМИ И ОРГАНИЗАЦИЯ ВВОДА-ВЫВОДА.....	108
5.1. Физическая организация устройств ввода-вывода	108

5.2. Основные понятия и концепции организации ввода-вывода.....	109
5.3. Логическая организация данных.....	112
5.4. Физическая организация данных на магнитных дисках	116
ГЛАВА 6. СЕТЕВЫЕ ОПЕРАЦИОННЫЕ СИСТЕМЫ	135
6.1. Структура сетевой операционной системы.....	135
6.2. Одноранговые сетевые ОС и ОС с выделенными серверами	139
ГЛАВА 7. ПРОБЛЕМЫ БЕЗОПАСНОСТИ ОПЕРАЦИОННЫХ СИСТЕМ.....	142
7.1. Классификация угроз.....	142
7.2. Классы информационной безопасности	144
7.3. Анализ некоторых ОС с точки зрения их защищенности	145
7.4. Основные понятия криптографии.....	148
7.5. Шифрование хранимых данных.....	159
ЛИТЕРАТУРА	164

Учебное издание

Безверхий Александр Анатольевич
Кашкевич Сергей Иванович

ВВЕДЕНИЕ В ОПЕРАЦИОННЫЕ СИСТЕМЫ

Макет и верстка Безверхий А.А., Кашкевич С.И.
Дизайн обложки Мисюченко Н.И.

Подписано в печать . Формат 60X84 1/16
Бумага офсетная. Гарнитура Таймс.
Печать офсетная. Усл. печ. л. , . Уч.-изд. л. .
Тираж 160 экз. Заказ 308

Издатель и полиграфическое исполнение: Республиканское
унитарное предприятие «Информационно-вычислительный
центр Министерства финансов Республики Беларусь»
Лицензия ЛВ № 469 от 20.11.2000.
Лицензия ЛП № 45 от 10.10.2002.
220004, г. Минск, ул. Кальварийская, 17